

CS146S: The Modern Software Developer

Stanford CS Fall 2025 — 繁中講義 + 50 篇 Reading 摘要

繁中譯解: WLK

Contents

1	前言	17
2	課程簡介與自學路線圖	19
2.1	為什麼這門課值得念	19
2.2	為什麼非資工背景也能念	19
2.3	課程資訊(完整)	19
2.3.1	修課負擔	19
2.3.2	評分組成	19
2.3.3	旁聽政策	19
2.4	三條自學路線 (從 00_index.md 重點展開)	20
2.4.1	Track A – Vibe Coder 速成路線	20
2.4.2	Track B – 完整軟體工程路線	20
2.4.3	Track C – Tech Lead / PM 視角	20
2.5	學習資源	20
2.5.1	必備工具 (免費可開始)	20
2.5.2	學習成本最低的兩篇 prereq	20
2.6	本講義的譯解原則	21
2.7	講義產出限制 / 已知 caveat	21

I

10 週講義

3	Week 1: Introduction to Coding LLMs and AI Development . .	25
3.1	學習目標	25
3.2	核心概念導讀	25
3.2.1	一、LLM 的「製造流程」決定它的能力與盲點	25
3.2.2	二、Prompt Engineering 是「給 contractor 的 brief」	26
3.2.3	三、Coding LLM 的真實工業使用樣貌	26
3.3	Monday Lecture (9/22): Introduction and how an LLM is made	26
3.4	Friday Lecture (9/26): Power prompting for LLMs	27
3.5	Reading 摘要	28
3.6	Assignment: LLM Prompting Playground	28
3.7	對 Vibe Coder 的應用	29
4	Week 2: The Anatomy of Coding Agents	31
4.1	學習目標	31
4.2	核心概念導讀	31
4.2.1	一、Agent ≠ LLM + 一個 while loop	31
4.2.2	二、Tool use / Function calling 的底層機制	31

4.2.3	三、MCP — 為什麼要再發明一個協定	32
4.2.4	四、寫 MCP server 的兩個核心 trap	32
4.3	Monday Lecture (9/29): Building a coding agent from scratch	33
4.4	Friday Lecture (10/3): Building a custom MCP server	33
4.5	Reading 摘要	34
4.6	Assignment: First Steps in the AI IDE	34
4.7	對 Vibe Coder 的應用	35
5	Week 3: The AI IDE	37
5.1	學習目標	37
5.2	核心概念導讀	37
5.2.1	一、Context Engineering 的三大課題: Selection、Compression、Decay	37
5.2.2	二、Specs Are the New Source Code — 工作流的反轉	38
5.2.3	三、IDE Integration 的演化: 從 autocomplete 到 manager mode	38
5.2.4	四、Devin vs Claude Code 的設計哲學張力	39
5.3	Monday Lecture (10/6): From first prompt to optimal IDE setup	39
5.4	Friday Lecture (10/10): Silas Alberti (Cognition)	40
5.5	Reading 摘要	41
5.6	Assignment: Build a Custom MCP Server	41
5.7	對 Vibe Coder 的應用	42
6	Week 4: Coding Agent Patterns	43
6.1	學習目標	43
6.2	核心概念導讀	43
6.2.1	一、Agent Autonomy 5 個層級 — 從 autocomplete 到 fully autonomous	43
6.2.2	二、Anthropic 自己怎麼用 Claude Code	43
6.2.3	三、Claude Code 的 4 種擴充 primitive: CLAUDE.md / Skill / Subagent / Hook	44
6.2.4	四、Community Framework 的兩種路線: specialized agent zoo vs meta-framework	44
6.3	Monday Lecture (10/13): How to be an agent manager	45
6.4	Friday Lecture (10/17): Boris Cherny (Claude Code creator)	46
6.5	Reading 摘要	47
6.6	Assignment: Coding with Claude Code	48
6.7	對 Vibe Coder 的應用	48
7	Week 5: The Modern Terminal	51
7.1	學習目標	51
7.2	核心概念導讀	51
7.2.1	一、Terminal 演化簡史 — 為什麼 1970 年代的設計會撐到現在	51
7.2.2	二、AI Terminal 的四個價值主張	51
7.2.3	三、CLI Agent vs IDE Agent — 設計取捨	52
7.2.4	四、為什麼 Vibe Coder 該認真考慮換 Terminal	52
7.3	Monday Lecture (10/20): How to Build a Breakout AI Developer Product	53

7.4	Friday Lecture (10/24): Zach Lloyd (Warp CEO)	54
7.5	Reading 摘要	55
7.6	Assignment: Agentic Development with Warp	55
7.7	對 Vibe Coder 的應用	55
8	Week 6: AI Testing and Security	57
8.1	學習目標	57
8.2	核心概念導讀	57
8.2.1	一、Vibe Coding 的五大資安盲點 – 為什麼速度越快越危險	57
8.2.2	二、SAST、DAST、SCA – 漏洞檢測的三角	57
8.2.3	三、Prompt Injection – Coding Agent 時代的全新攻擊面	58
8.2.4	四、AI-generated Test 與 AI Find Vulnerabilities – 實證告訴我們什麼	58
8.3	Monday Lecture (10/27): AI QA, SAST, DAST, and Beyond	59
8.4	Friday Lecture (10/31): Isaac Evans (Semgrep CEO)	60
8.5	Reading 摘要	61
8.6	Assignment: Writing Secure AI Code	61
8.7	對 Vibe Coder 的應用	61
9	Week 7: Modern Software Support	63
9.1	學習目標	63
9.2	核心概念導讀	63
9.2.1	一、Code review 從「品管把關」到「心智模型同步」的歷史轉折	63
9.2.2	二、AI code review 能做什麼、做不到什麼: 兩軸四象限框架	63
9.2.3	三、為什麼 millions of reviews 教我們的事比任何 paper 都重要	64
9.2.4	四、AI 寫 + AI review + 人類 approve 的 PR workflow	64
9.3	Monday Lecture (11/3): AI code review	65
9.4	Friday Lecture (11/7): Tomas Reimers (Graphite CPO)	66
9.5	Reading 摘要	67
9.6	Assignment: Code Review Reps	67
9.7	對 Vibe Coder 的應用	68
10	Week 8: Automated UI and App Building	69
10.1	學習目標	69
10.2	核心概念導讀	69
10.2.1	一、UI generation 的演化: 從「截圖轉 code」到「一句話生 app」	69
10.2.2	二、Design system 與 component library 在 AI 時代的角色重定位	69
10.2.3	三、Prompt-to-app 五大工具的職能差異	70
10.2.4	四、為什麼 Vercel 的全棧押注 (v0 + AI SDK + AI Gateway) 值得單獨理解	70
10.3	Monday Lecture (11/10): End-to-end apps with a single prompt	71
10.4	Friday Lecture (11/14): Gaspar Garcia (Vercel Head of AI Research)	72
10.5	Reading	72
10.6	Assignment: Multi-stack Web App Builds	72
10.7	對 Vibe Coder 的應用	73

11	Week 9: Agents Post-Deployment	75
11.1	學習目標	75
11.2	核心概念導讀	75
11.2.1	一、SRE 入門: 把營運當軟體工程問題解	75
11.2.2	二、Observability 三柱: logs / metrics / traces	76
11.2.3	三、AI agent 的特殊 observability 需求	76
11.2.4	四、Multi-agent system 在 SRE: MDT 會議模式	77
11.3	Monday Lecture (11/17): Incident response and DevOps	77
11.4	Friday Lecture (11/21): Mayank Agarwal + Milind Ganjoo (Resolve)	79
11.5	Reading 摘要	80
11.6	Assignment	81
11.7	對 Vibe Coder 的應用	81
12	Week 10: What's Next for AI Software Engineering	83
12.1	學習目標	83
12.2	核心概念導讀	83
12.2.1	一、Software Dev Role 的 5 種演化路徑	83
12.2.2	二、Emerging Paradigms: Spec-driven Dev / Formal Verification × AI / Autonomous Repo Maintenance	84
12.2.3	三、AI Dev Tool 市場的 Winner Pattern	84
12.2.4	四、a16z 的 AI Dev Tool 投資 Thesis	84
12.3	Monday Lecture (12/1): Software development in 10 years	85
12.4	Friday Lecture (12/5): Martin Casado (a16z General Partner)	85
12.5	Reading	86
12.6	Assignment	86
12.7	對 Vibe Coder 的應用	86

II

Reading 摘要

13	Reading 摘要總目錄	91
13.1	統計	91
13.2	來源類型分布	91
13.3	Week 1: Introduction to Coding LLMs and AI Development	91
13.4	Week 2: The Anatomy of Coding Agents (MCP)	92
13.5	Week 3: The AI IDE	92
13.6	Week 4: Coding Agent Patterns (Claude Code)	92
13.7	Week 5: The Modern Terminal (Warp)	93
13.8	Week 6: AI Testing and Security	93
13.9	Week 7: Modern Software Support (AI Code Review)	94
13.10	Week 8: Automated UI and App Building	94
13.11	Week 9: Agents Post-Deployment (SRE / Observability)	94

13.12	Week 10: What's Next for AI Software Engineering	94
13.13	高優先閱讀清單（給時間有限的讀者）	95
13.14	延伸主題群	95
14	Deep Dive into LLMs	97
15	Deep Dive into LLMs	99
15.1	核心論點（150-200 字繁中）	99
15.2	關鍵概念	99
15.3	對 CS146S 的意義	99
15.4	對 Vibe Coder 的 Takeaway	99
15.5	原文連結	99
16	Prompt Engineering Overview	101
17	Prompt Engineering Overview	103
17.1	核心論點（150-200 字繁中）	103
17.2	關鍵概念	103
17.3	對 CS146S 的意義	103
17.4	對 Vibe Coder 的 Takeaway	103
17.5	原文連結	103
18	Prompt Engineering Guide (Techniques)	105
19	Prompt Engineering Guide (Techniques)	107
19.1	核心論點（150-200 字繁中）	107
19.2	關鍵概念	107
19.3	對 CS146S 的意義	107
19.4	對 Vibe Coder 的 Takeaway	107
19.5	原文連結	107
20	AI Prompt Engineering: A Deep Dive	109
21	AI Prompt Engineering: A Deep Dive	111
21.1	核心論點（150-200 字繁中）	111
21.2	關鍵概念	111
21.3	對 CS146S 的意義	111
21.4	對 Vibe Coder 的 Takeaway	111
21.5	原文連結	111
22	How OpenAI Uses Codex	113
23	How OpenAI Uses Codex	115
23.1	核心論點（150-200 字繁中）	115

23.2	關鍵概念	115
23.3	對 CS146S 的意義	115
23.4	對 Vibe Coder 的 Takeaway	115
23.5	原文連結	115
24	MCP Introduction	117
25	MCP Introduction	119
25.1	核心論點 (150-200 字繁中)	119
25.2	關鍵概念	119
25.3	對 CS146S 的意義	119
25.4	對 Vibe Coder 的 Takeaway	119
25.5	原文連結	119
26	Sample MCP Server Implementations	121
27	Sample MCP Server Implementations	123
27.1	核心論點 (150-200 字繁中)	123
27.2	關鍵概念	123
27.3	對 CS146S 的意義	123
27.4	對 Vibe Coder 的 Takeaway	123
27.5	原文連結	123
28	MCP Server Authentication	125
29	MCP Server Authentication	127
29.1	核心論點 (150-200 字繁中)	127
29.2	關鍵概念	127
29.3	對 CS146S 的意義	127
29.4	對 Vibe Coder 的 Takeaway	127
29.5	原文連結	127
30	MCP Server SDK (TypeScript)	129
31	MCP Server SDK (TypeScript)	131
31.1	核心論點 (150-200 字繁中)	131
31.2	關鍵概念	131
31.3	對 CS146S 的意義	131
31.4	對 Vibe Coder 的 Takeaway	131
31.5	原文連結	131
32	MCP Registry	133

33	MCP Registry	135
33.1	核心論點 (150-200 字繁中)	135
33.2	關鍵概念	135
33.3	對 CS146S 的意義	135
33.4	對 Vibe Coder 的 Takeaway	135
33.5	原文連結	135
34	MCP Food-for-Thought (APIs don't make good MCP tools)	137
35	MCP Food-for-Thought (APIs don't make good MCP tools)	139
35.1	核心論點 (150-200 字繁中)	139
35.2	關鍵概念	139
35.3	對 CS146S 的意義	139
35.4	對 Vibe Coder 的 Takeaway	139
35.5	原文連結	139
36	Specs Are the New Source Code	141
37	Specs Are the New Source Code	143
37.1	核心論點 (150-200 字繁中)	143
37.2	關鍵概念	143
37.3	對 CS146S 的意義	143
37.4	對 Vibe Coder 的 Takeaway	143
37.5	原文連結	143
38	How Long Contexts Fail (and How to Fix Them)	145
39	How Long Contexts Fail (and How to Fix Them)	147
39.1	核心論點 (150-200 字繁中)	147
39.2	關鍵概念	147
39.3	對 CS146S 的意義	147
39.4	對 Vibe Coder 的 Takeaway	147
39.5	原文連結	147
40	Devin: Coding Agents 101	149
41	Devin: Coding Agents 101	151
41.1	核心論點 (150-200 字繁中)	151
41.2	關鍵概念	151
41.3	對 CS146S 的意義	151
41.4	對 Vibe Coder 的 Takeaway	151
41.5	原文連結	151

42	Getting AI to Work In Complex Codebases (ACE-FCA)	153
43	Getting AI to Work In Complex Codebases (ACE-FCA)	155
43.1	核心論點 (150-200 字繁中)	155
43.2	關鍵概念	155
43.3	對 CS146S 的意義	155
43.4	對 Vibe Coder 的 Takeaway	155
43.5	原文連結	156
44	How FAANG Vibe Codes	157
45	How FAANG Vibe Codes	159
45.1	抓取狀態說明	159
45.2	Best-Effort 推測 (基於標題與 context, 非原文)	159
45.3	關鍵概念(基於 vibe coding 領域共識)	159
45.4	對 CS146S 的意義	159
45.5	對 Vibe Coder 的 Takeaway	159
45.6	原文連結	159
46	Writing Effective Tools for Agents	161
47	Writing Effective Tools for Agents	163
47.1	核心論點 (150-200 字繁中)	163
47.2	關鍵概念	163
47.3	對 CS146S 的意義	163
47.4	對 Vibe Coder 的 Takeaway	163
47.5	原文連結	164
48	How Anthropic Uses Claude Code	165
49	How Anthropic Uses Claude Code	167
49.1	核心論點 (150-200 字繁中)	167
49.2	關鍵概念	167
49.3	對 CS146S 的意義	167
49.4	對 Vibe Coder 的 Takeaway	167
49.5	原文連結	167
50	Claude Code Best Practices	169
51	Claude Code Best Practices	171
51.1	核心論點 (150-200 字繁中)	171
51.2	關鍵概念	171
51.3	對 CS146S 的意義	171

51.4	對 Vibe Coder 的 Takeaway	171
51.5	原文連結	171
52	Awesome Claude Agents (curated list)	173
53	Awesome Claude Agents (curated list)	175
53.1	核心論點 (150-200 字繁中)	175
53.2	關鍵概念	175
53.3	對 CS146S 的意義	175
53.4	對 Vibe Coder 的 Takeaway	175
53.5	原文連結	175
54	Super Claude Framework	177
55	Super Claude Framework	179
55.1	核心論點 (150-200 字繁中)	179
55.2	關鍵概念	179
55.3	對 CS146S 的意義	179
55.4	對 Vibe Coder 的 Takeaway	179
55.5	原文連結	179
56	Good Context, Good Code	181
57	Good Context, Good Code	183
57.1	核心論點 (150-200 字繁中)	183
57.2	關鍵概念	183
57.3	對 CS146S 的意義	183
57.4	對 Vibe Coder 的 Takeaway	183
57.5	原文連結	183
58	Peeking Under the Hood of Claude Code	185
59	Peeking Under the Hood of Claude Code	187
59.1	核心論點 (150-200 字繁中)	187
59.2	關鍵概念	187
59.3	對 CS146S 的意義	187
59.4	對 Vibe Coder 的 Takeaway	187
59.5	原文連結	188
60	Warp University (Getting Started with Warp and Oz)	189
61	Warp University (Getting Started with Warp and Oz)	191
61.1	核心論點 (150-200 字繁中)	191
61.2	關鍵概念	191

61.3	對 CS146S 的意義	191
61.4	對 Vibe Coder 的 Takeaway	191
61.5	原文連結	191
62	Migrate to Warp from Claude Code (Warp vs Claude Code)	193
63	Migrate to Warp from Claude Code (Warp vs Claude Code)	195
63.1	核心論點 (150-200 字繁中)	195
63.2	關鍵概念	195
63.3	對 CS146S 的意義	195
63.4	對 Vibe Coder 的 Takeaway	195
63.5	原文連結	195
64	How Warp Uses Warp to Build Warp	197
65	How Warp Uses Warp to Build Warp	199
65.1	核心論點 (150-200 字繁中)	199
65.2	關鍵概念	199
65.3	對 CS146S 的意義	199
65.4	對 Vibe Coder 的 Takeaway	199
65.5	原文連結	199
66	SAST vs DAST	201
67	SAST vs DAST	203
67.1	核心論點 (150-200 字繁中)	203
67.2	關鍵概念	203
67.3	對 CS146S 的意義	203
67.4	對 Vibe Coder 的 Takeaway	203
67.5	原文連結	203
68	GitHub Copilot Remote Code Execution via Prompt Injection	205
69	GitHub Copilot Remote Code Execution via Prompt Injection	207
69.1	核心論點 (150-200 字繁中)	207
69.2	關鍵概念	207
69.3	對 CS146S 的意義	207
69.4	對 Vibe Coder 的 Takeaway	207
69.5	原文連結	207
70	Finding Vulnerabilities in Modern Web Apps Using Claude Code and OpenAI Codex	209

71	Finding Vulnerabilities in Modern Web Apps Using Claude Code and OpenAI Codex	211
71.1	核心論點 (150-200 字繁中)	211
71.2	關鍵概念	211
71.3	對 CS146S 的意義	211
71.4	對 Vibe Coder 的 Takeaway	211
71.5	原文連結	212
72	Agentic AI Threats: Identity Spoofing and Impersonation Risks	213
73	Agentic AI Threats: Identity Spoofing and Impersonation Risks	215
73.1	核心論點 (150-200 字繁中)	215
73.2	關鍵概念	215
73.3	對 CS146S 的意義	215
73.4	對 Vibe Coder 的 Takeaway	215
73.5	原文連結	215
74	OWASP Top Ten (2021)	217
75	OWASP Top Ten (2021)	219
75.1	核心論點 (150-200 字繁中)	219
75.2	關鍵概念	219
75.3	對 CS146S 的意義	219
75.4	對 Vibe Coder 的 Takeaway	219
75.5	原文連結	220
76	Context Rot: Understanding Degradation in AI Context Windows	221
77	Context Rot: Understanding Degradation in AI Context Windows	223
77.1	核心論點 (150-200 字繁中)	223
77.2	關鍵概念	223
77.3	對 CS146S 的意義	223
77.4	對 Vibe Coder 的 Takeaway	223
77.5	原文連結	223
78	Vulnerability Prompt Analysis with O3 (CVE-2025-37899 use-after-free)	225

79	Vulnerability Prompt Analysis with O3 (CVE-2025-37899 use-after-free)	227
79.1	核心論點 (150-200 字繁中)	227
79.2	關鍵概念	227
79.3	對 CS146S 的意義	227
79.4	對 Vibe Coder 的 Takeaway	227
79.5	原文連結	227
80	Code Reviews: Just Do It	229
81	Code Reviews: Just Do It	231
81.1	核心論點 (150-200 字繁中)	231
81.2	關鍵概念	231
81.3	對 CS146S 的意義	231
81.4	對 Vibe Coder 的 Takeaway	231
81.5	原文連結	231
82	How to Review Code Effectively: A GitHub Staff Engineer's Philosophy	233
83	How to Review Code Effectively: A GitHub Staff Engineer's Philosophy	235
83.1	核心論點 (150-200 字繁中)	235
83.2	關鍵概念	235
83.3	對 CS146S 的意義	235
83.4	對 Vibe Coder 的 Takeaway	235
83.5	原文連結	235
84	AI-Assisted Assessment of Coding Practices in Modern Code Review	237
85	AI-Assisted Assessment of Coding Practices in Modern Code Review	239
85.1	核心論點 (150-200 字繁中)	239
85.2	關鍵概念	239
85.3	對 CS146S 的意義	239
85.4	對 Vibe Coder 的 Takeaway	239
85.5	原文連結	240
86	AI Code Review Implementation Best Practices	241
87	AI Code Review Implementation Best Practices	243
87.1	核心論點 (150-200 字繁中)	243
87.2	關鍵概念	243

87.3	對 CS146S 的意義	243
87.4	對 Vibe Coder 的 Takeaway	243
87.5	原文連結	243
88	Code Review Essentials for Software Teams	245
89	Code Review Essentials for Software Teams	247
89.1	核心論點 (150-200 字繁中)	247
89.2	關鍵概念	247
89.3	對 CS146S 的意義	247
89.4	對 Vibe Coder 的 Takeaway	247
89.5	原文連結	248
90	Lessons from Millions of AI Code Reviews	249
91	Lessons from Millions of AI Code Reviews	251
91.1	核心論點 (150-200 字繁中)	251
91.2	關鍵概念	251
91.3	對 CS146S 的意義	251
91.4	對 Vibe Coder 的 Takeaway	251
91.5	原文連結	252
92	Introduction to Site Reliability Engineering	253
93	Introduction to Site Reliability Engineering	255
93.1	核心論點 (150-200 字繁中)	255
93.2	關鍵概念	255
93.3	對 CS146S 的意義	255
93.4	對 Vibe Coder 的 Takeaway	255
93.5	原文連結	255
94	Observability Basics You Should Know	257
95	Observability Basics You Should Know	259
95.1	核心論點 (150-200 字繁中)	259
95.2	關鍵概念	259
95.3	對 CS146S 的意義	259
95.4	對 Vibe Coder 的 Takeaway	259
95.5	原文連結	259
96	Kubernetes Troubleshooting with AI	261
97	Kubernetes Troubleshooting with AI	263
97.1	核心論點 (150-200 字繁中)	263

97.2	關鍵概念	263
97.3	對 CS146S 的意義	263
97.4	對 Vibe Coder 的 Takeaway	263
97.5	原文連結	263
98	Your New Autonomous Teammate	265
99	Your New Autonomous Teammate	267
99.1	核心論點 (150-200 字繁中)	267
99.2	關鍵概念	267
99.3	對 CS146S 的意義	267
99.4	對 Vibe Coder 的 Takeaway	267
99.5	原文連結	267
100	Role of Multi Agent Systems in Making Software Engineers AI-native	269
101	Role of Multi Agent Systems in Making Software Engineers AI-native	271
101.1	核心論點 (150-200 字繁中)	271
101.2	關鍵概念	271
101.3	對 CS146S 的意義	271
101.4	對 Vibe Coder 的 Takeaway	271
101.5	原文連結	272
102	Top 5 Benefits of Agentic AI in On-call Engineering	273
103	Top 5 Benefits of Agentic AI in On-call Engineering	275
103.1	核心論點 (150-200 字繁中)	275
103.2	關鍵概念	275
103.3	對 CS146S 的意義	275
103.4	對 Vibe Coder 的 Takeaway	275
103.5	原文連結	275
104	9 Weekly Assignments — 自學者可行性評估	277
104.0.1	各週 assignment 摘要	277
104.1	推薦做題順序 (給自學者)	277
104.1.1	短時間 (2 週內) 讀完課	277
104.1.2	中時間 (1 個月) 讀完課	277
104.1.3	完整跟課 (10 週)	278
104.2	Final Project 建議	278

1. 前言

這是一份 Stanford 計算機科學系 Fall 2025 開設的全新課程 CS146S 《The Modern Software Developer》的繁中講義。原版課程主軸是 AI-assisted coding (LLM 基礎、coding agents、Model Context Protocol、AI IDE、Claude Code、Warp、AI testing/security、code review、app building、observability、未來趨勢)，素材公開在 themodernsoftware.dev。

本講義將 10 週課程內容、50+ 篇 reading、9 個 weekly assignment 全部消化成繁中，並補上對非資工背景讀者的譯解，以及對 vibe coder (業餘 AI-assisted coding 使用者) 的應用建議。

詳見 [00_index.md](#) 與 [01_overview.md](#)。

2. 課程簡介與自學路線圖

2.1 為什麼這門課值得念

CS146S 《The Modern Software Developer》是 Stanford CS 在 Fall 2025 全新開的課，由 Mihail Eric ([mihail911](#) – ex-Amazon AGI、ex-Goodlight) 主講。它的特殊性在於：

1. 內容極度當前：講義連結 2025 年才出現的工具 (Claude Code、Warp、Devin、Vercel v0、Resolve.ai)，industry guest speaker 都是這些公司的創辦人或核心成員。
2. 聚焦「人 + AI 協作」的實作層：不是純 ML 理論，而是「身為一個軟體工程師，要怎麼用 AI 工具更快做出正確的東西」。
3. Final project 占 80%：強迫實作。沒寫過東西、只看課的話會看不出價值。

為什麼非資工背景也能念

2.2

雖然 prerequisite 寫 CS111 + CS221/229 recommended，但實際上：

- 大半週次的 lecture 內容是工具使用與工作流 (W1 prompt、W2 MCP、W3 IDE setup、W4 agent management、W5 terminal、W7 code review、W8 UI building)，不需要深 ML 背景。
- 真正需要 ML 知識的只有 W1 後半 (‘‘What is an LLM actually’’) 和 W6 vulnerability detection 的 prompt injection 攻擊面，本講義對這兩塊都會加倍譯解。
- W2/W3 的 agent + context engineering 是 vibe coder 的核心日常，不是學術內容。

對非資工背景讀者的建議：W1 LLM 基礎讀完即可，不必鑽 transformer architecture；把時間花在 W2-W4 的 agent + IDE + Claude Code 實作練習上，這是 ROI 最高的部分。

課程資訊(完整)

2.3

2.3.1 修課負擔

- 每週 10-12 hrs：含 lecture (3 hrs)、assignment (4-6 hrs)、reading (2-3 hrs)
- Final project 不計入週負擔，是學期末額外時間。

評分組成

2.3.2

項目	占比	說明
Final Project	80%	學期末個人專案,展示 modern dev practices
Weekly Assignments	15%	9 個 hands-on 練習,多半是用 Claude Code / Cursor / MCP / Warp 實作
Class Participation	5%	Stanford 課堂出席與討論

對自學者來說,weekly assignments 才是訓練的核心。Final project 沒有 peer / mentor 互相 review 的話，建議用「對應 vibe coding 副業 side project」當練習。

2.3.3 旁聽政策

‘‘We’re open to auditing requests by Stanford students and staff. You will be able to attend all the lectures, but we won’t be able to grade your homework or give advice on final projects.’’

非 Stanford 學生不能旁聽現場課,但 syllabus / slides / reading 都公開在 themodernsoftware.dev。本講義就是基於公開資源做的繁中化版本。

2.4 三條自學路線（從 [00_index.md](#) 重點展開）

2.4.1 Track A — Vibe Coder 速成路線

目標：1-2 週讀完,能用 Claude Code 從零做出一個 side project。

```
W1 (LLM 基礎 + prompting)
  → W2 (agents + MCP, 挑出 MCP 部分動手做)
    → W3 (AI IDE + context engineering, 學 PRD 寫法)
      → W4 (Claude Code 高階用法, Boris Cherny 的 agent manager 概念)
        → W8 (UI/app building, 學 v0 / Vercel)
          → 動手做 final project
```

跳過：W5 terminal（除非你已經在用 Warp）、W6 security（advanced）、W7 code review（團隊用）、W9 observability（production 用）、W10 trends。

2.4.2 Track B — 完整軟體工程路線

目標：跟 Stanford 學生同步進度(10 週),全 syllabus 走完。

每週流程：1. 讀本講義的週 .md（30 min）2. 讀該週 readings 至少 2 篇（90 min）3. 嘗試該週 assignment（90-180 min）4. 看 Mon lecture slides 對照(30 min）5. Fri guest speaker 看 industry context（30 min）

2.4.3 Track C — Tech Lead / PM 視角

目標：理解 AI-assisted coding 對團隊、流程、產品的衝擊,不寫 code。

```
W1 (LLM 基礎, 理解能力邊界)
  → W4 (agent autonomy levels, 理解人機分工)
    → W6 (security, 理解風險)
      → W7 (code review, 理解品控變革)
        → W9 (observability, 理解 production 風險)
          → W10 (industry trends, 預測未來 5 年)
```

學習資源

2.5

2.5.1 必備工具（免費可開始）

工具	用途	免費額度
Claude Code	AI coding agent CLI	免費 plan 有限額
Cursor	AI IDE	14 天 free trial 個人免費
Warp	AI terminal	
Vercel v0	UI 生成	免費額度

2.5.2 學習成本最低的兩篇 prereq

- Andrej Karpathy [Deep Dive into LLMs](#) — 3.5 hr 影片，看完就懂 W1 後半的所有概念
- [Prompt Engineering Guide](#) — 30 min 讀完,覆蓋 W1 的 prompt 技巧

2.6 本講義的譯解原則

1. 技術名詞: English (中文) 首次出現, 後續用 English。例: Model Context Protocol (模型上下文協定, MCP), 後續用 MCP。
2. 產品名稱: 只用英文。例: Claude Code、Cursor、Warp、Semgrep、Graphite。
3. 縮寫: 縮寫 (全稱) 首次出現。例: MCP (Model Context Protocol)、PRD (Product Requirements Document)。
4. 不需翻譯的通用詞: 直接使用。例: CLI、IDE、API、SDK、CI/CD。
5. 對非資工背景讀者的譯解: 在概念第一次出現時, 用「💡 譯解」block 補一句白話。

例: > MCP (Model Context Protocol) 是 Anthropic 提出的開放協定, 讓 AI agent 能用標準化方式呼叫外部工具與資料源。> > 💡 譯解: 可以想成 USB-C 之於充電線——以前每個工具廠都有自己的接頭, MCP 讓 AI 可以一個統一接口接所有外部工具 (Slack、GitHub、Google Drive 等)。

2.7 講義產出限制 / 已知 caveat

- Google Slides 大多需登入 → Mon lecture 內容是基於 topics + speaker context 做的 best-effort 重建, 不是逐字 transcript。
- Friday industry guest speaker 部分內容無 source → 只能依 speaker bio + 公司產品做高層摘要。
- W8 與 W10 沒有 reading list → 內容偏延伸思考 + 工具介紹, 會明確標註「我的延伸」。
- W6 與 W9 對非資工背景偏深 → 加倍譯解, 但仍假設讀者理解基本 web / 系統概念。

如發現摘要內容與原文有出入, 請以原文為準 (每篇摘要均含原文連結)。

I

7.6	Assignment: Agentic Development with warp	55
7.7	對 Vibe Coder 的應用	55
8	Week 6: AI Testing and Security	57
8.1	學習目標	57
8.2	核心概念導讀	57
8.3	Monday Lecture (10/27): AI QA, SAST, DAST, and Beyond	59
8.4	Friday Lecture (10/31): Isaac Evans (Sengrep CEO)	60
8.5	Reading 摘要	61
8.6	Assignment: Writing Secure AI Code	61
8.7	對 Vibe Coder 的應用	61
9	Week 7: Modern Software Support	63
9.1	學習目標	63
9.2	核心概念導讀	63
9.3	Monday Lecture (11/3): AI code review	65
9.4	Friday Lecture (11/7): Tomas Reimers (Graphite CPO)	66
9.5	Reading 摘要	67
9.6	Assignment: Code Review Reps	67
9.7	對 Vibe Coder 的應用	68
10	Week 8: Automated UI and App Building	69
10.1	學習目標	69
10.2	核心概念導讀	69
10.3	Monday Lecture (11/10): End-to-end apps with a single prompt	71
10.4	Friday Lecture (11/14): Gaspar Garcia (Vercel Head of AI Research)	72
10.5	Reading	72
10.6	Assignment: Multi-stack Web App Builds	72
10.7	對 Vibe Coder 的應用	73
11	Week 9: Agents Post-Deployment	75
11.1	學習目標	75
11.2	核心概念導讀	75
11.3	Monday Lecture (11/17): Incident response and DevOps	77
11.4	Friday Lecture (11/21): Mayank Agarwal + Milind Ganjoo (Resolve)	79
11.5	Reading 摘要	80
11.6	Assignment	81
11.7	對 Vibe Coder 的應用	81
12	Week 10: What's Next for AI Software Engineering	83
12.1	學習目標	83
12.2	核心概念導讀	83
12.3	Monday Lecture (12/1): Software development in 10 years	85
12.4	Friday Lecture (12/5): Martin Casado (a16z General Partner)	85
12.5	Reading	86
12.6	Assignment	86
12.7	對 Vibe Coder 的應用	86

10 週講義

3. Week 1: Introduction to Coding LLMs and AI Development

本週你會學到什麼：理解 LLM（Large Language Model，大型語言模型）到底是什麼、它怎麼被訓練出來、以及如何寫 prompt 讓它做你想做的事。這是後續 9 週所有 agent / IDE / coding 工具的基礎。

學習目標

3.1

完成本週後，你應該能：

1. 解釋 LLM 從原始文字資料 → 預訓練 → 後訓練 → instruction tuning → RLHF 的完整製造流程
2. 辨識一個 LLM 在哪些任務上會強、哪些會弱（hallucination、context window、reasoning chains）
3. 應用 5 種以上的 prompt engineering 技巧（few-shot、chain-of-thought、role prompting、structured output、prompt chaining）
4. 比較不同 LLM provider（OpenAI、Anthropic、Google）的 API 與 coding 場景表現

3.2 核心概念導讀

3.2.1 一、LLM 的「製造流程」決定它的能力與盲點

要看懂這門課後續九週講的所有工具（Claude Code、Cursor、Devin、Warp），你必須先理解 LLM 不是黑盒子，它是一條三段式生產線的產物：

1. **Pre-training**（預訓練）— 把整個網際網路 scale 的文本（Common Crawl、Wikipedia、書籍、GitHub code、論壇）餵進 transformer 做 next-token prediction。產出的 base model 本質上是「網路文件補全器」— 給它一段開頭，它會猜下一個 token、再下一個、再下一個。Karpathy 在 [Deep Dive into LLMs](#) 裡把 base model 比喻成「整個網路的有損壓縮」。
2. **Post-training**（後訓練）— **Supervised Fine-Tuning**（SFT，監督式微調）— 用人工標註的對話資料（人問什麼、helpful assistant 應該怎麼回）對 base model 做 fine-tune，把「網路補全器」轉成「會對話的 assistant」。
3. **RLHF**（**R**einforcement **L**earning from **H**uman **F**eedback，人類回饋強化學習）— 用人類偏好訓練一個 reward model，再用 PPO / DPO 之類演算法把 LLM 的回答品質拉上來。這階段決定了 LLM「願不願意拒絕請求」「會不會諂媚」「邏輯是否一致」。

為什麼這個 **mental model** 重要？因為 LLM 的所有奇怪行為都能從這三段推得：- **Hallucination**（幻覺）= pre-training 時把「合理的下一個 token」當目標，statistical pattern 偶爾會生成看似合理但事實錯誤的內容 - **Knowledge cutoff**（知識截止）= pre-training data 截止日之後的事它不知道（除非接 web search tool）- **算術 / 字元計數失敗** = tokenization 把文字切成 BPE token，模型看到的不是「字母」是 token id - **Capability spectrum**（能力光譜）不均勻 = 某些任務（寫 React component）強到嚇人，某些任務（畫 ASCII art、簡單算術）爛到爆

💡 譯解：你可以想成「LLM = 一個讀過全網路的學生 + 經過 RLHF 公司新訓練的客服」。它對網路上常見的東西超熟（程式、論文、Wikipedia 主題），但對訓練後才出現的事（你公司的 codebase、你昨天 commit 的程式）一無所知 — 必須透過 prompt 把那些 context 餵給它。

3.2.2 二、Prompt Engineering 是「給 contractor 的 brief」

[Anthropic 的圓桌討論](#) 裡有個比喻最精準：寫 prompt 像「請一個聰明但完全不認識你公司的 contractor 做一件事」。好 prompt 的本質不是奇技淫巧，是清楚的技術寫作。

[Google Cloud 的 prompt engineering overview](#) 把這件事拆成可操作的四要素：

1. **Format**（格式）－自然語言問句 vs 結構化指令 vs JSON schema，依任務選
2. **Context and examples**（情境與範例）－提供任務背景與 1-3 個 input-output pair（few-shot）
3. **Fine-tuning**（微調 prompt）－看 model 失敗 case 反推 prompt 該補什麼
4. **Multi-turn conversations**（多輪對話）－設計能維持 context 的對話流程

[Prompt Engineering Guide](#) 列了 18 種主流技巧，分四層：

層級	技巧	何時用
基礎	zero-shot / few-shot	80% 場景的起手式
推理	Chain-of-Thought (CoT) / Self-Consistency / Tree of Thoughts	多步驟問題、數學、邏輯
工具	RAG / ReAct / PAL / ART	需要外部資料或執行 code
Meta	APE / Reflexion / Meta Prompting / Prompt Chaining	把 prompting 本身自動化

實務升級路徑通常是：zero-shot → 不行就 few-shot → 還是不行加 CoT → 還是不行接 RAG。80% 的問題在第三步前就解掉。

3.2.3 三、Coding LLM 的真實工業使用樣貌

OpenAI 自家的 [How OpenAI Uses Codex](#) 是這週最重要的 case study。它不是 demo 也不是 marketing，是 OpenAI 內部 6 個團隊（Security、Product Engineering、Frontend、API、Infrastructure、Performance）每天怎麼用 Codex 的真實使用報告。歸納出 7 個高 ROI use case：

1. **Code understanding**－摸熟陌生 repo、追資料流、on-call incident triage
2. **Refactoring & migrations**－跨多檔案的一致性改動（callback → async/await）
3. **Performance optimization**－找 hot path、批次化 DB query
4. **Improving test coverage**－補 unit / integration test
5. **Increasing dev velocity**－scaffold boilerplate、收尾 last-mile
6. **Staying in flow**－在 meeting / on-call 碎片時間 fire-and-forget 任務
7. **Exploration & ideation**－找替代方案、辨識潛在 regression

更重要的是 5 條 best practice，是這 1-2 年才被工業界沉澱出來的：

- **Ask Mode 先於 Code Mode**：先讓 model 出 implementation plan，你 review 後再讓它寫 code（避免寫完才發現方向錯）
- **AGENTS.md / CLAUDE.md** 放 repo-level persistent context（命名慣例、業務邏輯、quirks）
- **Best-of-N**：同 task 同時跑多個版本，挑或合併
- **Task queue as backlog**：把碎想丟 queue，不必當下完成 PR
- **GitHub Issue-style prompt**：含檔案路徑、模組名、diff、doc snippet 的 prompt 效果最好

這 12 頁 PDF 是這週最該認真讀的，它把後續 W2-W4 要講的所有 agent 工具用法都濃縮成一份 production-tested 守則。

3.3 Monday Lecture (9/22): Introduction and how an LLM is made

- **Slides**: [Google Slides 公開連結](#)
- **講者**: Mihail Eric（course instructor, Stanford undergrad/grad、前 Amazon Alexa 早期 LLM 團隊、ML 教育新創創辦人、YC-backed AI coding company 創辦人）

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

第一節要校準兩件事: (1) 這門課不是 **vibe coding** 課、不是教非工程背景使用者怎麼避開請工程師; (2) LLM 不是黑盒子, 理解它的製造流程才能用好它。Mihail 把 lecture 切成三段:

1. **State of the world** 與軟體業現況 – 軟體工程行業正在被 AI 重塑, CS 主修申請數已下滑 20%。但 Mihail 的核心命題是: 「**You won't be replaced by AI. You'll be replaced by a competent engineer who knows how to use AI.**」如果你的 value-add 只剩會 copy-paste Stack Overflow, 就會被取代; 但若能想 system architecture、抓 business context、設計可維運的 abstraction, AI 反而會把你的生產力推到 10 倍。
2. 這門課的核心命題: **human-agent engineering** (取代 vibe coding 的標準說法) – 包括 (a) 聚焦尚未被 AI 取代的能力 (business understanding、tech lead 思維、好的 taste)、(b) 「LLMs are only as good as you are, good context leads to good code, if you can't understand your codebase, neither will an LLM」、(c) 大量讀 code、激進實驗。Mihail 強調目前沒有定型的 software pattern, 整個業界都在摸索。
3. 5 張投影片速講 LLM 製造流程 (給工程師的版本) –
 - **Basics:** LLM 是 autoregressive next-token predictor。Tokenize → embedding (1-3K 維) → 12-96+ 層 transformer + causal self-attention → 下個 token 機率分布
 - 三階段訓練: (a) **Pretraining** 用 100B-1T+ token (Common Crawl / Wikipedia / StackExchange / GitHub) 做 self-supervised next-token prediction、(b) **SFT** 用數萬到數十萬筆人工 prompt-response pair 教 model 跟 instruction、(c) **Preference tuning** 收集成對 output 訓練 reward model 對齊 helpfulness / correctness / readability
 - 資料量級對照: GPT-3 ~300B token / 570 GB, PaLM 780B token, LLaMA-65B 1.4T token; code-specific 像 Codex 額外吃幾十 GB GitHub code, StarCoder 吃 3.1 TB (English Wikipedia 才 3B token 當作參考點)
 - **Reasoning model:** 在 SFT/RLHF 之上加 chain-of-thought trace + tool use + 對 reasoning step 收集人類偏好。Model size: Claude 3.5 Sonnet ~175B, LLaMA 3.1 405B, GPT-4 reportedly 1.8T
4. **In practice** 的 **strengths vs limitations** – 強: expert-level code completion、code understanding、code fixing。弱: hallucination (在 less-represented language 更嚴重)、context window 雖然 100-200K 但有 primacy/recency bias 與 lost-in-the-middle、latency (秒到分)、cost (最頂模型 input ~\$1-3/M token、output \$10+/M token, 但每年降約 10×)

Key takeaway: 把 LLM 當讀過全網路的工程實習生而非魔法 – 它在 pre-training 看過的 pattern (常見 framework、open source code) 很強, 但 SFT/RLHF 沒看過的 (你公司 codebase、你昨天的 commit) 必須透過 prompt 餵 context。後續九週所有工具都是把這個 mental model 的不同瓶頸拆開來解決。

3.4 Friday Lecture (9/26): Power prompting for LLMs

- **Slides:** [Google Slides 公開連結](#)
- **講者:** Mihail Eric

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

Mihail 借用 Karpathy 的觀點開場: **prompting** 是 **programming language** 演化的下一階段。就像搜尋引擎的 query 從 boolean algebra 演化成自然語言, prompting 也越來越自然語言化。但「自然」不代表「隨便」 – prompt 是 art + science 的混合體: LLM 的 black-box 本質讓它有「whispering」的玄學成分, 但業界已經沉澱出一套 empirically improved 的技巧。Lecture 系統地走過這套技巧 catalog:

1. **Zero-shot prompting** – 直接 ask, 沒範例。例: 「Make me a heap allocator in C」。適合 LLM 已熟的 well-known library / general coding task。
2. **K-shot prompting (in-context learning)** – 給 1 / 3 / 5 個 example (empirical 上這幾個數字最有用)。用在: domain-specific API、enterprise 內部風格、命名慣例。不用在: well-known library、過度約束。Demo 用「在我們 repo 命名風格下寫 for-loop」, 先 naive 提問再用 `<example>` 標籤包進兩段公司 convention 範例, 立刻看到輸出差異。

3. **Chain-of-Thought (CoT)** – 顯式 show reasoning step。兩種變體：(a) **Multi-shot CoT** 提供 worked-out reasoning trace, (b) **Zero-shot CoT** 用 “Let’s think step-by-step” 觸發。是 reasoning model 的主力技術。適合 multi-step logic / programming / math。
4. **Self-consistency prompting** – 同 prompt sample 多次 (通常配 CoT) 後取 majority vote, 相當於 model ensembling, 能降 hallucination。Demo 用「用 5 次取多數」debug 一個 `IndexError`。
5. **Tool use** – 讓 LLM 把無法獨自完成的事 delegate 給外部系統。最重要的 hallucination 緩解技術之一,也是 autonomy 的基礎。Demo 用 `<tools>` 標籤列出 `pytest -s ... / pytest -v ...` 給 model 自己決定何時 call。
6. **Retrieval Augmented Generation (RAG)** – 把 contextual data 注入 prompt。優點:保持 up-to-date、可解釋、自帶 citation、降 hallucination。Demo 把 `UserAuthService` 既有 code snippet + `requests-oauthlib` 文件 URL 一起塞進 prompt。Cursor / Windsurf 的 `@context` 就是 RAG。
7. **Reflexion (self-critique)** – 多輪: Turn 1 model 出第一版 → Turn 2 加「critique your answer, was it correct?」讓 model 自我修正。Mihail 說這是 modern coding agent 完整 agentic 行為的 workhorse。
8. **System / user / assistant prompt 三段結構** – 用 Claude 4.1 Opus 的真實 system prompt 當教材 (「鬆散版的 Asimov 三大機器人定律」)。System prompt 通常 user 看不到, 是 persona / 規則 / output style 的設定處。
9. **Best practice 收尾** – (a) 給 prompt 給沒有背景的人看,他困惑 LLM 也會困惑; (b) 激進使用 role prompting (「You are a helpful assistant that loves programming...」vs「You are a Gen Z digital bestie...」展示巨大差異); (c) 用結構化標籤包 data: `<log>...<log> <error>...<error>`; (d) explicit 寫出 language / stack / library / constraint; (e) decompose task。

Key takeaway: Prompt 是可被工程化的物件, 不是奇技淫巧。實務 escalation path: zero-shot → 不行用 few-shot → 還不行加 CoT → 還不行接 RAG / tool use。Reflexion 與 self-consistency 是當代 coding agent autonomy 的底層機制, 理解這幾招就能看懂 Claude Code / Cursor 內部的 prompting layer 在做什麼。

3.5 Reading 摘要

篇名	來源	一句話重點
Deep Dive into LLMs	Karpathy YouTube 3.5hr	LLM = pre-training + post-training + RLHF 三段生產線, 所有 weirdness 都從這裡推
Prompt Engineering Overview	Google Cloud	Prompt 設計四要素 + 5 種 prompt 類型 (zero/one/few/multi-shot, CoT)
Prompt Engineering Guide	promptingguide.ai	18 種 prompting 技巧 catalog, 分基礎 / 推理 / 工具 / meta 四層
AI Prompt Engineering: A Deep Dive	Anthropic 圓桌	寫 prompt = 給聰明 contractor 的清楚 brief, 不是奇技淫巧
How OpenAI Uses Codex	OpenAI 內部 PDF	7 個 production use case + 5 條 best practice, 最該讀的一篇

閱讀優先順序: 時間有限的话, 先讀 [How OpenAI Uses Codex](#) (最 actionable) → 再讀 [Anthropic deep dive](#) (業界視角) → 有時間補 Karpathy YouTube (基礎理論)。

3.6 Assignment: LLM Prompting Playground

- **Source:** github.com/mihail911/modern-software-dev-assignments/tree/master/week1
- **任務描述:** 練習用不同 prompting 技巧 (zero-shot, few-shot, CoT, structured output) 解決一系列 task, 比較 output 差異, 培養「prompt 是可工程化物件」的直覺。
- **自學者可行性:** ★★★★★ 完全可做。需要 OpenAI 或 Anthropic API key (前者有 \$5 免費額度, 後者有 free tier)。預估 2-4 hr。

💡 沒有 API key 的替代方案: 用 [Anthropic console](#) 或 [OpenAI playground](#) 直接在 web UI 操作, 免錢上手。

3.7 對 Vibe Coder 的應用

這週的概念怎麼套到你日常用 Claude Code / Cursor 的工作流?

1. 建立 LLM 心智模型 – 下次 Claude 給你錯答案, 先問自己「這是 hallucination (pre-training 沒學到) 還是 context 沒給夠 (post-training 沒看過你 codebase)?」前者要 RAG 或網路搜尋, 後者要把 context 餵清楚
2. 養成「Ask Mode 先於 Code Mode」習慣 – 用 Claude Code 寫新功能前先問「請幫我列出實作計畫, 先不要寫 code」, 等你 review 完再讓它落地。這一招會省超多 rollback 時間
3. Repo 加 **CLAUDE.md** – 把專案的命名慣例、業務邏輯、常見 quirks 寫進去 (同 OpenAI 內部的 **AGENTS.md**)。Claude Code 每次都會讀, 等於 persistent context
4. **Few-shot examples** 不只給格式 – 給 example 時挑「你期望它怎麼推理」的 case, 不只是「你期望它輸出什麼格式」。這個差異對 reasoning task 特別大
5. **Structured output** 走 XML tag – Claude 對 `<output>` `<thinking>` 之類 XML 比 JSON 配合度更高, 逼它產 structured data 時用 XML 比較穩

💡 **vibe coder 的 Day-1 Quick Win:** 今天就在你的 side project repo 加 **CLAUDE.md**, 內容寫 (a) 這個 repo 在做什麼 (一段話)、(b) 命名慣例 (snake_case / camelCase / PascalCase)、(c) 任何「Claude 容易搞錯的事」(例: 用 pnpm 不是 npm、用 Tailwind v4 syntax 不是 v3)。你會發現 Claude Code 的輸出立刻變得不一樣。

上一週: (無 – 本週是第 1 週) | 下一週: [W2 The Anatomy of Coding Agents](#)

4. Week 2: The Anatomy of Coding Agents

本週你會學到什麼：拆解 coding agent (Claude Code、Cursor、Devin、Codex) 的內部結構，理解 LLM + tool use + memory + planning 怎麼組合成一個會自己寫 code 的 agent。並動手做出一個 custom MCP (Model Context Protocol) server。

學習目標

4.1

完成本週後，你應該能：

1. 拆解 coding agent 的核心組成 (LLM、tool registry、planner、memory、observer)
2. 解釋 function calling / tool use 在 LLM API 層面的運作機制
3. 設計一個 MCP server 把外部資料源接進 Claude / Cursor
4. 比較 Claude Code、Cursor、Devin 三種 agent 在架構決策上的差異

4.2 核心概念導讀

4.2.1 一、Agent ≠ LLM + 一個 while loop

W1 把 LLM 講成「讀過全網路的 contractor」，但只有 contractor 沒有手腳，做不了事。**Agent = LLM + 能執行動作的能力 + 會記得自己做過什麼的記憶。**最小可用 agent 的架構長這樣：

```
loop:
  1. 讀取當前 context (系統訊息 + 對話 + 上次 tool result)
  2. LLM 推理 → 決定要 call 哪個 tool 還是直接回答
  3. 若 call tool: 執行 → 拿 result 塞回 context
  4. 重複直到 LLM 認為任務完成
```

這個 loop 的每一個環節都是一個設計決策：

元件	設計問題
LLM	用哪家？OpenAI / Anthropic / Google / open-source？跨 model 兼容嗎？
Tool registry	Tool 怎麼定義？Tool 數量多少會 overload model？怎麼分組？
Tool execution	Tool 跑在哪？sandbox 還是直接 host shell？怎麼處理 long-running task？
Memory	Context window 撐不下時怎麼壓縮？要不要外掛 vector DB？對話跨 session 嗎？
Planner	直接 reactive 還是先 plan 再 act？Plan 出錯怎麼回滾？
Observer / Reflexion	Agent 跑錯時誰來糾正？人類介入還是 self-critique？

Claude Code、Cursor agent、Devin、Codex 的差異全在這六個槽位的決策上。例如 Devin 強調 autonomous (少人介入)，Claude Code 強調 interactive (人類隨時打斷)，對應的 memory 與 observer 設計就差很多。

4.2.2 二、Tool use / Function calling 的底層機制

Tool use 是 agent 的「手腳」。在 LLM API 層面，function calling 的 protocol 大致長這樣：

1. **API request** 同時送 prompt 和 tool schema (JSON 格式描述每個 tool 的 name、description、parameters)
2. **LLM 回應** 不是直接回答而是回 `tool_use` block, 含 tool name + 參數
3. **Client** (你的 **agent loop**) 看到 `tool_use` 就執行對應 tool function
4. **Tool result** 包成 `tool_result` 塞回 context, 再呼一次 LLM
5. LLM 看到 tool result 後決定: 再 call 一次 tool (多步驟) 還是給最終 answer

關鍵設計細節: - **Tool description** 是給 LLM 看的 **prompt** – 寫得越清楚, model 越會在對的時機 call
 - **Parameter schema** 用 JSON Schema / Zod / Pydantic – 強制 model 產生可被 parse 的結構化參數
 - **Tool result** 的格式直接影響下一輪推理 – Reilly Wood 在 [MCP Food-for-Thought](#) 裡實測「同樣資訊用 CSV 比 JSON 省一半 token」
 - **Error handling** – Tool 失敗時要回 model 「可恢復的錯誤訊息」, 不要只 throw exception

4.2.3 三、MCP – 為什麼要再發明一個協定

Function calling 的 protocol 各家不同 (OpenAI 的 `tools` schema、Anthropic 的 `tool_use` block、Google 的 `functionDeclarations`), 意味著你寫的每個 tool 要為每個 LLM client 重做一次。MCP (**Model Context Protocol**, 模型上下文協定) 就是要解決這個問題 – 讓 tool 寫一次, Claude Desktop、Cursor、Continue.dev、所有 MCP-compatible client 都能用。

💡 譯解: MCP 之於 LLM tools, 就像 USB-C 之於充電線。以前每個工具廠都有自己的接頭 (OpenAI plugin / Cursor extension / Claude artifact), MCP 讓你用一個統一接口接所有外部工具 (Slack、GitHub、Google Drive、你自己的資料庫)。

Stytch 的 [MCP Introduction](#) 把 MCP 拆成三類 capability:

Capability	用途	範例
Tools	可被 LLM 呼叫執行動作 的函式	<code>send_email</code> , <code>query_database</code> , <code>create_pr</code>
Resources	模型可讀取的 context 資料	檔案內容、DB 紀錄、API 回傳
Prompts	預先定義的任務模板	“/code-review”、 “/ex- plain-this”

底層用 JSON-RPC 2.0 做 message format, **transport** 分 local (stdio) 和 remote (HTTP/SSE) 兩種。Local stdio transport 開發成本最低, 原型階段不必處理 OAuth; 要做 multi-user 服務再上 [Cloudflare Workers + OAuth](#)。

[MCP Registry](#) (2025 年 9 月推出) 解決「server 找不到、寫的人沒地方掛」的問題, 採 federation-friendly 設計 (中央 registry + sub-registry 共存), 不做 app store 壟斷。

4.2.4 四、寫 MCP server 的兩個核心 trap

[MCP Food-for-Thought](#) 給了非常實際的反思: 別把 OpenAPI spec 機械式包成 MCP tool。理由:

1. **Tool proliferation** (工具擴增) – Tool 數量在 128 之前就會明顯惡化 agent 的選 tool 準確率
2. **Context 浪費** – API 回的 wide record 含一堆用不到的 field、JSON 重複 key 名稱浪費 token
3. **錯失 agent 原生能力** – 為 agent 設計的 tool 應該回 free-form text + 引導 next step, 不是 strict schema dump
4. **冗餘** – Claude Code 之類現代 agent 本來就會自己寫 code 直接 call API

實務原則: - 把多支相關 API 合併成一個彈性 tool (含 `fields` 參數讓 caller 自選欄位) - Response 用 markdown table 或 CSV, 別 dump 整坨 JSON - 在 description 裡寫「這個 tool 適合在 X 情境用, 後續通常呼叫 Y」幫 agent routing - 做 layered tool chaining: response 引導 next step

4.3 Monday Lecture (9/29): Building a coding agent from scratch

- Slides: [Google Slides 公開連結](#)
- Completed Exercise: [Drive 連結](#)
- 講者: Mihail Eric

以下基於 Google Slides 公開內容 (TXT export, slides 內容簡潔) 整理的繁中摘要:

這節 slides 內容相當精簡 (只有 ~1.2KB 文字), 核心命題是「 coding agent 沒你想得那麼複雜」。Mihail 提出最小可用 agent 的三句話骨架: 使用者跟 client (Windsurf / Cursor / Claude Code) 互動, client 內部跑一個 loop 接著一個 LLM; LLM 偶爾發出 tool call, 由 client 在 LLM 之外 (off-LLM) 執行。實作步驟:

1. 三段 prompt 結構 — system prompt 定義 LLM 行為與 directive、user prompt 是使用者請求、assistant prompt 是 LLM 回應
2. Agent loop 機制 — 從 terminal 讀 input、不斷 append 到 conversation; 告訴 LLM 有什麼 tool 可用; LLM 在適當時機要求 call tool; offline 執行 tool 並把 result 塞回 context
3. 三個基本 tool — read_file、list_dir、edit_file (建立新檔、修改檔案)
4. Live build session — 現場用幾十行 code 把上述 loop 接起來

Lecture 收尾揭露 Claude Code 的「 secret sauce 」(Mihail 對外公開觀察 Claude Code 內部運作的整理):

- Front-load context: 用小而精準的 prompt 在 session 開頭壓縮目標
- <system-reminder> 散布全程: 在 system prompt、user prompt、tool call、tool result 各層都注入 system reminder 標籤抵抗 long-context drift
- Command prefix extraction: 對使用者輸入做前綴解析來分流意圖
- Spawns sub-agents: 主動 fork 出 sub-agent 處理特定 sub-task, 避免主對話 context overload

Key takeaway: Coding agent 不是新物種 — 它就是 LLM + tool registry + while loop 的最小組合。Claude Code / Cursor / Devin 的差別不在這個骨架, 而在 prompt scaffolding (系統提示、<system-reminder> 散布、sub-agent 隔離) 這些工程紀律。寫過一次自己的 100 行 agent 你會發現: 難的不是 loop, 是讓 model 在對的時機選對 tool。

4.4 Friday Lecture (10/3): Building a custom MCP server

- Slides: [Google Slides 公開連結](#)
- Completed Exercise: [Drive 連結](#)
- 講者: Mihail Eric

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

這節從 W2 Monday 的「 LLM 知識是 vast 但 static 」痛點出發。要建 fully autonomous system, 必須有可靠機制把 dynamic data (今天天氣、誰是現任總統、Bitcoin 價格、Nike 廣告旁白是誰) 餵進 LLM。RAG 與 tool-calling 是目前最佳解 — 但 tool-calling 在 MCP 之前是 N×M 噩夢。

1. Why MCP — MCP (Model Context Protocol) 是 2024 年 11 月才被 Anthropic 推出的開放協定, 用一句話講就是「把 tool 暴露給 LLM 的標準格式」。在 MCP 之前, 每個 LLM app (Cursor、Claude Desktop、Continue.dev) 要接每個 third-party API (Slack、GitHub、Notion) 都要寫 N×M 個 connector, 還要各自處理 poor doc、不一致 data format、authentication、error handling。MCP 把這個複雜度從 N×M 壓到 N+M。
2. 設計傳承 — MCP 概念 extend 自 LSP (Language Server Protocol, VS Code 用來支援多語言的同類 N×M 解法), 但 MCP 多了 proactive agentic workflow 的能力, 不像 LSP 純 reactive。Output 強制 JSON-RPC 格式。

3. **MCP 術語** – **Host** (Cursor、Claude Desktop)、**MCP Client** (Host 內嵌的 library, 每個 server 一個 stateful session)、**MCP Server** (包裝 tool 的輕量 wrapper)、**Tool** (可被呼叫的 function, 可以是 data source 或 API)。
4. **Flow** – Client 對 server 發 `tools/list` 問「你能做什麼」→ server 回 JSON 描述每個 tool 的 name / summary / JSON schema → host 把這些 JSON 注入 model context → user prompt 觸發 model 發出 structured tool call → MCP server 執行後對話繼續。
5. **Transport** – MCP 提供 stdio (local stdin/stdout) 與 SSE (remote HTTP server-sent event) 兩種。Local stdio 開發成本最低, 原型階段不必處理 OAuth。
6. **Live build session** – 從零寫一個 custom MCP server, 現場 demo `list_files` / `edit_file` 兩個 tool 被 server 實際呼叫的情況。注意: tool 多時 model 不一定會挑對 server, 要在 prompt 中明說用哪個 server。
7. **Limitations** – 今天的 agent 處理多 tool 能力還不行 (Cursor 有 hard limit); API 回的資料會快速吃掉 context window; APIs 應該設計成 AI-native 而非把 rigid REST 直接搬過來。Mihail 推薦延伸閱讀 [arXiv 2505.03275](https://arxiv.org/abs/2505.03275) 講 MCP mitigation 策略。

Key takeaway: MCP 不是又一套技術 buzzword – 它解決的是 N×M connector 爆炸這個工業界真實痛點。寫過一次自己的 MCP server, 你就能把任何外部資料源 (RemNote、PubMed、阿摩錯題庫、Google Calendar、健保碼) 統一接到 Claude Desktop / Cursor / 任何 MCP host, 零 plugin、零 extension。Limitation 也別忘 – tool 數量在 30-100 個之間就會明顯惡化 model 的選 tool 能力。

4.5 Reading 摘要

篇名	來源	一句話重點
MCP Introduction	Stytch	MCP = AI 應用與外部系統之間的 universal adapter, client-server + JSON-RPC 架構
Sample MCP Server Implementations	GitHub	官方 reference servers (filesystem、GitHub、Slack、Postgres 等), 抄來改最快
MCP Server Authentication	Cloudflare	OAuth 2.0 加在 remote MCP server, hobby → production 必經之路
MCP Server SDK	GitHub	TypeScript SDK README, 寫 server 的 starting point
MCP Registry	blog.modelcontextprotocol.com	官方中央目錄 + federation-friendly sub-registry 設計
MCP Food-for-Thought	reillywood.com	別把 OpenAPI 機械式包成 MCP, 要為 agent 重新設計 tool

閱讀優先順序: 先 [MCP Introduction](#) (建立 mental model) → [MCP Food-for-Thought](#) (設計品味) → [MCP Server SDK](#) (動手) → 其他依需求補。

4.6 Assignment: First Steps in the AI IDE

- **Source:** github.com/mihail911/modern-software-dev-assignments/tree/master/week2
- **任務描述:** 在 Cursor / Claude Code / VS Code GitHub Copilot 之一的 AI IDE 內完成一系列 hands-on 練習, 建立 IDE 工作流的肌肉記憶。
- **自學者可行性:** ★★★★★ 完全可做。需 Cursor 14 天 trial 或 Claude Code 訂閱(前者免費試用、後者 free tier 有限額)。預估 3-5 hr。

4.7 對 Vibe Coder 的應用

W2 是 vibe coder 應該重投入時間的一週 – 學會寫 MCP server 之後, Claude Code 的能力上限會直接拉一個 level。實戰建議:

1. 第一個 MCP server 從「自己最常查的資料」開始 – 你可能查最多的: 自己的 Obsidian / RemNote 筆記、PubMed、特定 API (你習慣用的天氣、台股、健保碼)。挑一個, 寫成 MCP server, 接到 Claude Desktop。20 分鐘就能跑起來。
2. **Tool description 比 implementation 重要** – Description 是給 LLM 讀的 prompt, 寫得清楚 model 才會在對的時機 call。例: 別寫 "description": "Query database", 要寫 "description": "Search the user's Obsidian vault for notes by keyword. Use this when user asks about their personal notes, journals, or saved knowledge. Returns markdown content with file paths."
3. **永遠加 fields / limit 參數** – 別讓 tool 一次回 1000 筆, 給 caller 自選 page size。對 token 經濟學 **至關重要**
4. **Local stdio 先做, OAuth 之後再說** – 90% 的 vibe coding use case 是個人用, stdio transport 配 Claude Desktop 就夠。要開放給朋友再上 Cloudflare Workers + OAuth
5. 善用 **Sample servers** – 官方 repo 有 filesystem、GitHub、Slack、Postgres 等 reference 實作, 抄一份來改是最快路徑
6. 裝完 server 第一件事去 **Registry** 提交 – 免費、無審核延遲, 別人能在 Claude Desktop 內搜到你的 server

💡 **vibe coder 的 Day-1 Quick Win:** 今天裝 Claude Desktop, 從 [modelcontextprotocol/servers](#) 挑一個你會用到的 server (filesystem、GitHub、brave-search 都好), 改 `~/Library/Application Support/Claude/claude_desktop_config.json` 加進去, 重啟 Claude Desktop, 立刻就有外部能力。再來才是寫自己的 server。

上一週: [W1 Introduction](#) | 下一週: [W3 The AI IDE](#)

5. Week 3: The AI IDE

本週你會學到什麼: AI IDE (Cursor、Windsurf、Zed、Continue.dev) 怎麼管理 codebase context、怎麼讀懂大型 codebase; 以及 spec / PRD 在 agent 時代的角色變化(“specs are the new source code”)。Friday 由 Cognition (Devin 母公司) 的 Head of Research Silas Alberti 開講。

學習目標

5.1

完成本週後，你應該能:

1. 解釋 context window 為什麼是 AI IDE 最關鍵的瓶頸，以及 RAG / chunking / re-ranking 怎麼解決
2. 撰寫一份能餵給 agent 的 PRD (Product Requirements Document)，含 spec、acceptance criteria、guardrails
3. 設定 Claude Code / Cursor 的 context loading (CLAUDE.md / .cursorrules、MCP servers、custom commands)
4. 評估 Devin 跟 Claude Code 的設計哲學差異 (autonomous agent vs interactive collaborator)

5.2 核心概念導讀

5.2.1 一、Context Engineering 的三大課題: Selection、Compression、Decay

W2 把 agent 拆成「LLM + tool + while loop」，但只要你實際用 Claude Code 寫過 non-trivial 任務，就會遇到一個 W2 沒講的瓶頸 — **context window** 不是越大越好。Drew Breunig 在 [How Long Contexts Fail](#) 把這件事拆成 4 種具體的 failure mode:

1. **Context poisoning** (脈絡中毒) — Hallucination 一旦寫進對話 context 就會被反覆 self-reference, agent 朝不可能的目標前進。Gemini 玩 Pokémon 的著名 case: 模型某輪生成「我已經拿到 X 道館徽章」(其實沒有)，之後幾百輪 reasoning 都基於這個錯誤前提。
2. **Context distraction** (脈絡分心) — Context 過長時模型會偏向「重複歷史 action」而非用 train 出來的 reasoning 重新規劃。小模型 ~32k token 後開始崩、大模型 ~100k 後也會。
3. **Context confusion** (脈絡混淆) — Berkeley Function-Calling Leaderboard 顯示 8B 模型給 19 個 tool 還能用、46 個就崩。tool 越多 ≠ agent 越強，反而 routing accuracy 直接下降。
4. **Context clash** (脈絡衝突) — Microsoft / Salesforce 研究顯示「分輪餵相同資訊」比「一次性餵」accuracy 下降 39%。多輪對話一旦走偏，回不來。

理解這 4 種 failure 後，**context engineering** 就拆成三件事:

課題	問題	解法
Selection (選擇)	從 1M token codebase 裡,這次 task 該餵哪些 file 進 context?	RAG、grep + re-rank、@file reference、CLAUDE.md routing
Compression (壓縮)	Context 用到 60% 後怎麼濃縮?	/compact summary、sub-agent isolation、phase-boundary checkpoint
Decay (衰減)	長 session 怎麼防 drift?	<system-reminder> 重述目標、/clear 重啟、phase-spec re-anchor

humanlayer 團隊在 [Getting AI to Work In Complex Codebases](#) 直接喊出 ACE-FCA (Advanced Context Engineering for Coding Agents) 的核心命題: 「**context window** 的內容是你影響 **output** 品質的唯

一槓桿」。Agent 是 stateless function — 同樣的 context window 給同樣的 model 永遠產出同樣品質。所以使用者最高槓桿的工作就是 curate 那個 window。

💡 譯解: context engineering 跟臨床問診同構。問診 (context selection) 漏問就鑑別診斷漏; 問診冗長 (context bloat) 就抓不到重點; 前後病史矛盾 (context clash) 就推不出結論。Claude Code 寫不好 code 時,先檢查的不是 prompt 措辭, 是「我有沒有給它對的 file」。

5.2.2 二、Specs Are the New Source Code — 工作流的反轉

W1/W2 還在「prompt 怎麼寫」的層級。Week 3 把視角拉高一階: 當 AI 把 implementation 從週縮到分鐘,瓶頸從「會不會寫 code」變成「能不能描述清楚需求」。Ravi Mehta 在 [Specs Are the New Source Code](#) 借 Sean Grove 的話: 傳統世界是 source code (人讀) → binary (機器讀),AI 時代等於「把 source 撕碎, 反而謹慎 version control 那個 binary」— 因為 code 只是 intent 的 lossy projection (有損投射), spec 才是完整需求。

Workflow 的反轉是這週的核心命題:

傳統 workflow	AI agent workflow
Idea → Spec → Prototype → 痛苦修改	Rapid prototype → 真實 customer feedback → 凝結成 crystal-clear spec → 大量再生
Spec 寫得糊也能補救(人類工程師會問問題)	Spec 寫得糊 = code 寫得糊 (agent 不會主動 challenge)
Code = single source of truth	Spec = single source of truth , code 是 spec 的可重生產出
Code review 抓 bug	Spec review 抓 bug (一行 spec 錯 = 100 行 code 錯)

對 PM, designer 也有解放性影響 — 過去想改一個 button 行為要排工程 sprint, 現在寫得清楚的 spec 直接餵給 Claude Code 就能落地。Mihail 在 Monday lecture 會花相當時間講 PRD (Product Requirements Document) 怎麼寫成「agent 看得懂的 spec」— 含 user story、acceptance criteria、guardrails、non-goals 四個 block。

5.2.3 三、IDE Integration 的演化: 從 autocomplete 到 manager mode

[Devin: Coding Agents 101](#) 把開發者 AI tool 的演化分成四代, 每一代的 IDE integration 都不同:

世代	時期	代表	IDE 整合方式
Autocomplete	~2015	TabNine、Kite	method 級補完,typing 旁邊跳建議
Copilot	~2021	GitHub Copilot	多行 inline suggestion, 按 Tab 接受
Generative chatbot	~2023	Cursor Chat、Claude.ai	檔案級 chat panel, 貼 code 改 code
Autonomous agent	2024-now	Devin、Claude Code、Cursor agent mode	end-to-end task 委派, 從 prompt 到 PR

每一代的 IDE 都在解決前一代的瓶頸: autocomplete 解決打字慢、copilot 解決 boilerplate、chatbot 解決「跨檔案 context」、agent 解決「多步驟自我 iterate」。但 agent 帶來新問題 — IDE 不再只是 editor, 要兼容 background task、long-running process、agent state visualization。Cursor 的 agent panel、Claude Code 的 CLI session、Devin 的 web UI 各有不同設計取捨。

5.2.4 四、Devin vs Claude Code 的設計哲學張力

Friday lecture 由 Cognition 的 Silas Alberti 開講,這節的張力預設是「Devin 的 autonomous-first 哲學 vs Anthropic Claude Code 的 interactive-first 哲學」。對比軸向:

軸向	Devin (Cognition)	Claude Code (Anthropic)
類羣介入	低 – 接 task 後 hours 級自跑	高 – 每個 tool call 都可被打斷
UI 形態	Web app + cloud sandbox VM	CLI / IDE plugin, 跑在 user 機器
錯誤恢復	自我 iterate test/CI 直到 green	多半丟回 user 決定
適合任務	scoped、有 clear acceptance criteria	exploratory、需 user judgment
Engi- neering manager mode	強調,sales pitch 是「同時 supervise 5 個 Devin」	弱, focus 在「你跟一個 Claude 對話」

Devin 強調 **architecture-first prompting** (講 *how* 不只 *what*)、**defensive prompting** (預先告訴 agent 哪些坑)、**feedback loop quality** (type / test / lint signal)。但作者明確降溫: 對大型任務 expect ~80% 時間節省,不是完全自動化。這個誠實的 ratio 是工業界共識。

[Writing Effective Tools for Agents](#) (Anthropic engineering blog) 則提供 tool design 的深度原則 – strategic selection (少而精)、meaningful naming (namespace prefix)、contextual response (semantic name 取代 UUID)、token-efficient implementation、descriptive specifications (像跟新進員工解釋)。這些 principle 是 Devin / Claude Code / Cursor 共通的 agent infrastructure layer。

5.3 Monday Lecture(10/6): From first prompt to optimal IDE setup

- Slides: [Google Slides 公開連結](#)
- Design Doc Template: [Drive 連結](#)
- 講者: Mihail Eric

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

Mihail 把 IDE 演化壓縮成一條 timeline, 再導出「為什麼 AI IDE 是 2023 年才可能」的論點。Lecture 的核心命題是: AI IDE 的 evolution 是「**functionality consolidation vs developer customization**」之間的 see-saw。

1. **IDE 簡史** – Turbo Pascal (1983) first true IDE → Microsoft Visual Studio (1997) first IntelliSense (autocomplete 前身) → IntelliJ IDEA (2001) contextual code navigation / refactoring → VS Code (2015) lightweight + 高度可擴充 ecosystem, 也引進 LSP (MCP 的概念祖先) → Cursor (2023) 第一個真正 AI-native IDE。
2. **Cursor 使用模式分層** – Bread-and-butter mode (Tab 簡單 inline, Cmd-K 幾行的 quick edit, Cmd-L multi-file), True AI-native (background agent、MCP integration、learn memories、Bugbot PR review)。
3. **AI IDE 底層運作** –
 - **Tab-complete**: 圍繞 cursor 的小 context window 加密送到 server, 跑 infilling LLM 後送回建議
 - **Chat**: 把 code chunk 存成 embedding 在 server 的 semantic index (檔名 obfuscated), query 時 retrieve 最相關 chunk 餵 LLM; IDE 定期 re-index 並用 Merkle tree 算 chunk diff 做 efficient update
4. **Best practice: simple change vs complex task** –
 - Simple change: 不必太用心 prompting
 - Complex task: 你要進入 PM 模式, 寫精雕的 specs doc, 包含: **Goal**(這次改動目的)、**Definitions** (LLM 需要的 prereq)、**Plan** (high-level breakdown)、**Source files being changed**、**Test cases**、**Edge cases**、**Out-of-scope** (什麼不要改)、**Extensions** (之後可能加什麼, 讓 LLM 不要走捷徑)

5. **Codebase optimization for human + agent** – 多數 LLM 混亂來自 messy repo。優化清單: repo orientation、file structure、setup & environment、best practices、code style、access patterns、APIs and contracts。建議 monorepo design。「A messy repo can be as simple as having different access patterns, variable naming, multiple functions that do the same thing.」
6. **Agent configuration files 對比** –
 - **CLAUDE.md**: Claude 自動 pull 進 context, 放 common bash command、core file、code style、testing instruction
 - **.cursorrules**、**AGENTS.md** (OpenAI Codex 用, [官方範例 repo](#)) – README 是給人看的 (quick start、project description、contribution guideline), AGENTS.md 是給 agent 看的 (build step、test、convention、code style、security)
 - **llms.txt**: 給網頁 scraping LLM 的導航
 - 重點 caveat: 「The agents won't always adhere to these descriptions/directives. They are intended as guidance.」這些檔案是建議不是強制
7. **Live demo 段** – 走一個帶 specs doc 的 complex coding task, 看 cursorrules / Claude Code spec 在實 repo 裡長什麼樣。

Key takeaway: Optimal IDE setup 不是「工具越多越好」 – 而是把 repo 整理到 human 跟 agent 都能讀得懂的狀態, 再配上一份對 task 量身寫的 specs doc。messy repo + sloppy prompt 是 LLM 寫不出好 code 的最大原因, 永遠先檢查這兩件事再去改 prompt 措辭。

5.4 Friday Lecture (10/10): Silas Alberty (Cognition)

- **Speaker:** [Silas Alberty](#), Founding Team @ [Cognition](#) (前 Stanford PhD Student)
- **Slides:** [Google Slides 公開連結](#)

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要。Silas 的演講題目是「**IDE ❤️ Agents: An opinionated guide to AI coding in 2025**」:

Silas 把 AI coding tool 演化分成三世代, 每一代的「效率提升」量級截然不同:

1. **Three Eras of AI Coding Tools**
 - **Era 1: Code Completion** (GitHub Copilot) – ~10% efficiency gain, speed up coding, 純 local
 - **Era 2: IDE Automation / AI IDEs** (Cursor、Windsurf) – ~20% gain, single-player task completion, 仍 local
 - **Era 3: AI Software Engineer** (Devin) – 6-12× efficiency gain, 移到 cloud + asynchronous, scale workflow in parallel
2. **Sync vs Async 的本質差異** –
 - **Sync**: single-threaded、human-in-the-loop、注意力鎖在單一 task → AI agent 工作 20 秒到 1.5 分鐘
 - **Async**: multi-threaded、人類 delegate 給 AI、注意力可在多 task 間切換 → AI agent 工作 10 分鐘到數小時
 - **Cloud + Async 解鎖 10× parallelism**: 每個工程師可同時跑多個 Devin instance (in VPC), 一對多、共享組織知識; 對比 Local AI IDE 是一對一、knowledge 隔離
3. **Devin 的 workflow autonomy 三階段** –
 - **Plan (engineer in the loop)**: 用 Search / Wiki 問 Devin 找 file / API / test → clarify scope → Devin 寫 step-by-step plan。耗時: 分鐘級
 - **Build (Devin in the loop)**: Devin 在 isolated workspace (shell + editor + browser) 執行 → plan → execute → test → iterate 直到 task 全綠。完全 async, 你可以去做別的事
 - **Review (engineer 回 loop)**: Devin 開乾淨的 PR 含 passing test 與 change summary → 你 review、comment 或 merge → Devin 可依要求修。Human effort ≤ 15% of original task
4. 「**Semi-Async**: 尷尬的中間地帶」 – 5 秒 / 10 秒 / 30 秒 / 1 分 / 3 分 / 5 分 / 10 分 / 1 時 / 3 時的 spectrum 上, 3-10 分鐘是 **Avoid** 區段: 太慢 stay in flow、太短 multi-task。要嘛做更快保持 flow, 要嘛用更多時間換更高 intelligence。

5. **Async agent 是 hard but learnable skill** — 雖然 async agent 解鎖 10× gain, 但多數人還在用 sync agent。原因: 管理 / 委派本身就是難 skill, 不論對象是人還是 agent。需要 cycle 多 task 的能力 + 快速理解新 context 的能力。
6. **Planning / Coding / Testing 三階段的 sync vs async 配置** —
 - 今天: planning sync (DeepWiki、Ask Devin、Codemap、DeepWiki in Windsurf) → coding async (delegate 給 agent) → testing sync (在 Windsurf 裡 refine)
 - 未來: testing 也會 async (agent 自己驗收), 那時 leverage 會跨另一個量級
7. **Common workflow demo:** (1) delegate task to Devin async、(2) test & refine changes in Windsurf sync — Devin 處理 long-form 工作, Windsurf 接手 last-mile sync 微調。
8. **Where are we headed:** human engineer 變成 agent manager, sync tool 解最難的問題、async tool 拿 10× leverage。對未來 valuable 的能力: (a) delegation & multi-threading、(b) code reading、(c) planning / scoping / architecting。

Key takeaway: Devin 的設計哲學不是「IDE 升級版」, 是把工程工作從 local sync 推到 cloud async 的範式轉移。6-12× 不是 marketing 數字 — 是 **parallelism × autonomy** 的乘積。但要拿到這個 gain, 工程師必須學會「同時 supervise 多個 agent」這項新技能, 這是比 prompt engineering 更難、更值錢的能力。

5.5 Reading 摘要

篇名	來源	一句話重點
Specs Are the New Source Code	Ravi Mehta blog	Code 是 intent 的 lossy projection, spec 才是 AI 時代的 single source of truth
How Long Contexts Fail	dbreunig.com	1M context ≠ 更好答案, 4 種 failure mode (poisoning / distraction / confusion / clash) 都會搞爛 agent
Devin: Coding Agents 101	devin.ai	Coding agent ≠ autocomplete / copilot / chatbot, 是 end-to-end 的不同物種, 工程師變 manager
Getting AI to Work In Complex Codebases (ACE-FCA)	github.com/humanlayer	Research → Plan → Implement 三段式 + frequent intentional compaction, brownfield codebase 工業級配方
How FAANG Vibe Codes	X / Twitter	原文需登入 X, best-effort 推測: 個人 vibe coding workflow 在大型 codebase 不能照搬
Writing Effective Tools for Agents	Anthropic engineering	Tool 不是 API — 少而精、namespace 命名、semantic output、actionable error, evaluation-driven 迭代

閱讀優先順序: 時間有限的話, 先讀 [ACE-FCA](#) (最 actionable, 三段式 workflow 直接套用) → [How Long Contexts Fail](#) (理解 failure mode 才知道為什麼三段式有效) → [Specs Are the New Source Code](#) (mindset 翻轉) → [Writing Effective Tools](#) (自己寫 MCP / skill 時必讀)。

5.6 Assignment: Build a Custom MCP Server

- **Source:** github.com/mihail911/modern-software-dev-assignments/blob/master/week3/assignment.md
- **任務描述:** 用 W2 學的 MCP SDK 實作一個 production-grade MCP server, 把外部資料源 (資料庫 / API / 檔案系統 / SaaS 工具) 包成 Claude Desktop / Cursor 可直接呼叫的 tool。重點是套用 [Writing Effective Tools](#) 的設計原則: strategic tool consolidation (不要把每個 endpoint 都 wrap 成 tool)、semantic output、descriptive specifications、actionable error message。建議的 server 主題: 你日常

會查的 API (個人筆記、研究資料庫、健保碼、PubMed、Google Calendar), 或把 W2 的 MCP server 升級成 production 品質。

- 自學者可行性: ★★★★★ 完全可做但比 W2 進階。需要 W2 的 MCP 基礎、能寫 TypeScript 或 Python、有想包的外部資料源 (自己的 PubMed query / Obsidian vault / 個人 API key)。預估 4-8 hr, 含寫 code + 在 Claude Desktop 實測迭代。

💡 沒有外部資料源也能做: 用 SQLite + 假資料 dump 模擬資料庫, 重點是練 tool consolidation 與 description 寫法 — design quality 不需要真實資料才能證明。

5.7 對 Vibe Coder 的應用

W3 是 vibe coder 從「會用 Claude Code」升級到「能規模化用 Claude Code」的關鍵一週。實戰建議:

1. 任何 **non-trivial** 任務先寫 1 頁 spec — 不寫的代價是反覆改 5 次都不對。Spec 含 4 個 block: (a) user 是誰、(b) 要做什麼、(c) 成功長怎樣 (驗收標準)、(d) 不做什麼 (non-goal)。5 分鐘 spec 換 2 小時 debug 是夯爆的 ROI
2. 採用 **Research → Plan → Implement** 三段式 — 別讓 Claude Code 一接 task 就開寫。先讓它 research (理解 codebase / 現有 pattern)、產 plan.md (你 review 完才放行)、最後 implement。人力放在 **plan review** 而非 **code review**, 因為一行 plan 錯 = 100 行 code 錯
3. **CLAUDE.md** 當 **code 維護** — 每行都問「刪掉會不會出錯」。過長會讓重要 rule 被淹 (Drew Breunig 的 context distraction)。建議結構: repo 在做什麼 (一段話)、命名慣例、業務 quirks、絕不做的事。其他 domain knowledge 走 skill 或 subagent, 別塞 CLAUDE.md
4. **Tool / MCP server** 不是越多越好 — Berkeley function-calling 數據顯示 tool > 30 個就開始崩。挑你真會用的 5-10 個 MCP server 就好。寫自己的 MCP 時, 把「3 個 low-level operation」合併成「1 個 high-level tool」, agent 友善太多
5. **Long session** 出現重複錯誤 = **context poisoning**, 立刻 **/clear** — 不要繼續解釋, 會越解越糟。重啟, 把正確結論寫進 CLAUDE.md / spec, 再開新 session 跑一次
6. **Subagent** 處理 **expensive exploration** — grep、找檔案、跑長 search 派 subagent 跑, 主對話只接 summary。Claude Code 的 Task tool 直接支援, 這招會讓主 session 的 context 維持 40-60% 健康水位

💡 **vibe coder 的 Day-1 Quick Win:** 今天就在你最常用的 side project 試一次完整三段式: (1) 寫 1 頁 spec.md (含 user story + acceptance criteria), (2) **/clear** 開新 session、(3) 對 Claude Code 說「先讀 spec.md, 產出 plan.md, 先不要 implement」、(4) review plan、(5) 確認後說「按 plan 執行」你會發現一次到位的 ratio 從 30% 跳到 80%。從此回不去舊 workflow。

上一週: [W2 The Anatomy of Coding Agents](#) | 下一週: [W4 Coding Agent Patterns](#)

6. Week 4: Coding Agent Patterns

本週你會學到什麼: 當你用 Claude Code 寫東西時, agent 應該多自動 vs 你應該插手到什麼程度?
本週講 agent autonomy levels (從「打 autocomplete」到「全自動接管」5 個層級) 和 human-agent collaboration patterns。Friday 由 Claude Code 的創造者 Boris Cherny 親自開講。

學習目標

6.1

完成本週後, 你應該能:

1. 辨識 agent autonomy 5 個層級 (suggest → preview → execute-with-review → autonomous-with-checkpoints → fully autonomous)
2. 應用 Claude Code best practices (CLAUDE.md、custom slash commands、hooks、subagents)
3. 設計 human-agent handoff 流程 (什麼任務該全自動、什麼該停下來問)
4. 評估 Claude Code、Cursor agent mode、Codex 在 agent management 哲學上的差異

6.2 核心概念導讀

6.2.1 一、Agent Autonomy 5 個層級 — 從 autocomplete 到 fully autonomous

W3 用 Devin 的 4 代分類描述了「世代」演化 (autocomplete → copilot → chatbot → agent)。Week 4 把鏡頭拉到「同一個 agent 在不同任務下, autonomy 應該調到哪一級」— 因為實務上你不會永遠用同一個 mode。Mihail 在 lecture 會用 5 個層級的 autonomy spectrum 教這件事:

Level	名稱	介入頻率	適用任務
L1	Suggest	每 token / 每行	補完一段 boilerplate、寫 docstring
L2	Preview	每 diff	改一個 function、加一個 test
L3	Execute-with-review	每個 tool call 前 ack	跨檔案 refactor、跑 destructive command
L4	Autonomous-with-checkpoints	每 phase 邊界 review	feature implementation、含 plan / test / commit
L5	Fully autonomous	task end review only	scoped bug fix、依 clear acceptance criteria 跑

關鍵 insight: **autonomy** 不是越高越好 — 反而要依任務類型動態調整。Exploratory (不知道答案、需要 user judgment) → L2/L3; scoped 且有 verification (type / test / lint 全綠就算對) → L4/L5。Claude Code 的 default 在 L3 (每 destructive command 問), auto mode 把它推到 L4, 背後是 classifier 自動審 risky operation。Devin 的 default 是 L5。Cursor agent mode 是 L4。

從 [Devin: Coding Agents 101](#) 來的 80% rule 在這裡發揮: 對大型任務 expect ~80% 時間節省, 剩下 20% 是 you-as-manager 的 review / re-direct 成本, 跟 autonomy level 無關。Level 拉得再高也省不掉這 20%。

6.2.2 二、Anthropic 自己怎麼用 Claude Code

[How Anthropic Uses Claude Code](#) 是這週最 grounded 的 reading — 不是 marketing pitch, 是 Anthropic 內部 10 個團隊的真實使用報告。共通 pattern 有三個:

1. 非技術人員大量使用 — finance、legal、growth marketing、product design 都是 power user。他們的工作流是「用 **plain text** 描述流程 → 餵給 **Claude Code** 自動執行」。Legal 團隊用它生成法規 mapping、growth marketing 用它跑 cohort analysis、design 用它把 Figma spec 翻成 production code
2. **Screenshot-driven debugging** — Kubernetes pod IP 耗盡、Google Cloud UI 導覽都靠截圖 + 自然語言描述就能 diagnose。這不是 demo，是 on-call engineer 真實 workflow
3. **Skill-gap bridging** — 工程師可以 vibe-code 出設計稿、設計師可以改 production code、legal 自動生成法規 mapping。過去要排工程 **sprint** 的事，現在自己用 **Claude Code** 跑

對 CS146S 學生的核心訊息：Claude Code 不是「coding assistant」是「team multiplier」。瓶頸從「會不會寫 code」轉成「能不能描述清楚需求」— 這正好接回 W3 的 spec-driven 命題。

6.2.3 三、Claude Code 的 4 種擴充 primitive: CLAUDE.md / Skill / Subagent / Hook

[Claude Best Practices](#) 把 Claude Code 的擴充機制整理成 4 種 primitive，每個解決不同層級的問題。理解這個分工是用好 Claude Code 的關鍵：

Primitive	何時 load	解決什麼問題	用途範例
CLAUDE.md	每次對話 auto load	persistent context (永遠該記得的事)	repo 命名慣例、業務邏輯、絕不做的事
Skill	按需 load (trigger 詞 / 路由)	domain knowledge / workflow	NCCN guideline、IRB 表格、統計 pipeline
Subagent	spawn 獨立 context window	context isolation (避免污染主對話)	grep 大 codebase、跑 long-horizon research
Hook	deterministic trigger (pre/post tool)	必須每次發生的動作	lint、block migration 寫入、auto git commit

重要原則 — 「必須每次發生」走 **hook**，不是 **CLAUDE.md**。**CLAUDE.md** 是 prompt 給 model 看，model 偶爾會忽略；hook 是 deterministic script，harness 強制執行「想做但不確定」走 skill / subagent，按需 load 不污染主 context。

[Peeking Under the Hood of Claude Code](#) 用 LiteLLM proxy 攔截 Claude Code 真實送出的 API call，逆向工程出 4 個 internal pattern: (1) **context front-loading** session 開頭主動濃縮對話 / 判斷主題切換、(2) **<system-reminder>** tag 散布在多層 prompt 抵抗 long-context drift、(3) **prompt-based safety** 用專門 prompt 偵測 command injection 而非 hardcoded rule、(4) **conditional sub-agent reminder** 預設不注入 reminder 保持 focus、行為偏離才注入。結論：Claude Code 的成功不是 model 優勢，是系統化的 context scaffolding。

💡 譯解：Claude Code 的「魔法」其實全是 prompt engineering 紀律 — 每個 session 開頭都把目標壓成 50 字 title、每個 tool result 都包進 system reminder、每個 subagent 都隔離 context。這意味著你在自己的 long workflow 上也能模仿 — 在關鍵步驟前手動重述目標，比期待 model 自己記得可靠。

6.2.4 四、Community Framework 的兩種路線: specialized agent zoo vs meta-framework

社群已經沉澱出兩種主流的 Claude Code 擴充路線。

路線 1: **Specialized agent zoo** ([Awesome Claude Agents](#)) — 把 24 個 specialized subagent 分四層編成「虛擬開發團隊」: (1) Orchestrators 3 個(Tech Lead、Project Analyst、Team Configurator)做路由、(2)

Framework Specialists 13 個(Laravel、Django、Rails、React、Vue 各自的 backend / API / ORM 三組)、(3) Universal Experts 4 個跨棧、(4) Core Team 4 個(code reviewer、performance optimizer、documentation specialist、code archaeologist) 做 QA。內建 auto-configurator 偵測 stack 自動派 specialist。命題是「specialized expertise + division of labor 勝過 solo Claude」。

路線 2: **Meta-framework (SuperClaude)** – 不是換 agent, 是在 Claude Code 上層疊加 30 個 slash command + 20 個 specialized agent + 7 個 behavioral mode + 8 個 MCP server。Behavioral mode 是 SuperClaude 最獨到的概念 – 不是 agent 也不是 command, 是「整段對話的操作脈絡」(Brainstorming Mode 讓所有後續回應變成 Q&A 探索風格、Token-Efficiency Mode 強制簡短)。設計哲學強調「framework footprint 要小, 把 context window 留給 project code」。

兩條路線的 trade-off: specialized agent zoo 邊界清楚但 maintenance 成本高 (24 個 agent 要維護); meta-framework 整合度高但 lock-in 強(會跟你既有 CLAUDE.md / skill 衝突)。

Good Context Good Code (標題即論點) – agent 寫不好 code, 瓶頸不在 model 而在 context 品質。寫不好時先別罵 model 也別重寫 prompt – 先問三個問題: (a) 我有給它 reference file 嗎? (b) 我有給它 verification (test / expected output) 嗎? (c) CLAUDE.md / skill 裡有沒有相關 domain knowledge? 三個都做了還錯, 再考慮其他原因。這跟臨床 history-taking 同構: 問診 (context) 做不好, 鑑別診斷 (output) 一定不會準。

6.3 Monday Lecture (10/13): How to be an agent manager

- Slides: [Google Slides 公開連結](#)
- 講者: Mihail Eric

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

Mihail 用一張軟體團隊演化簡史鋪 mindset shift: solo developer (1960) → 軟體團隊出現、開始專業化(NASA / DoD 推動, 1970-1990) → 主流軟體團隊 (2000s) → AI 輔助的開發團隊(2023) → 每個工程師都是 tech lead, 管自己的一支 agent 軍團 (2025+)。Lecture 的核心命題是: 未來每個 developer 都會像 tech lead 一樣, operate 一支由不同專長 agent 組成的隊伍 – 處理 PR / QA、跨語言 (React / FastAPI / Svelte)、跨角色 (data engineer / ML engineer / DevOps)。最後甚至中等技術的 PM 都能做這件事。

1. **Software task** 的責任分配演化 – 把典型工作流標 ■ (人) / ■ (agent):
 - Provide high-level requirements ■
 - Convert requirements into design doc ■ / ■
 - Implement solution from doc ■
 - Add tests ■
 - Ensure CI passes ■
 - Code review ■
 - Update docs ■
 - 人類愈來愈集中在 requirements 與 design alignment, 其餘交給 agent
2. **Directing agent** 的四個 **primitive** (後面 Boris 會深入講) –
 - **Agent behavior file** (CLAUDE.md / cursorrules / AGENTS.md): 定義 agent 整體行為的 anchor
 - **Hooks**: deterministic script 在 PreToolUse / PostToolUse / UserPromptSubmit / PreCompact 等預定事件觸發
 - **Commands**: 常用 prompt 存成檔, agent 可執行 (例: 跑 test、review code、git commit + push、Claude Code 的 ship-it command)
 - **Subagents**: runtime delegation, 建立不同 persona (frontend / backend)、cleanly 隔離 context、自帶 system prompt + tool + 獨立 context window; 目的是 agents managing other agents。可參考 [awesome-claude-agents](#)、[SuperClaude Framework](#)
3. **Best practice**: 你需要小心的安全網 (careful backstops)
 - Codebase 內要有 test
 - CI/CD best practice 落實

- 每個 agent 的可稽核性 (auditability)
 - 每個 diff 都標是哪個 agent 做的
 - 不同 task class 用不同 model (Opus 拿來 plan、Sonnet 當 workhorse)
 - 複雜 task 前期多花一點時間 hand-hold; fully async 任務則放手
 - 經常 commit 做 checkpoint
4. **Live workflow walkthrough** – Mihail demo 在實 task set 上看 agent 怎麼被 spin up、用 Claude proxy 觀察 LLM I/O。
 5. **Open question** – 怎麼自動化每個 task 開頭的 10-20% research phase? 怎麼維護 pending task queue (特別是一次性小改動)?

Key takeaway: Agent manager 不是技術升級，是身分升級。寫 code 從「pair programming」變成「delegate + audit」。能不能在 fully async 任務放手、能不能對複雜 task 多 hand-hold 一點 – 這個分寸感比 prompt engineering 更決定一個 vibe coder 的天花板。Best practice 裡最被低估的一條是「Opus 用來 plan、Sonnet 用來 implement」 – 這個分工比同一 model 跑全程便宜又準確。

6.4 Friday Lecture (10/17): Boris Cherny (Claude Code creator)

- **Speaker:** [Boris Cherny](#), Creator of [Claude Code](#)
- **Slides:** [Google Slides](#) [公開連結](#)

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要。Boris 演講 tl;dr 是「**programming is changing → choose your path with claude code → think six months out**」:

1. **Programming is at an inflection point** – Boris 用一張 $\log(\text{productivity})$ vs year 圖把過去 70 年的程式語言演化攤開: Fortran → Algol → Cobol → BASIC → C → Pascal → Prolog → C++ → Python → Java → JS → Go → Rust → Haskell → Swift → TypeScript。productivity 是 exponential growth，現在被 AI 推到第二段斜率。同時間 IDE 也在 exponential: ed → emacs → vi → Turbo Pascal → QBasic → VB → Eclipse → IntelliJ → Sublime → Cursor → Copilot → Devin → Claude Code。
2. **History 速覽** – IBM 029 (1964 punch card) → ed (1969, Ken Thompson) → Smalltalk-80 (1980 第一個 GUI IDE, live editing) → Visual Basic (1991 第一個主流 visual editor) → Eclipse (2001, 3rd-party plugin ecosystem, rich autocomplete) → Copilot (2021, multi-line AI autocomplete) → Devin (2024, conversation-first, 不再直接操作 code)。IDE DevX 演化加速中，下一個十年只會更快。
3. **Verification 也在 evolution** – manual debugging → static type (Algol) → formal verification → abstract interpretation → automated testing → CI → property-based testing (QuickCheck) → dependent typing → e2e testing → chaos testing (Chaos Monkey) → AI-powered vulnerability testing → AI-powered unit testing (TestGen) → AI-powered fuzz testing (Sapienz) → self-play。
4. **Claude Code 的 approach** – 三條核心設計原則:
 - **Works everywhere:** terminal-native, 所有 dev tool 都已經跑在 terminal
 - **Low-level model access:** 直接 expose model 能力, 不過度抽象
 - **Infinitely hackable:** 使用者可在任何層擴充
5. **Works across the whole SDLC** – 五階段全覆蓋: (1) Discover – 探 codebase / 看 git history / 搜文件 / onboard, (2) Design – plan project / 寫 tech spec / 定 architecture, (3) Build – implement / 寫 + 跑 test / 開 commit & PR, (4) Deploy – 自動 CI/CD / 設環境 / manage deployment, (5) Support & Scale – debug error / large-scale refactor / 監控 usage & performance。「使用並掌握你 team 所有 CLI tool (git / docker / bq) 以聚焦在解法而非 syntax」。
6. **One code, many faces** – 同一個 Claude Code 有多種 surface: terminal、IDE、web & iOS、/ install-github-app、SDK。SDK 範例:

```
claude -p "what did i do this week?" --allowedTools Bash(git log:*) --output-format stream-json
get-gcp-logs luhd832d | claude -p "correlate errors + commits" --output-format=json
| jq '.result'
```

底層 model 同時支援 Anthropic / Bedrock / Vertex API。

7. 四個 use case —

- **Codebase Q&A + research:** 例 「 how do I make a new `@app/services/ValidationTemplateFactory?` 」 「 why does `recoverFromException` take so many arguments? look through git history 」 「 why did we fix issue #18363 by adding the if/else in `@src/login.ts?` 」 「 what did I ship last week? 」
- **Write code 三模式:** 1-shot / sidekick / prototype
- **Integrate tools & MCPs:** `claude mcp add barley_server -- node myserver` 然後 > use the barley mcp server to check for error logs
- **Power automation**

8. Fit the workflow to the task — 三條 demo workflow:

- **explore > plan > confirm > code > commit:** 「 figure out the root cause for issue #983, then propose a few fixes. Let me choose an approach before you code. **ultrathink** 」
- **tests > commit > code > iterate > commit:** 「 write tests for `@utils/markdown.ts` to make sure links render properly (note tests won't pass yet). then commit. then update the code to make tests pass. 」 (test-driven)
- **code > screenshot > iterate:** 「 implement [mock.png]. Then screenshot it with puppeteer and iterate till it looks like the mock. 」 (visual feedback loop)

9. Prototyping 範例 — Boris 秀一連串 「 actually, what if... 」 prompt 展示用 Claude Code 對 todo UI 做 8-10 輪 iteration, 從 inline 顯示 → fixed list above input → pill 形式 → spinner merge → ctrl+T 展開 — 全在自然語言中完成 UI 設計探索。

10. Lessons —

- **Build for the model six months from now** (最重要) — 不要對著今天的能力做 product, 對著六個月後的能力做
- **Be ready to evolve**
- **Ask not what the model can do for you** (而是問你能怎麼配合 model)
- **Models get better, compute gets cheaper**

Key takeaway: Boris 的核心訊息是「對六個月後的 model 設計你的工作流」。今天的 Claude Code 是 opinionated primitive — 把 leverage 留給 model 而非 harness、CLI 而非 IDE plugin、infinitely hackable 而非 turnkey。理解這個 thesis 才看得懂為什麼它跟 Cursor / Devin 走完全不同的路 — 不是「比較好的 IDE」, 是「比較少的 IDE」。

6.5 Reading 摘要

篇名	來源	一句話重點
How Anthropic Uses Claude Code	Anthropic PDF	內部 10 個團隊(含 legal、design、growth marketing) 都用 Claude Code, 是 team multiplier 不是 coding assistant
Claude Code Best Practices	Anthropic engineering	4 種 primitive (CLAUDE.md / Skill / Subagent / Hook) + Explore→Plan→Implement→Commit 工作流
Awesome Claude Agents	github.com/vijaythe-coder	24 個 specialized subagent 編成「虛擬開發團隊」(orchestrator + specialist + universal + core team)

篇名	來源	一句話重點
SuperClaude work	Frame-work github.com/Super-Claude-Org	Meta-framework: 30 command + 20 agent + 7 mode + 8 MCP 疊加在 Claude Code 上層
Good Code	Context, Good Stock app blog	Code quality $\propto$ context quality; agent 寫不好先檢查 context, 不是 prompt 措辭
Peeking Under the Hood of Claude Code	OutsightAI Medium	LiteLLM 攔截逆向工程出 4 個 pattern (front-loading / <system-reminder> / prompt-based safety / conditional sub-agent)

閱讀優先順序: 先讀 [How Anthropic Uses Claude Code](#) (看真實使用樣貌建立期待) → [Claude Code Best Practices](#) (4 primitive 是核心教科書) → [Peeking Under the Hood](#) (理解設計哲學) → [Good Context, Good Code](#) (debug mindset) → [Awesome Agents](#) / [SuperClaude](#) 兩個 community framework 看興趣選讀。

6.6 Assignment: Coding with Claude Code

- **Source:** [github.com/mihail911/modern-software-dev-assignments/blob/master/week4/assignment.md](#)
- **任務描述:** 在你選的 repo 裡完整實踐 Claude Code 4 種 primitive: (1) 寫一份精簡 CLAUDE.md (每行可審), (2) 建一個 custom slash command (`.claude/commands/<name>.md`), (3) 寫一個 subagent definition (`.claude/agents/<name>.md`) 做 expensive search, (4) 設一個 hook (`.claude/settings.json`) 強制 lint 或 block 危險命令。再用這套配置完成一個 non-trivial 的 feature 或 refactor, 產出含 `plan.md` + `verification.md` + 最終 PR 的完整 trace。重點是體驗「Explore → Plan → Implement → Commit」工作流的 friction 與 ROI。
- **自學者可行性:** ★★★★★ 100% 可做。Claude Code 有 free tier, 配 GitHub free repo 就能跑完。預估 4-6 hr, 含 setup (1 hr) + 寫 4 種 primitive (1.5 hr) + 跑完整工作流 (2.5 hr)。

💡 不知道用什麼 repo 練? 拿你既有的 side project (哪怕只有 50 行 code) 就好。重點不是 codebase 大小, 是把 4 個 primitive 各寫一個跑過一次的肌肉記憶。沒 side project 的話, 從 scratch 開個 todo list app 也行。

6.7 對 Vibe Coder 的應用

W4 是 vibe coder 整個 10 週課程裡 ROI 最高的一週 — 把 Claude Code 4 個 primitive 練熟, 後續 6 週的所有工具 (Modern Terminal、CI/CD、Production Ops) 都會省一半時間。實戰建議:

1. **CLAUDE.md** 三層結構 — 不是流水帳, 要有結構: (a) repo 在做什麼 (一段話), (b) 絕不做的事 (最重要、列 5-8 條), (c) 命名 / 風格慣例。每行都問「刪掉會不會出錯」— 會的留下、不會的刪。Boris Cherny 自己在 podcast 講過 Anthropic 內部 CLAUDE.md 平均才 30-50 行, 越短越有效
2. **Custom slash command** 從「你最常打的同一段 prompt」開始 — 你發現自己每次都打「先讀 X、再分析 Y、最後輸出 Z」? 把它寫成 `.claude/commands/<name>.md`, 下次 `/<name>` 就跑完。對醫學研究 workflow 特別有用 — 「`/table1`」「`/km-curve`」「`/oe-search`」這類重複任務都該是 slash command
3. **Subagent 處理 expensive 探索** — 主對話開始爆 context 前就派 subagent。三個 typical use case: (a) 探索陌生 codebase (`code-archaeologist` 風格 agent), (b) 跨 repo grep + summarize, (c) long literature search。主對話只接 subagent 的 summary, 乾淨太多
4. **Hook 處理「必須每次發生」的事** — 不要把「commit 前要跑 lint」寫進 CLAUDE.md (model 會忘), 寫成 PreToolUse hook 強制執行。常見 hook: (a) auto git commit after Edit, (b) lint after Write, (c) block dangerous command (`rm -rf / git push --force / DROP TABLE`), (d) auto-format on save。Hook 是 deterministic, 永遠會跑
5. **Skill 系統 = domain knowledge as plugin** — 比 CLAUDE.md 更進階的 reuse 機制。你已經有的 `clinical-scores` / `openevidence` / `kaplan-meier` 都是 skill 範例: trigger 詞觸發、按需 load、不污染

預設 context。寫 skill 的甜蜜點: (a) 跨 project 重複用、(b) 有明確 trigger、(c) 需要 domain knowledge 才寫得對

6. **Community framework** 選擇性引入, 不要無腦 install – [SuperClaude](#) / [Awesome Agents](#) 有 inspiration 價值但會跟你既有 CLAUDE.md / skill 衝突。建議: 抄 1-2 個你會真的用的 agent / command 進自己的 `.claude/`, 不要 full install

Vibe coder 進階一級的徵兆: 當你發現自己花在「寫 spec + plan + 設 verification」的時間 > 「打 prompt」的時間 – 你已經是 agent manager 不是 pair programmer 了。這個 ratio 翻轉是 W4 想灌輸的核心 mindset shift。

💡 **vibe coder** 的 Day-1 Quick Win: 今天就在你最常用的 repo 加 `.claude/commands/plan.md`, 內容寫「請讀 spec.md (如有), 分析 codebase, 產出 plan.md (含 file-level step + verification 策略), 先不要 implement」。下次任何 non-trivial task 開始前打 `/plan`, 跑完 review, 再說「按 plan 執行」3 行 markdown 就把你的工作流升級到 ACE-FCA 三段式。比裝整套 SuperClaude 有效 10 倍。

上一週: [W3 The AI IDE](#) | 下一週: [W5 The Modern Terminal](#)

7. Week 5: The Modern Terminal

本週你會學到什麼：傳統 terminal (bash/zsh) 是 1970 年代設計，2025 年的 AI terminal (Warp、Wave、Claude Code CLI、Codex CLI) 長什麼樣？本週講 AI-enhanced CLI 與 terminal automation。Friday 由 Warp CEO Zach Lloyd 親自開講。

學習目標

7.1

完成本週後，你應該能：

1. 辨識 AI terminal 與傳統 terminal 在 input、output、history、collaboration 四個維度的差異
2. 應用 Warp 的 AI features (natural language → command、AI agents、workflows、shared workspaces)
3. 比較 Warp、Claude Code CLI、Codex CLI 三種 CLI agent 的設計取捨
4. 設計一個 terminal-first 工作流 (含 AI-assisted scripting、history search、命令解釋)

7.2 核心概念導讀

7.2.1 一、Terminal 演化簡史 — 為什麼 1970 年代的設計會撐到現在

要理解 modern terminal (如 Warp、Wave、Ghostty) 為什麼會冒出來，得先看清楚我們現在用的這個 terminal 是什麼東西。今天你打開 macOS 內建的 Terminal.app 或 iTerm2，看到的本質上是 **VT100 terminal emulator** — 一個模擬 1978 年 DEC VT100 硬體終端機的軟體。VT100 的設計目標是讓打字機式硬體裝置能跟 mainframe 通訊：input 是一行 text、output 是一行 text，所有狀態存在「螢幕緩衝區 (screen buffer)」這個一維 character grid 裡。

這套設計撐了 50 年沒被換掉的原因有兩個：(1) 它「夠用」— 工程師寫 shell script、跑 git、看 log 都不需要更花俏的 UI；(2) 生態鎖死 — bash、zsh、vim、tmux、ssh、所有 CLI tool 都假設輸出是 line-buffered text、控制是 ANSI escape code，要換 terminal 就得讓上萬個既有工具相容。所以即使 GitHub 在 2017 推出 Hyper、Microsoft 在 2019 推出 Windows Terminal，本質都還是「比較好看的 VT100」。

真正的轉折點是 2022 年的 **Warp**。Warp 第一次在 terminal 層做兩件激進的事：(a) 把 line-buffered output 重構成 **block** (每條 command + output 是一個獨立物件，可選取、複製、分享、搜尋)，(b) 把 LLM 直接縫進 input editor (自然語言轉 command、執行錯誤即時解釋)。這讓 terminal 從「字符串 I/O 通道」進化成「結構化 agent runtime」。後續 Wave、Ghostty、Claude Code CLI、Codex CLI 都在這個賽道上展開。

💡 譯解：你可以把傳統 terminal 想成「醫院 1970 年代的紙本病歷」— 一切都是 free-text、沒結構、難搜尋；modern terminal 就像「電子病歷系統」— 每個 progress note 是有 metadata 的物件，可以 query、tag、跨病人比對。介面長得像但底層資料結構完全不同。

7.2.2 二、AI Terminal 的四個價值主張

Warp University 把 Warp 自我定位從「modern terminal」升級為 **Agentic Development Environment** (代理式開發環境，ADE)。對使用者來說，AI terminal 比傳統 terminal 多出四個 affordance：

維度	傳統 terminal	AI terminal (Warp 為例)
Input	單行 text、按 ↑ 翻 history	Natural language → command、Ctrl+G 多行編輯、@ mention 檔案、語音輸入
Output	Line-buffered text、ANSI escape	Block (可分享、搜尋、附 metadata)、agent 自動解釋 error

維度	傳統 terminal	AI terminal (Warp 為例)
History	Per-shell .zsh_history 檔	Cloud-synced、可 query 「上週那個 docker build 怎麼下的」
Collabo-ration	截圖丟 Slack	Block 一鍵分享 link、shared workspace、cloud agent 跑非同步任務

關鍵概念是 **block-based terminal** — 每條 `command + output` 是一個 first-class object, 不再是 line buffer 裡的一坨 text。這個 mental model 變化的影響極大: 你能對單一 block 做「重跑」「分享」「丟給 agent 解釋」「轉成 workflow 模板」等動作, 這些在傳統 terminal 裡需要先把輸出複製出來、貼進別處才能做。

Warp University 還提出 **Warp (local terminal) + Oz (cloud agent orchestration)** 的雙產品架構: local agent 是 interactive、人在 loop 裡審 diff; cloud agent 是 background、被 webhook / cron 觸發、可大規模並行(PR review、issue triage、dependency update)。兩端共用同一個 agent core 與 **Warp Drive** (rules / prompts / env var / workflow 的 persistent context layer), 所以「雲端開工、local 接手」是 seamless 的。

7.2.3 三、CLI Agent vs IDE Agent — 設計取舍

W3 講 IDE agent (Cursor、Continue.dev), W4 講 CLI agent (Claude Code、Codex), W5 把兩個放在同一個 terminal 裡。三種 agent 的取舍可這樣比:

設計面	IDE Agent (Cursor)	CLI Agent (Claude Code)	Terminal Agent (Warp Agent Mode)
Host	VS Code fork	任何 terminal	Warp 本身
Con-text source	Editor open files、selection	CLAUDE.md + 工作目錄	Codebase index + WARP.md + Warp Drive + MCP
互動模式	在 editor 旁邊 chat panel	純 CLI prompt	Block-based、Ctrl+G 多行、 <code>⌘+I</code> mode 切換
Tool access	Editor command + shell	Shell + file system	Shell + file system + 整套 Warp affordance
長 task	中等 (chat panel 易斷)	強 (CLI 天生 async)	強 (cloud agent 可背景跑)
Multi-file refactor	強 (diff view 內建)	中 (需 review CLI diff)	強 (內建 code review panel)

Warp vs Claude Code 文中講得很直白: Warp 不把 Claude Code 當對手而是當 first-class CLI agent host — 你可以在 Warp 裡跑 `claude`, 自動拿到 rich input editor、agent notification、inline code review、vertical tabs 等 IDE 級 affordance。或者你也可以直接用 Warp 內建的 Agent Mode (`⌘+Enter`)。最關鍵的 migration 提示: 把 `CLAUDE.md` 改名為 `AGENTS.md` 或 `WARP.md` 放專案 root, Warp 會自動讀取, 不必重寫。

7.2.4 四、為什麼 Vibe Coder 該認真考慮換 Terminal

How Warp Uses Warp to Build Warp 揭露 Warp 公司內部的 **Coding Mandate** — 每個 coding task 都從在 Warp 裡 prompt 起手, 連續卡關 10 分鐘才能 fallback 到別的工具。這條規則對個人開發者太嚴格, 但精神對非資工背景的 vibe coder 反而更重要:

1. **Prompt-first** 強迫你想清楚需求 — 用 natural language 描述「我要的是什麼」比直接寫 code 更容易發現需求模糊

2. **Block** 化讓你能回頭審視 — 不再是滾動的 line buffer, 每個操作都是可被 review、re-run、share 的物件
3. **Persistent context** 取代碎片記憶 — Warp Drive 的 Rules / Prompts / Workflow 等同 Claude Code 的 CLAUDE.md, 重複指令不必每次重打
4. **Cloud agent** 解放等待時間 — 大 task (跑完整 test suite、build、deploy) 丟去 background, 不必盯著 terminal

對台大醫學系背景轉 vibe coder 的讀者, 這個對應關係特別有用: Warp Drive 之於 terminal, 等於 RemNote 之於知識管理 — 都是「把碎片化的 context 沉澱成可重用 artifact」。學會一邊就能類推另一邊。

7.3 Monday Lecture(10/20): How to Build a Breakout AI Developer Product

- Slides: [Google Slides 公開連結](#)
- 講者: Mihail Eric (用 Warp 當 demo 工具)

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

這節從產品策略角度拆「為什麼 AI dev tool 在 2024-2025 出現史上最快的採用速度」。Mihail 用 7 條產品設計原則作為主軸, 每條都對應 Warp 等成功 AI dev tool 的具體決策:

1. **Start with what developers know** — 從既有 interface 過渡而非另起爐灶。每個 segment 都接到 developer 已熟的 metaphor: Cursor 接 IDE、Warp 接 terminal、Bolt 接 chat。範例: [explainshell.com](#) 是把 `tail -f /etc/passwd` 這種公認痛點當切入點, 逐步把使用者從 raw command line 解放, 能在 code 與自然語言間自由切換。
2. **Configuration flexibility** — 對 average user 零設定就能展示價值; power user 拿到完整 configurability: seamless 切換 model、prompts、project rules、MCP、Warp profiles per user。
3. **Focus on developer ergonomics** — 「If you can shave off a single keystroke, do it.」keyboard hotkey、零 onboarding friction (「5 minutes to WOW」是 Warp 內部目標)、tab / enter 等小動作的黏著度。
4. **Chat as a first-class citizen** — Code 本來就是 human intent 的 contrived representation, 工具進化到一定程度後, 更多互動會直接變自然語言。Demo Warp 內 debug error 流程就是用聊天替代 grep。
5. **MCP integration** — MCP 已成為 LLM 與真實世界互動的 lingua franca。可擴展工具生態給 LLM 任何資源 / 動作。Demo: 用 context7 給 Warp 抓最新 BrainTrust 文件。
6. **Rapid feedback loops** — 改完立刻看到效果。Bolt / Lovable 的 real-time canvas、Warp 的 take action + immediate reaction 都在這個範式。可解釋性是 first-class citizen。「entire projects committed to just reducing build time」是這條原則的最強佐證。
7. **Agent workflows** — 對實質任務給 full autonomy。「agent-take-the-wheel」的接受度逐月上升, 但要支援不同等級的 human-in-the-loop (agent 問 clarifying question、YOLO mode)。Demo Warp 的 YOLO mode。

最後 Mihail 拋出未來 open question:

- 會不會看到大規模整合? code review / app building / monitoring 等 point solution 會被 all-in-one platform 吞掉嗎?
- AI IDE 會不會演化成 AI terminal、再演化成 AI browser-replit?
- 像 Warp / Cursor 會不會 verticalize 變成「full-stack 開發專用」之類?
- 給 coding agent 個人化 / project-specific rule 的 universal standard 會是什麼? 目前 fragmented (`.cursorrules` / `CLAUDE.md` / `AGENTS.md`), `AGENTS.md` 是首個 step in that direction。

Key takeaway: AI dev tool 爆紅不是「把 chatbot 黏在編輯器旁邊」 — 它需要從 (1) 接既有 mental model 開始、(2) 對 power user 開全部 configurability、(3) 偷每一個 keystroke 的 ergonomic 紀律、(4) 把 chat 當 first-class、(5) MCP 接通生態、(6) feedback loop 越快越好、(7) 對實質任務放 autonomy 同時保留人類介入點。這 7 條原則是 vibe coder 評估「該不該換工具」的 checklist。

7.4 Friday Lecture (10/24): Zach Lloyd (Warp CEO)

- **Speaker:** [Zach Lloyd](#), CEO of [Warp](#)
- **Slides:** [Figma Slides \(公開連結 \)](#)

以下基於 slide deck 公開內容 (PDF 39 張 slide vision OCR) 整理的繁中摘要。Slide 結構為六段：
(1) The world is changing、(2) From development by hand to by prompt、(3) What is Warp、(4) Warp tour、(5) Using agents to go from prompt to production、(6) The business of agentic development、最後 Q&A。

1. **論點主軸 — 從 by-hand 到 by-prompt 的 dev paradigm shift:** Lloyd 開場用 Greylock 的引言定錨「為 code generation 與 engineering workflow 解鎖 high-fidelity、reliable AI 是巨大機會」。整場演講的核心張力是「**Current: Development by hand** → **Future: Development by prompt**」，而 prompt-driven development 又分兩支：使用者主動觸發的 **interactive agent**，以及由 webhook / cron / event 觸發的 **ambient agent**。Warp 的賭注是同時做好這兩端的開發者體驗，把產品定位為 **ADE (Agentic Development Environment, 代理式開發環境)**。
2. **為什麼是 terminal — 三個結構性理由:** Lloyd 認為 command line 是 agent 最自然的住所。理由：
(a) **Lowest tool in the dev stack** (在開發堆疊最底層、其他工具都跑在它上面)；
(b) **Leverages CLI ecosystem** (git、docker、gcloud、curl 全都已經 CLI 化，不必重造)；
(c) **Already set up for orchestrating long-running tasks** (terminal 從第一天就是非同步、process-spawning friendly)。但他立刻補一句「**terminal needs to be modernized**」— terminal 是對的介面，VT100 是錯的實作。
3. **Warp's history (2020 → 2025 timeline):** Lloyd 用一條時間軸 walk through 公司演化，每年加上對應的產品里程碑：**2020 spring** had an idea, **2020 fall** started building, **2021** Warp MVP + Private Beta(強調 "Better UX")，**2022** Mac Public Release, **2023** Warp Drive + AI Chat + AI Command Search 三件 AI feature 一次推出, **2024** velocity 暴增 (Linux, Agent Mode, Session Sharing)，**2025** Windows release + ADE 重新定位。後面接一張密密麻麻的 feature grid (Split Panes、Markdown Viewer、Command Search、Block Sharing、Secret Redaction、SSO、Active AI...)，標題就叫「**Warp continues to grow**」。Mission 寫得很短：「**Empower all developers to deliver great software, quickly and reliably.**」
4. **Product Design Roadmap — 四大 pillar 與 Agent Quality 三角:** Warp 把 ADE 拆成四個 product surface: **Terminal** (modern intelligent terminal)、**Code** (在真實 codebase 裡跑 coding agent, 原生 review/edit)、**Agents** (同時部署多 agent、集中追蹤)、**Drive** (centralize knowledge / context)。對應 slide 上 4 張產品截圖 (git rebase block、code review diff、agent 跑 eval、Drive folder tree)。Ambient agent 線路另外講：**API / Hosting / Management / Integrations** 四件事讓 agent 能「從任何環境呼叫 Warp 的 agent runtime」「跑在 Warp server 或 self-hosted」「跨 session 追蹤」「跟常用 app 一鍵串接」。最後拋出 **Agent Quality = Models + Harness + Dev Ex** 公式 — model 由 Warp 動態挑 (resilient to provider outage)、harness 做 prompt tuning + context summarization + A/B eval、dev ex 把 agent 做成「不用滑鼠、不用 text edit、能 reuse / share context、能接 MCP」的純文字介面。
5. **How Warp uses Warp to build Warp (內部 5 條 coding mandate):** (1) **Start with a prompt** (描述「怎麼做」而不只是「做什麼」、附上 file/image context)、(2) **Iterate on a plan** (先和 agent 把問題搞清楚、把方案 script 出來再放手讓它跑)、(3) **Steer the agent** (不要 set-and-forget、邊跑邊 re-prompt + review diff、必要時手 edit)、(4) **Engineers own the PR** (送 review 的 PR 作者必須能像自己寫的一樣解釋每一行)、(5) **Document context for the next PR** (每次學到的東西回寫到 **WARP.md** 或其他 rules 檔，下次 agent 直接拿)。這條清單是整場演講最具體的可操作守則。
6. **Business 與市場定位 — 2×2 framework:** Lloyd 用一張 2×2 圖把 dev tool 市場分成 X 軸 Code by hand ↔ Code by prompt、Y 軸 Editor-first ↔ Terminal-first; JetBrains / VS Code 在左下、Cursor 在中下、Windsurf / Augment 在右下、**Warp 與 Claude Code** 同坐右上 (**Code by prompt + Terminal-first**)。Lloyd 強調 Warp vs AI IDE 的差異是：橫跨整個 setup → production 生命週期、跨多 project、原生支援多個 long-running agent; vs CLI tool 的差異是：原生 in-app 編輯(不用 context switch)、有

codebase embedding、有 multi-agent 通知 UX。市場判斷: **huge market** (automating development = 11m+ 開發者)、**not winner-take-all** (dev 喜歡 try new thing、公司也願意 honor preference)、**still early** (最大玩家還沒進場、model 還在進步)、風險端坦誠 **competitive market** (Cursor 等對手錢多)、**inference expensive** (usage-based 成本壓邊際)、**complex relation with labs** (Anthropic / OpenAI 既是供應商也是競爭者)。

7. 給學生的 closing – What's next: 「Keep studying CS !」Lloyd 認為不論 model 怎麼進化, 深度 CS 知識 + **problem solving skill** 永遠值得投資; 「It's never been a better time to be a builder」是他對 Stanford 學生的結語, 呼應 W1 Karpathy 講的「軟體開發民主化」。

Key takeaway: Warp 的 thesis 不是「terminal + AI 黏一黏」, 而是把 command line 當作下一代 agent runtime 的最佳 host。產品設計圍繞「interactive agent + ambient agent」雙軌, agent 品質拆成 model / harness / dev ex 三件可獨立優化的事; 對 vibe coder 的啟示是 **prompt-first**、**own your PR**、把 **learning** 寫回 **WARP.md** / **AGENTS.md** 這條 dogfooding 紀律比換不換 terminal 重要。

7.5 Reading 摘要

篇名	來源	一句話重點
Warp University	warp.dev	Warp = 「Agentic Development Environment」= local terminal + cloud agent platform (Oz) + Warp Drive 跨端 context
Warp vs Claude Code	warp.dev/university	Warp 與 Claude Code 不是競品而是 stack 兩層, 把 CLAUDE.md rename 成 AGENTS.md 即可零成本遷移
How Warp Uses Warp to Build Warp	notion.warp.dev	Warp 內部 Coding Mandate: 每個 task 從 prompt 起手、卡 10 分鐘要回報、bug 一律先變 eval 不直接 patch

閱讀優先順序: 先讀 [Warp University](#) 建立 ADE 心智模型 → 再讀 [Warp vs Claude Code](#) 看 Warp 與 Claude Code 怎麼共存 → 最後 [How Warp Uses Warp](#) 看 dogfooding 紀律 – 三篇加起來不到 30 分鐘但能拿到完整圖像。

7.6 Assignment: Agentic Development with Warp

- **Source:** github.com/mihail911/modern-software-dev-assignments/tree/master/week5
- 任務描述: 安裝 Warp, 跑一系列 hands-on 練習: (a) 用 Agent Mode 對陌生 repo 做 codebase exploration, (b) 寫 **WARP.md** (或 rename 既有 **CLAUDE.md**) 讓 agent 拿到 persistent context, (c) 用 Warp Drive 把重複指令存成 workflow, (d) 在 Warp 內跑 Claude Code CLI 體驗 first-class wrapper (Ctrl+G 多行、agent notification、code review panel), (e) 設一個 cloud agent task 跑 background test。
- 自學者可行性: ★★★★★ 完全可做。Warp 個人 free tier 包含 AI quota, 足以跑完作業。預估 3-5 hr。需 macOS 13+ / Windows 11 / Ubuntu 22.04+。

💡 不想換 terminal 的替代方案: 可以只在練習期間裝 Warp、做完 export workflow 後 uninstall。Warp 不會 hijack 你 default shell, 只是另一個 GUI app。

7.7 對 Vibe Coder 的應用

對 Claude Code 重度用戶 (也就是這份講義的目標讀者) 來說, 「該不該換 terminal」是個值得認真評估的問題。決策框架:

1. 如果你已經把 **Claude Code** 用得很順 — 不必勉強切，至少把 **CLAUDE.md** 複製一份成 **AGENTS.md** 放專案 root。這個檔在 Cursor、Warp、Codex CLI、未來 MCP-aware agent 都會被自動讀取，等於同一份 context 投入多家工具，零 lock-in
2. 若你 **daily** 主力是 **iTerm2 + Claude Code** — 評估 Warp 主要看 (a) 你常開幾個並行 agent session? 三個以上 Warp 的 vertical tabs + agent metadata 真的會省心; (b) 你有沒有「碎片時間想跑 background task」的需求? 有則 cloud agent 直接打到痛點
3. 絕對先學會的 **Warp feature** — **Ctrl+G** 多行 prompt (給 Claude Code 寫長 prompt 不再要逃跳脫換行)、**lock, share link** (把錯誤訊息一鍵丟同事比截圖好十倍)、Warp Drive Rules (重複指令 / 風格規範一次寫、永久套用)
4. **Warp Drive vs CLAUDE.md** 的分工 — **AGENTS.md** 放專案層級 context (這 repo 是什麼、命名慣例、業務邏輯); Warp Drive 放跨專案的個人 preference (總是用 pnpm、commit message 用英文、push 前必跑 lint)。兩層不重疊
5. 不要打開 **YOLO mode** — Warp 也有「全自動 approve all tool」選項,跟 W6 會講到的 Copilot RCE 是同一種風險。Vibe coding 速度再快也別省這個 confirm
6. **Cloud agent** 適合的 **task** 類型 — 跑完整 test suite、build production bundle、跑 dependency audit、批次 rename file、每週一鍵更新 lock file。不適合: 實際邏輯改動 (你沒在現場 review diff、跑出來的 code 你不知道對不對)

💡 **vibe coder** 的 **Day-1 Quick Win**: 今天裝 Warp、把現有 **CLAUDE.md** 複製成 **AGENTS.md**、在 Warp 內 **cd** 進你的 side project、按 **⌘+Enter** 進 Agent Mode 問「what does this repo do?」。20 秒內你會看到 Warp 自動 index codebase 並回給你一份比 README 還完整的 architectural summary — 這是傳統 terminal 永遠做不到的事，體驗一次你就會知道值不值得遷移。

上一週: [W4 Coding Agent Patterns](#) | 下一週: [W6 AI Testing and Security](#)

8. Week 6: AI Testing and Security

本週你會學到什麼: vibe coding (用 AI 一句話生整個 app) 的最大盲點 = 資安。本週講 secure vibe coding、vulnerability detection 的歷史 (SAST/DAST/SCA)、prompt injection 攻擊面、以及怎麼用 AI 自動產生 test suites。Friday 由 Semgrep CEO Isaac Evans 親自開講。

💡 對非資工背景讀者: 本週是 10 週裡最 technical 的一週。可以用醫療類比理解 — vulnerability ≈ 病灶, SAST ≈ 預防醫學(驗血), DAST ≈ 臨床檢查(影像), prompt injection ≈ 社交工程詐騙。

學習目標

8.1

完成本週後, 你應該能:

1. 辨識 SAST (Static Application Security Testing) vs DAST (Dynamic Application Security Testing) 的適用情境
2. 解釋 prompt injection 怎麼攻擊 coding agent (含 GitHub Copilot RCE 的真實 case)
3. 應用 Semgrep / Snyk / GitHub Advanced Security 對 vibe-coded 專案做 security audit
4. 評估 OWASP Top 10 在 LLM 應用上的對應風險(OWASP LLM Top 10)

8.2 核心概念導讀

8.2.1 一、Vibe Coding 的五大資安盲點 — 為什麼速度越快越危險

W1-W5 教你怎麼用 LLM 把開發速度拉到 10x, 這週要回頭算帳:速度的另一面是 **attack surface** 同步擴張。對非資工背景的 vibe coder (醫學生、生科研究者、設計師), 五個最常踩的盲點是:

1. **API key** 直接寫進 **code** — Claude Code 跑出來的 demo 常常把 OpenAI API key、AWS credential、DB password 直接放在 source file 裡, commit 上 GitHub 等於把鑰匙公開貼在 7-11 門口
2. **User input** 直接拼 **SQL / shell / HTML** — Vibe coding 一句「做個搜尋 page」生出的 code 99% 沒做 input sanitization, 給點機會就是 SQL injection (注入式攻擊) / XSS (cross-site scripting, 跨站腳本攻擊) / command injection
3. **Authentication** 自己寫 — 真正該用 Auth0、Clerk、Supabase Auth 之類成熟方案, 但 vibe coder 常被 LLM 忽悠寫個自製 JWT (JSON Web Token) 系統, sign / verify 做錯就是直接登入別人帳號
4. **Dependency** 來路不明 — `npm install something-vibe-good` 之前沒人 audit, 惡意 package 偷 env var、安裝 backdoor 是常態
5. **Agent** 的全自動 **approve mode** — Claude Code / Cursor / Warp 都有「all tool calls auto-approve」選項, 打開 = 把 shell 鑰匙交給可被 prompt injection 劫持的 LLM (見下方 [Copilot RCE 案例](#))

💡 譯解(醫療類比): Vibe coding 的 security 風險就像「治療速度太快、沒做完整 history taking」— 你救了當下的 acute 症狀 (功能 work 了), 但漏掉的 underlying disease (漏洞) 會在後面爆掉。SAST 對應預防醫學的全套 lab work (驗血、尿液、影像 — 看靜態指標), DAST 對應 stress test (運動心電圖、心導管 — 對運作中的系統打壓力測試), prompt injection 對應社交工程詐騙(騙病人本人說出密碼、騙醫護越權給藥)。

8.2.2 二、SAST、DAST、SCA — 漏洞檢測的三角

SAST vs DAST 把 application security testing 拆成兩種互補典範。實務上要再加上 SCA (Software Composition Analysis, 軟體成分分析), 合稱檢測三角:

工具類別	看什麼	何時跑	醫療類比	實務工具
SAST(靜態應用程式安全測試)	Source code / binary (不執行)	Commit 前、IDE 內、CI early stage	預防醫學 — 看 lab 數值找潛在病灶	Semgrep、CodeQL、SonarQube
DAST(動態應用程式安全測試)	Running app (送 malicious request 觀察反應)	Staging deploy 後、scheduled scan	臨床檢查 — 對活體做運動測試找症狀	OWASP ZAP、Burp Suite、Nuclei
SCA(軟體成分分析)	package.json/requirements.txt 等 dependency 清單	每次 npm install 後、weekly cron	流行病學監測 — 看你吃進的「藥」(package) 有沒有已知 contraindication	Dependabot、Snyk、Renovate

三者的 trade-off: - SAST 早期便宜但 false positive 高 — 一掃幾秒、commit 前就能跑，但會誤報一堆「理論上有風險但實際 unreachable」的 code path (醫療類比: 腫瘤 marker 偽陽性多, 需臨床判斷複核) - DAST 貼近真實攻擊但發現晚 — 等 app 跑起來才能掃, bug 修復成本高 (醫療類比: 到 ER 時 acute MI 已經發作, 治療成本比預防高 100 倍) - SCA 是 prerequisite — 你寫得再安全, npm 套件帶後門也沒救 (醫療類比: 自己生活作息再好, 吃到污染食品照樣中毒)

OWASP Top Ten 是這個檢測三角的「分類字典」— A01 Broken Access Control、A03 Injection、A05 Security Misconfiguration 是 vibe coder 最常踩的三類, 每寫一個 endpoint 自問「這條對 Top Ten 哪幾項有曝險?」是基本紀律。

8.2.3 三、Prompt Injection — Coding Agent 時代的全新攻擊面

傳統 web app 的攻擊面 OWASP 列在 Top Ten 裡, 但 **prompt injection** (提示詞注入) 是 LLM 時代才出現的新類別。原理: 把惡意指令藏在 LLM 會 ingest 的內容 (code comment、GitHub issue、web page、PDF、image OCR), 讓它覆蓋原本的 system instruction。

Copilot RCE via Prompt Injection 是這類攻擊的教科書級案例。研究員 Johann Rehberger 發現 CVE-2025-53773 (CVE = 全球漏洞身份證編號): 攻擊鏈為 (1) malicious instruction 藏進 source code 或 GitHub issue → (2) Copilot 讀進 context → (3) injection 指示 Copilot 自行寫入 .vscode/settings.json 開啟 chat.tools.autoApprove: true (YOLO mode) → (4) 從此 Copilot 任何 shell command 都不再向 user 確認 → (5) 達成 RCE (Remote Code Execution, 遠端任意程式執行)。

💡 譯解(醫療類比): Prompt injection 像「social engineering 詐騙打電話到護理站、聲稱是主治醫師、指示護士改某病人的醫囑」— 系統本身 (電話、HIS) 沒漏洞, 但 trust boundary 設計錯了: 你不能把「會講人話的東西」自動當成有權限的人。Coding agent 的 prompt injection 之所以致命, 是因為它能把「文字攻擊」轉成「真的執行 shell command」, 從詐騙電話升級為直接動手簽醫囑單。

Agentic AI Threats 把這個攻擊面再往前推: agentic AI 操作 enterprise infra (GCP、AWS、internal API) 會被劫持去偷 service account token (cloud metadata service 的 169.254.169.254 IP 是經典攻擊目標), 或藉 BOLA (Broken Object Level Authorization, 物件層級授權失敗) 越權讀其他 user 資料。Defense 必須是 layered: prompt hardening + sandbox + tool input sanitization + runtime filtering, 沒有單一 mitigation 夠用。

8.2.4 四、AI-generated Test 與 AI Find Vulnerabilities — 實證告訴我們什麼

LLM 不只是攻擊面, 它也是新工具。但這個工具好不好用? **Finding Vulnerabilities with Claude Code & Codex** (Semgrep blog) 給了一份冷靜的實證: 對 11 個真實 Python web app (>800,000 LoC) 跑 Claude Code (Sonnet 4) 與 OpenAI Codex (o4-mini) 做 security audit, 結果:

- Claude 找到 46 個真實漏洞、Codex 找到 21 個 — 真的能找到 bug

- 但 false positive 率分別 86% 與 82% — 十條警報只兩條是真
- 同一份 code 跑三次得到 3 / 6 / 11 個不同 bug — 嚴重 non-deterministic

Non-determinism 的根因在 [Context Rot](#): Chroma Research 對 18 個 SOTA LLM 做擴展版 NIAH(Needle in a Haystack) 測試,發現 input 越長 performance 越不穩,即使是百萬 token context window 也一樣 — distractor、coherent narrative、low semantic similarity 都會加劇衰減。所以 LLM 掃大 codebase 時,後段檔案可能根本沒被「真的看到」。

對照組是 [Vulnerability Prompt Analysis with O3](#) 的 success case — Sean Heelan 用嚴格的 system prompt (三階段: code path → conditional analysis → contradiction detection; 強制原則「寧可漏報也不誤報」) 驅動 OpenAI o3 在 Linux kernel 找出 CVE-2025-37899 (一個真實的 use-after-free 漏洞)。差別在哪? **Scope** 窄 (一類漏洞、特定模組)、紀律嚴 (強制 step-by-step trace、禁止 hypothetical)、反幻覺 (allow 「no findings」 是合法輸出)。

歸納:LLM 找 bug 不是 silver bullet, 但配對 (a) 鎖定特定 vulnerability class、(b) 多次採樣取聯集、(c) 仍跑傳統 SAST 補 taint tracking 盲區、(d) 把 LLM 警報當「嫌疑名單」而非結論,可以變成有用的補強層。

8.3 Monday Lecture (10/27): AI QA, SAST, DAST, and Beyond

- **Slides:** [Google Slides 公開連結](#)
- **講者:** Mihail Eric

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

Mihail 開場切入點直接「軟體錯誤可以毀掉使用者對產品 / 公司的信任、造成巨大金錢成本。當 LLM 在寫你大部分 code 時,需要強而廣的 guardrail 才不會出包。」

1. 既有 **threat landscape** 速覽 — SQL injection、Cross-site scripting (XSS)、broken authentication、insecure direct object reference、security misconfiguration、sensitive data exposure 六大類。
2. **Vulnerability detection 三角: SAST / DAST / SCA** —
 - **SAST (Static Application Security Testing)**: 白箱、看 binary + source code、SDLC 早期跑 (修 bug 最便宜)。Pattern matching scan codebase。可抓 SQL injection、command injection、XSS。工具範例: Bandit、Semgrep、ESLint + extensions。
 - **DAST (Dynamic Application Security Testing)**: 黑箱、模擬真實 hacker 攻 running app、SDLC 全程都可跑 (false positive 比 SAST 少)。技術: input fuzzing、session token manipulation、config / header testing、brute force rate limit。
 - **SCA (Software Composition Analysis)**: 深度分析 OSS package。技術: 分析 package metadata、transitive dependency resolution、跟 vulnerability DB 比對、binary / artifact scanning。
 - 三者 cover 三層: code (SAST) + runtime (DAST) + dependency (SCA)。
3. **What's changed: Bad — 新 AI agent attack vector** —
 - **Prompt injection**: 藏 hidden / 誤導指令在 LLM ingest 的內容裡讓它偏離原本行為。Slide 引用 AMP code 案例: 用 prompt injection 從 AI agent 提取 system prompt。
 - **Tool misuse**: 用欺騙 prompt 讓 agent 濫用整合的 tool。引用 [embracethered.com](#) 的 AMP escape blog — agent 修改 system configuration 並 escape sandbox。
 - **Code attacks**: 利用 agent 的 code execution 能力獲得執行環境的未授權存取。
 - **Intent breaking**: 操縱 agent 的 plan, 讓 action 偏離原本目的。
 - **Identity spoofing**: 用 compromised authentication 假冒合法 agent。
4. **What's changed: Good — 新 SAST/DAST/SCA 增強技術** —
 - 「Shift left」 security 比以前更可及 — LLM 可被嵌入 workflow 抓問題
 - **Automated penetration testing**
5. **LLM 在 security testing 的限制** —
 - **AI SAST 的 false positive 率高得驚人** — Claude Code / Codex 對某些漏洞類別 50-100% false positive, 比傳統 SAST (50+%) 還糟
 - 既有 **benchmark** 不貼近真實情境, 難 evaluate LLM
 - **Non-deterministic**: 同 prompt 多跑得不同結果, 怎麼確認 catch 完所有 vulnerability?

- **Context rot:** not all context is created equal; compaction (壓縮 context) 會讓資訊變形
- 6. **Open question** –
 - 怎麼降 vulnerability detection 的 false positive 與 hallucination?
 - 怎麼 verify LLM 生的 patch 真的安全、不引入 regression?
 - 怎麼讓 LLM 解釋它為什麼 flag 這個 vulnerability、為什麼 propose 這個 fix?
 - 衡量 LLM AppSec 表現的對的 benchmark 是什麼?
 - LLM 怎麼嵌進 CI/CD 不淹沒 team 的告警雜訊?
 - AI 生成的 patch 引入新 vulnerability 時誰要負責?

Key takeaway: AI 沒讓 QA / Security 變不需要 – 反而讓它變更重要。SAST + DAST + SCA 的三角依然是基礎，加上對 prompt injection / tool misuse / code attack 三類 AI-era attack vector 的特別防護才完整。LLM 找漏洞能力有但 false positive 高、非確定性、context rot 三個根本問題還沒解 – 把 LLM 警報當「嫌疑名單」處理而非結論。

8.4 Friday Lecture (10/31): Isaac Evans (Semgrep CEO)

- **Speaker:** [Isaac Evans](#), CEO of [Semgrep](#)
- **Slides:** 未公開

Slides 未公開，依 Isaac 公開訪談 + Semgrep 公開技術文件 + 同期 Semgrep blog ([Finding Vulnerabilities with Claude Code](#)) best-effort 重建:

Isaac Evans 是 MIT Lincoln Laboratory cybersecurity researcher 出身，2017 年創立 Semgrep (前身 r2c) – 把學術界的 program analysis 技術做成 developer-friendly 的 SAST 工具。Semgrep 的 unique stance 是「rule-as-code」：每條 security rule 寫成 YAML pattern，跟 grep 一樣好懂但比 grep 強得多(語法樹層級的 pattern matching)。預期演講內容：

1. **Semgrep origin story** – 為什麼從 r2c rebrand 成 Semgrep、為什麼選擇 open-source rule registry (任何人能寫 rule、社群共享) 這個 distribution model
2. **Static analysis 的歷史** – 從 lint (1979 Bell Labs) → Coverity (commercial SAST 開山) → FindBugs / SpotBugs (學術界 Java tool) → CodeQL (GitHub 收購、query language 取向) → Semgrep (pattern matching + 開源 rule registry)
3. **為什麼 LLM 不會取代 SAST** – 對應 reading 那篇實證：LLM 缺 inter-procedural taint flow 能力、non-deterministic、context rot; rule-based SAST 反而在 injection 類 deterministic 偵測強。兩者是互補不是取代
4. **AI 怎麼增強 SAST** – Semgrep 自己怎麼用 AI: (a) 用 LLM 自動生 rule (看一個 CVE patch 反推可掃同類問題的 rule)、(b) 用 LLM 做 false positive triage (rule 報的警報先讓 LLM 篩一輪) (c) 用 LLM 自動 suggest fix (不只報 bug 還給 patch suggestion)
5. **AI Coding 對 Semgrep 的衝擊** – AI 寫的 code 是不是 less secure? 實證上是的 (Stanford 2023 study 已經證實)，所以 vibe coding 浪潮反而擴大 SAST 市場
6. **企業導入經驗** – Semgrep 服務的客戶 (Snowflake、Slack、Figma 等) 怎麼把 SAST 整進 dev workflow、怎麼平衡 false positive 與 dev velocity、security team 怎麼跟 AI dev tool 共處
7. **Roadmap** – Supply chain security (SCA + SAST 融合)、prompt injection-aware rule、LLM-aware threat model

Key takeaway: Static analysis 沒被 LLM 殺掉，反而因為 vibe coding 浪潮變得更重要。Semgrep 的 stance 是「rule-based + AI-augmented」混合方法 – rule 給 deterministic、AI 給 contextual reasoning，兩者各取所長。對 vibe coder 的 implication: 別以為「找 Claude 幫我 audit」就夠了，必須加上 deterministic SAST (Semgrep 有 free tier) 才能蓋住 LLM 的 taint tracking 盲區。

8.5 Reading 摘要

篇名	來源	一句話重點
SAST vs DAST	Splunk blog	SAST (白箱、看 source) + DAST (黑箱、跑 runtime) + 可加 RASP 形成多層防禦, 現代 pipeline 必整合兩者
Copilot RCE via Prompt Injection	embracethered.com	CVE-2025-53773: prompt injection 騙 Copilot 自寫 <code>.vscode/settings.json</code> 開啟 YOLO mode → 完全 RCE
Finding Vulnerabilities with Claude Code & Codex	Semgrep blog	11 repo / >800k LoC 實測: 找到真 bug 但 86% false positive、non-deterministic, LLM 不能取代 SAST
Agentic AI Threats: Identity Spoofing	Palo Alto Unit 42	Agent 可被騙去偷 cloud metadata token / 越權讀資料(BOLA), 漏洞 framework-agnostic、源於 insecure design
OWASP Top Ten	owasp.org	十大 web app 風險清單, 2021 版以 incidence rate 排序, 是 secure coding 的共同詞彙
Context Rot	research.trychroma.com	18 個 SOTA LLM 實證 — context window 越大 不 善於越可靠, 10k token 後 performance 顯著衰減
Vulnerability Prompt Analysis with O3	github.com/SeanHeelan	三階段 prompt + 寧可漏報原則, o3 真的找到 CVE-2025-37899 (Linux kernel UAF)

閱讀優先順序: 時間有限的话先讀 [OWASP Top Ten](#) (建立詞彙) → [SAST vs DAST](#) (工具光譜) → [Copilot RCE](#) (prompt injection 教科書 case) → [Finding Vulnerabilities with Claude Code](#) (LLM 實戰能力 baseline) → [Vulnerability Prompt with O3](#) (怎麼把 LLM 用對)。其他兩篇是延伸 — [Agentic AI Threats](#) 適合做 enterprise agent app 的人、[Context Rot](#) 是 root-cause 補充。

8.6 Assignment: Writing Secure AI Code

- **Source:** github.com/mihail911/modern-software-dev-assignments/blob/master/week6/assignment.md
- 任務描述: 對一個 vibe-coded web app (assignment 提供或自帶 side project) 做完整 security review: (a) 跑 Semgrep / Snyk 等 SAST tool 並 triage 結果、(b) 對 OWASP Top Ten 每一類盤點 risk、(c) 用 [Vulnerability Prompt with O3](#) 的三階段 prompt 範本驅動 Claude 找特定類型漏洞、(d) 模擬 prompt injection attack (在 README 或 comment 藏指令) 測試自家 agent 抗性、(e) 寫 fix 並驗證。
- 自學者可行性: ★★★ 可做但偏深。需要的工具都有 free tier (Semgrep、Snyk free、OWASP ZAP open source), 但 OWASP 詞彙與 web security mental model 對非資工背景讀者有學習曲線。預估 5-8 hr。

💡 非資工背景讀者的減量做法: 把 (a) (跑 Semgrep) + (e) (修出來的 critical bug) 做完, 其他 task 看 reading 理解就夠。最終目標是建立「每次 commit 前要跑哪些 check」的肌肉記憶, 不是變成 pen-tester。

8.7 對 Vibe Coder 的應用

W6 的內容對非資工背景讀者門檻高, 但本節不要求你成為 security expert — 目標是建立最小可行的 security hygiene, 把高頻、致命的風險擋掉。實戰建議:

1. API key 管理 — 鐵律三條

- 絕不寫進 **source code**: 用 `.env` (要加進 `.gitignore`)、生產環境用 cloud secret manager (Vercel/Railway env var、AWS Secrets Manager)
 - 公開 **repo** 第一件事跑 `gitleaks` 掃 commit history, 看有沒有歷史不慎 leak 的 key (醫療類比: 每次新工作報到都驗血看有沒有 hidden chronic condition)
 - **API key** 一發現外洩立刻 **revoke + rotate**, 別存僥倖。OpenAI / Anthropic 都有 monitoring, 1 小時內可能就被盜刷數百美金
2. **Vibe-coded side project 的 minimal viable security check** (每次 push 前跑)
- **SAST**: `semgrep --config=auto` 或 `snyc code test` (兩者皆 free tier)
 - **SCA**: `npm audit` / `pip-audit`, 或讓 GitHub Dependabot 自動開 PR
 - **Secret scan**: `gitleaks pre-commit hook`
 - **OWASP Top Ten** 自查: endpoint 寫完問自己「有沒有 access control? user input 有 sanitize? config secret 有沒有寫 code 裡?
- 這四個加起來 5 分鐘以內, 是 P3 人上人 等級的最小 viable hygiene
3. **Claude Code / Cursor / Warp 的 security risk pattern** — 別做的事
- 絕不開 **YOLO mode** / **auto-approve all tools**: 對應 **Copilot RCE** 的 attack chain, 這是 single point of failure
 - **clone 別人的 repo** 第一件事檢查 `.vscode/`、`.cursor/`、`.claude/`: 這些是 agent config, 可能藏 prompt injection
 - 限制 **agent** 的 **file write / shell exec scope**: 用 Claude Code 的 `permissions` 設定把可寫範圍鎖在專案資料夾內, 別讓它能改 `~/.zshrc` 或 `/etc/`
 - 對外部 **untrusted source** 抱「假設裡面有 **prompt injection**」心態: fetch 來的網頁、PDF、image OCR 都該被視為 hostile input
4. 請 LLM 找 bug 的 4 條紀律 (從 **Vulnerability Prompt with O3** 提煉)
- **Scope** 要窄: 指定一類漏洞 (如 SQL injection)、一個檔案, 而非「找所有漏洞」
 - 強制三階段輸出: `code path` → `conditional analysis` → `contradiction detection`, 不要只要結論
 - 明確說「找不到回 **no findings**、別亂掰」: 寧可漏報不要誤報這條紀律會大幅降 false positive
 - 跑多次 (≥3) 取聯集: 對應 **Context Rot** 與 **Semgrep 實證** 的 non-determinism
5. **Authentication / Payment 等 critical path** 永遠用成熟方案
- Auth: Clerk、Auth0、Supabase Auth、NextAuth — 別自己寫 JWT
 - Payment: Stripe、Paddle — 別自己處理信用卡資料 (PCI-DSS 是合規地獄)
 - 上傳檔案: 用 cloud storage signed URL (S3、GCS), 別讓檔案直接過你的 server
 - 這些東西踩錯一次代價極大 (資料外洩、被罰、被盜刷), 比花時間自製划算太多倍
6. **Claude Code 的好習慣** — 把 security 寫進 **CLAUDE.md**
- 加一段 `## Security Guardrails`: 禁止把 secret 寫進 code、user input 必須 sanitize、新 endpoint 必加 access control check
 - 加一段 `## Forbidden Patterns`: 列出已踩過的雷 (例: 「絕不用 `eval()`」 「絕不直接拼 SQL」), Claude 每次都會讀

💡 **vibe coder 的 Day-1 Quick Win**: 今天就在你最 active 的 side project 跑這三條 — (1) `npm install -g gitleaks && gitleaks detect` (10 秒看有沒有歷史 leak 的 key), (2) 申請 **Semgrep free account** 把 repo 連上、自動跑一次掃描看 finding, (3) 在 **CLAUDE.md** 加上 `## Security Guardrails: never put secrets in source code; sanitize all user input; require auth check on every endpoint.` 30 分鐘建好的這三條 hygiene 在 95% 的情境下就贏過大多數 vibe-coded project。

上一週: [W5 The Modern Terminal](#) | 下一週: [W7 Modern Software Support](#)

9. Week 7: Modern Software Support

本週你會學到什麼：AI code review、AI-assisted debugging、智慧型文件生成。當你的 codebase 被 AI 寫出 70% 之後，code review 的意義變了 — 不是「同事看人寫的 code」而是「人類審 AI 寫的 code」。Friday 由 Graphite CPO Tomas Reimers 開講(Graphite 是頂尖的 AI code review 平台)。

學習目標

9.1

完成本週後，你應該能：

1. 辨識 AI code review 在 catch bug、style enforcement、knowledge transfer 三個面向的能力與盲點
2. 應用 Graphite / CodeRabbit / GitHub Copilot Code Review 設定 PR review 流程
3. 設計一個「AI 寫 + AI review + 人類最終 approve」的 PR workflow
4. 評估 哪些 code review 任務適合全自動 vs 哪些必須留給人類

9.2 核心概念導讀

9.2.1 一、Code review 從「品管把關」到「心智模型同步」的歷史轉折

要看懂為什麼 2026 年大家還在認真討論 code review(明明 AI 都會寫 code 了)，你必須先理解 review 在過去 50 年的角色變遷。早期 IBM 1976 年提出的 Fagan inspection (Fagan 形式審查) 是一套重型流程，目的單純：在 code ship 出去之前再多一雙眼睛抓 defect (缺陷)。Steve McConnell 在《Code Complete》整理的 industry data 給了這個流程量化背書 — design + code inspection 平均能抓到 55-60% 的 defect，遠高於各種 testing method (25-45%)。Jeff Atwood 2008 年寫的 [Code Reviews: Just Do It](#) 把這個論點直接搬上檯面：別爭辯做不做、有做就贏，每一分鐘 review 投資都會 paid back tenfold (十倍奉還)。

但 review 真正的價值很快超越「抓 bug」。Blake Smith 2015 年 [Code Review Essentials](#) 把 review 重新 framing 成 **mental model synchronization**(心智模型同步) — 「當 Bob 提交 accounting subsystem 的 PR，Amy 在 review 中同步更新她對該 system 的 mental model」。Bug 被擋下來只是 side effect，真正的 ROI 是團隊知識擴散、降低未來修改的 onboarding 成本。GitHub staff engineer Sarah Vessels 在 [How to Review Code Effectively](#) 進一步把 review 拉高到「shape the product's implementation」— review 不是末端 quality gate，是設計階段的延伸。

💡 譯解：把 code review 當「品管站」是 1980 年代的思維。2020 年代的 framing 是「review = 一場結構化對話」，bug detection、knowledge transfer、architectural alignment 三件事一起發生。理解這個轉變，後面看 AI reviewer 才不會誤以為「AI 會抓 bug 就能取代 review」— 它取代的只是其中一層。

9.2.2 二、AI code review 能做什麼、做不到什麼：兩軸四象限框架

進入 LLM 時代，code review 被拆成兩種勞力：mechanical layer (機械層) 跟 alignment layer (對齊層)。Google 2024 年發在 AIware '24 的論文 [AI-Assisted Assessment of Coding Practices](#) 提供了第一份 academic baseline — Google 內部 AutoCommenter 對數萬名 developer 部署，覆蓋 C++ / Java / Python / Go，能涵蓋 68% 的 **human reviewer** 常引用 **best practice**，而且其中 66% 是傳統 linter 抓不到的 (comment clarity、justified naming exception、context-dependent 慣例)。Comment resolution rate (評論解決率) 約 40%，與 human reviewer 的 ~50% 接近。

Graphite co-founder Tomas Reimers 在 [Lessons from Millions of AI Code Reviews](#) 提出更實用的 2x2 quadrant (兩軸四象限) 框架：

	LLM 能 reliably catch	LLM 抓不準
Human 想收	✓ 右上: 留 comment	✗ 左上: 人類 reviewer 守備
Human 不想收	✗ 右下: always-correct-but-unwelcome (add test / extract function)	✗ 左下: 別碰

只有右上象限的 comment 該被 AI 留下。Reimers 點出兩個典型失敗模式：

- 左上盲區: LLM 抓不到 codebase-specific convention (公司命名慣例、業務 domain 邏輯、住在資深工程師腦中沒寫成文件的 tribal knowledge)
- 右下過度發言: LLM 愛留「你應該加 test」「這應該 extract function」這類技術上沒錯但 reviewer 不該無腦留的 comment, 留多了 author 直接 ignore 整個 reviewer

Graphite 的 [AI Code Review Implementation Best Practices](#) 把這個 insight 翻譯成商業 framework — AI reviewer 應走 **human-in-the-loop** (人類介入式) 路線, AI 跑 first pass、human 驗證並 approve。signal-to-noise ratio (訊號雜訊比) 是首要指標, 三個操作槓桿: (1) per-issue-type sensitivity tuning (敏感度調校), (2) false-positive feedback loop (讓 developer 標 false positive 訓練系統), (3) priority level (火力集中在 security + obvious bug, nit / style 暫時關掉)。

9.2.3 三、為什麼 millions of reviews 教我們的事比任何 paper 都重要

Graphite 的 Diamond AI reviewer 跑在數百萬個 production PR 上, 沉澱出三條關鍵 lesson, 是任何 academic benchmark 都看不到的:

1. **Metric 設計決定產品命運**: 傳統 thumbs-up / thumbs-down 收集率太低 (<4% 下票率沒區辨力)。Graphite 改用「**action rate**」— comment 是否導致 author 在後續 commit 改 code — 當作 success metric。這個 metric 直接 align 商業價值 (comment 沒被 act on = 浪費 reviewer 時間 = 浪費 LLM API call)。
2. **52% vs 50% parity 的意義**: Human reviewer 的 comment 約 50% 帶來 action, Diamond 經過調校到 2025 March 已達 52% — 等同 human baseline。這個數字翻譯成白話: AI reviewer 已經追平人類 reviewer 的 actionability, 剩下的是「哪些象限該留給誰」的分工問題, 不是「能不能用」的問題。
3. **False positive 比 model size 重要**: Google AutoCommenter 與 Graphite Diamond 不約而同得出同一結論 — production 成功的關鍵不是 model quality, 是「承認 LLM 會錯、留出 retract 通道」。Google 用 URL suppression (per-rule mute), Graphite 用 false-positive feedback loop。一個 false positive 比例高的 reviewer, 再準的部分也會被 developer 一起 ignore。

💡 譯解: 商用 AI reviewer 像一個「永遠在試用期的新員工」。它技術力到位, 但說話會 over-commit、會踩公司潛規則。產品設計的關鍵不是讓它變更聰明, 是給它「不知道就閉嘴」的機制。Diamond 的 quadrant 框架本質上是教 LLM 自己 self-censor。

9.2.4 四、AI 寫 + AI review + 人類 approve 的 PR workflow

把 W7 所有概念整合起來, 2026 年 vibe coder 的標準 PR workflow 應該長這樣:

1. Author (你 + Claude Code / Cursor) 寫 code → 自動 commit
2. 自己 self-review diff (Vessels 原則) → 切成 scalpel-like 小 PR (Smith 原則)
3. 寫 PR description: problem + structural change + testing (Smith 原則)
4. 開 PR → AI reviewer (Diamond / CodeRabbit) 跑 first pass
5. AI reviewer 留右上象限 comment (mechanical layer: bug / security / style violation)
6. 你決定哪些是 blocker / nit / false positive, 回應或標記
7. False positive 標 thumbs-down 訓練 reviewer
8. 自己再過一輪 — 確認 alignment layer (這個改動跟 codebase 整體設計合不合)
9. 人類最終 approve (在團隊裡是 reviewer / 一人專案就是你自己延後一天再看一次)
10. Merge

每一步對應一篇 reading 的核心 insight: 1-3 步是 Atwood / Smith 的 review fundamentals; 4-7 步是 AutoCommenter / Graphite 的 AI reviewer engineering; 8-9 步是 Vessels 強調的 mental model synchronization、AI 永遠取代不了的部分。

9.3 Monday Lecture (11/3): AI code review

- Slides: [Google Slides 公開連結](#)
- 講者: Mihail Eric

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

Mihail 開場直接給 statistics 擄住論點: **code review** 是工程師能做的最高槓桿活動之一。

1. **Why – code review 的硬數據** –
 - Code review 的 error detection rate 達 55-60%, 而各類 testing 是 25-45%
 - 一份比較有 vs 沒 review 的程式錯誤研究: 4.5 → 0.82 errors / 100 lines of code
 - AT&T 內部研究: code review 帶來 14% productivity 增加 + 90% defect 減少 (Source: Coding Horror)
2. **What to catch – Review 該抓什麼**: (a) logic & correctness errors、(b) readability & maintainability、(c) performance、(d) security、(e) **best practices / codebase** 內部 **idiom** (例: 用 service 而非 direct DB lookup、特定 access pattern、命名慣例)。
3. **Review 的多重 ROI** (Blake Smith framing) –
 - 對 team members 適應 system 演化的 mental model
 - 確保改動真的解決原問題
 - 開出設計優缺點的討論
 - production 前抓 bug
 - 維持 code style / 組織一致
4. **What's a good review comment** – 對比 anti-pattern 「This won't work」與好 comment: 「I see your new method matches the existing style in this file, taking [X] parameters. Having that many parameters hurts readability and implies the function is doing too much. What do you think about refactoring this method and the existing ones in a later PR to reduce how many parameters they take?」 – 提供具體細節、引用 specific code / issue、suggest resolution、給 evidence / explanation。
5. **The new AI world** – 商業 AI reviewer 景觀: **Graphite**(Friday guest lecture), Greptile、CodeRabbit、Claude Code / Codex。
6. **What has changed – AI reviewer 的六大優勢** (出處 Greptile + Graphite) –
 - **Efficiency**: 自動化降低 review 時間、提早抓問題
 - **Knowledge sharing**: developer 從 AI suggestion 學 best practice
 - **Consistency**: AI 對所有 PR 套同一標準
 - **Reduced cognitive load**: AI 處理 routine check, human 聚焦複雜邏輯
 - **Continuous improvement**: AI system 隨資料量變強
 - **Holistic understanding**: modern AI reviewer 提供 contextual analysis、pattern recognition, 比 linter 深得多
7. **Live demo** – Graphite example 走實 PR review 流程。
8. **Limitations** –
 - 比 linter 多設定 / setup 成本
 - False positive
 - 需要持續訓練 / continuous learning
 - 抓不到 codebase-specific idiom 與 best practice (最大盲區)
 - 處理不了複雜 business logic 與 architecture decision (永遠是 human 的領域)
 - **Security change** 必須額外謹慎
 - 常常漏 edge case
 - **Code review** 在 AI coding system 之下比以前更重要 – 你 own merged + shipped 的 code, 不能 blame AI
 - 必須明確告訴 AI 什麼不要 review、codebase 的 pattern / idiom 是什麼

- 對 user input、authentication、file operation、network request 要特別 mindful

Key takeaway: AI code review 不是替代 human reviewer, 是增強 review 紀律。McConnell 55-60% defect detection 的歷史紅利還在, AI 讓「跑 review」的邊際成本趨近於零, 但「讀懂 codebase idiom + 評斷 architecture」這條 alignment layer 永遠是 human 的責任。AI 寫得越多, review 越不能省。

9.4 Friday Lecture (11/7): Tomas Reimers (Graphite CPO)

- **Speaker:** [Tomas Reimers](#), CPO of Graphite
- **Slides:** [Drive 連結 \(公開\)](#)

以下基於 Drive PDF 公開內容 (pdftotext 抽出) 整理的繁中摘要。Tomas 演講題目「**The Modern Software Developer: Code Review**」, 分四段 (Hi → Collaborating with humans → Collaborating with AI → Software development in the limit):

段 0: Graphite 簡介 — Graphite 是 AI-powered code review platform, help developer create / review / merge code change。AI agent 能回答 PR 相關問題、fully codebase aware、可主動 propose updates 幫 reviewer 解 comment 與 CI。Trusted by 100,000s+ developer、1,000s+ organization。Production 數據: Shopify 採用 Graphite 後 PR shipped per developer +33%、Asana LoC/eng +21%、Ramp time between PRs merged -74%。

段 1: Collaborating with humans — Code review 的歷史

1. 三步驟 collaboration: CREATE (developer 提一組 code change, 這個 atomic unit 叫 pull request、diff、patch、changelist 或 merge request) → REVIEW (另一個 developer 給 comment、suggest improvement、approve) → MERGE (原 developer 把 PR merge 回 codebase)
2. AI 讓 code 比以往多 → 也讓需要 review / merge 的 code 比以往多 (這是 Graphite 整個產品論點的根基)
3. Code review 簡史四階段 —
 - **Fagan Inspections** (IBM, 1976, Michael Fagan 提出) — 重型形式審查
 - **Email patches** (Linux kernel 至今仍用) — printed code → emailed patch
 - **Mondrian** (Google, 2000s 早期, Guido van Rossum 主導) — 第一個 web UI for review
 - **Online tools** (Review Board / Gerrit / Critique / Phabricator / GitHub) — review 從工程紀律變成主流 dev workflow

段 2: Collaborating with AI — 跟 super-human author 共事

4. AI 已能抓的不只 typo — Graphite 主動掃 PR 找 bug、把 potential issue post 到 Graphite 與 GitHub 兩邊、out-of-the-box work、fully customizable
5. 核心問題: 有了 AI, human 還需要 review 嗎? Tomas 答: 要, 因為 review 的 purpose 排序是 (01) Alignment confirmation → (02) Knowledge diffusion → (03) Proofreading。Bug catching 只是第三順位。
6. Human 怎麼跟 AI 合作? — 兩條路 PLATFORM vs PARTICIPANT
 - **PLATFORM** route: AI 較適合做 context gathering、augment human reviewer
 - **PARTICIPANT** route: AI 直接取代 human reviewer
7. **The limits of AI (today vs tomorrow)** — slide 暗示 today 的 AI 邊界明天會被推得更遠

段 3: Software development in the limit — 未來方向

8. **Software dev** 永遠有兩部分 — **Inner loop** (聚焦 development) + **Outer loop** (聚焦 review & collaboration)
9. AI 已經把 inner loop 加速 10× — 那 outer loop 會怎樣?
10. **Three doors for the next generation software dev** —
 - **CYBORG**: developer 直接 review change, 靠 AI 增強。問題: 如果作者不是 author, 這還合理嗎?
 - **EM** (engineering manager): developer 直接管 AI, underwrite architecture 但不細究 technical specifics
 - **AGENCY**: developer 把 AI 當 third-party contractor, underwrite product requirement 但不 own code

11. Graphite 的產品策略對應 —

- **AI Review Agent:** Graphite Agent 是 AI-powered companion 抓 bug、enforce convention、擋 error / security vulnerability
- **Stacking:** 堆疊式 PR git workflow, 讓 author 在不被 reviewer 卡的前提下繼續開發
- **Smarter CI:** predictive CI 只在需要時跑, 省 team 時間與錢
- **Review experience built for teams:** reviewer assignment、merge queue、automation、insight
- **Measure developer productivity:** 精確衡量 AI coding tool 與 Graphite 給 developer 帶來多少 efficiency gain

Key takeaway: Tomas 的核心論點是 AI 不是讓 review 變不需要 — 反而讓 review 變得更重要。Code review 的真正 ROI 一直都在 alignment confirmation 與 knowledge diffusion, bug catching 是 side effect。AI 加速 inner loop 後, outer loop 會在 CYBORG / EM / AGENCY 三條路徑分流, 但無論哪條, 「human underwrite 什麼」決定 review 的形態。對 vibe coder 的 implication: 你 ship 的 code 你負責, AI reviewer 是助力不是替罪羊。

9.5 Reading 摘要

篇名	來源	一句話重點
Code Reviews: Just Do It	Coding Horror (Atwood, 2008)	Review 是性價比最高的 quality practice, inspection 抓 55-60% defect 遠勝 testing 25-45%, 有做就贏
How to Review Code Effectively	GitHub blog (Vessels)	三原則: ask questions over demands、separate substance from preference、provide specific examples
AI-Assisted Assessment of Coding Practices	arXiv 2405.13565 (Google AIware '24)	AutoCommenter 覆蓋 68% best practice, 66% 是 linter 做不到的, 40% resolution rate
AI Code Review Implementation Best Practices	Graphite	Human-in-the-loop + sensitivity tuning + false-positive feedback loop = 可用的 AI reviewer
Code Review Essentials for Software Teams	blakesmith.me (2015)	Review 的核心是 mental model synchronization, scalpel-driven 小 PR + clarifying tone
Lessons from Millions of AI Code Reviews	YouTube (Tomas Reimers, Graphite)	2x2 quadrant: 只留「LLM 能 catch × human 想收」象限; action rate 52% 已追平 human

閱讀優先順序: 時間有限的話, 先讀 [Tomas Reimers YouTube](#) (最具體的 industry insight) → 再讀 [Google AutoCommenter paper](#) (academic baseline) → 補 [Vessels GitHub blog](#) (human-side 原則)。Atwood / Smith 兩篇是經典背景, 建立完 mental model 再回頭讀。

9.6 Assignment: Code Review Reps

- **Source:** github.com/mihail911/modern-software-dev-assignments/tree/master/week7
- **任務描述:** 練習 AI code review 的完整 loop — 拿一個有 bug 的 starter repo、用 Cursor / Claude Code 寫 fix、開 PR、跑 Diamond 或 CodeRabbit、評估 AI reviewer 的 comment 品質 (用 Reimers 的 quadrant 框架打分)、回應 comment、merge。重點是建立「你能判斷哪些 comment 該收、哪些該 dismiss」的肌肉記憶。
- **自學者可行性:** ★★★★★ 完全可做。需 GitHub 帳號 + 一個 AI reviewer service (Graphite 有 free tier、CodeRabbit 個人 plan 約 \$15/月、GitHub Copilot Code Review 內含於 Copilot 訂閱)。預估 4-6 hr — 比 W1-W2 assignment 重, 因為要實際開 PR 跑完整 loop 而非沙盒練習。

💡 沒有付費訂閱的替代方案: (1) GitHub Copilot 學生方案免費, 包含 Copilot Code Review; (2) Graphite Diamond 個人 free tier 每月有 review quota; (3) 用 Claude Code 在 local 模擬: 自己 commit 後對 Claude 說「你現在是 senior code reviewer, 請對這個 diff 留 comment, 限 mechanical layer, 不要留 architectural 建議」。

9.7 對 Vibe Coder 的應用

W7 是把 vibe coder 從「寫得出來」升級到「寫得對且維持得住」的關鍵一週。實戰建議:

1. **side project** 馬上裝一個 **AI reviewer** – 推薦順序: GitHub Copilot Code Review (免費學生方案 / 已訂 Copilot 直接內含) → Graphite Diamond (free tier 夠個人專案用) → CodeRabbit (功能多但要付費)。裝完先別期待奇蹟, 先觀察 false positive 比例兩週
2. **PR description** 永遠寫三段 – 即使一人專案: (a) 這次解決什麼問題、(b) 改了哪幾個 file 跟整體結構變化、(c) 怎麼測 (手測 / unit test / 跑哪個 script)。三個月後的自己跟 AI agent 都會感謝你
3. 學會用 **Reimers quadrant** 篩 **comment** – AI reviewer 留 comment 後, 先問自己兩問: (a) 它抓的這件事 LLM 真的能可靠判斷嗎? (如果是 codebase-specific convention 直接 dismiss) (b) 我願意收這類 comment 嗎? (如果是「請加 test」這種 always-correct-but-unwelcome, 當下 dismiss、之後 batch 處理)
4. **PR** 永遠切 **scalpel-like** 小 cut – 500 行以上 PR 不論 human 還是 AI reviewer 都會 skim 過去; 切成 5 個 100 行 PR 的 review 品質會好 5 倍。Graphite stacked PR 工具就是為此設計
5. 跟 **Claude Code** 的對話加一層 **review reflex** – 每次 Claude 寫完 code, 固定問一句「請以 senior code reviewer 角度檢視剛才的改動, 列出 3 個你會留 comment 的點, 並標記是 blocker / nit / false positive」。這等於把 review 內建進 Claude Code 的 loop, 配合 W6 學的 testing 一起, 能擋掉大量 bug
6. **codebase** 的 **tribal knowledge** 寫進 **CLAUDE.md** – Reimers 點出 LLM 最大盲區是 codebase-specific convention。把你專案的命名慣例、業務邏輯、奇怪的 quirk 寫進 **CLAUDE.md** (同 W1 學的), AI reviewer 會自動讀到 (如果工具支援), 等於把左上象限的盲區補起來

💡 **vibe coder** 的 **Day-1 Quick Win**: 今天就在你 main side project 開一個小 PR (隨便重構一個 function 也好), 裝 Graphite Diamond 或 GitHub Copilot Code Review, 看 AI reviewer 留什麼 comment, 用 Reimers 的 quadrant 對每條 comment 標籤 (右上 / 左上 / 右下 / 左下)。你會發現大部分 comment 落在右下 (always-correct-but-unwelcome) – 那就是你該調 sensitivity 的訊號。一輪下來 30 分鐘, 你對 AI code review 的直覺會完全不一樣。

上一週: [W6 AI Testing and Security](#) | 下一週: [W8 Automated UI and App Building](#)

10. Week 8: Automated UI and App Building

本週你會學到什麼: v0、Lovable、Bolt、Replit Agent、Cursor Composer — 這一波「一句話生整個 app」的工具怎麼運作? 前端設計與 UX 對非設計師變得更容易? 本週由 Vercel (v0 母公司) 的 Head of AI Research Gaspar Garcia 親自開講。

學習目標

10.1

完成本週後, 你應該能:

1. 比較 v0 / Lovable / Bolt / Replit Agent / Cursor Composer 在「一句話生 app」的能力差異
2. 應用 v0 prompt 技巧生成 production-ready React/Next.js component
3. 設計一個「prompt → preview → iterate → deploy」的完整 vibe coding workflow
4. 評估 哪些 app 適合 AI 全自動生成 vs 哪些需要人工架構設計

10.2 核心概念導讀

10.2.1 一、UI generation 的演化: 從「截圖轉 code」到「一句話生 app」

過去三年 UI generation 的能力曲線跳了三個世代。第一代 (2023, v0 1.0): Vercel 把 v0 推出時定位是「screenshot → JSX」, 你貼一張 Figma 截圖或 mockup, 它輸出對應的 React code。可用但有限, 只能處理單一 component、設計一複雜就崩。第二代 (2024, v0 2.0 + 競品爆發): 從 single-component 變 single-page, 從只生 markup 變生帶 state 的互動元件。同時間 Lovable、Bolt、Replit Agent 跳出來, 各自切不同 niche — Bolt 強調 in-browser dev environment、Replit Agent 強調全棧 + 部署、Lovable 強調 design-first。第三代 (2025, v0 3.0 / Cursor Composer / Claude Artifacts): 跨入「一句話生整個 app」— 包含 routing、auth、database schema、API route、UI 全套。能力到 production-grade demo level 但離 production-ready 還有距離 (multi-tenancy、payment、observability 多半還是要人工接)。

理解這個演化曲線, 重點不是記住每個版本能做什麼, 而是看清這條技術曲線的驅動力: (a) LLM model 能力提升 (context window 從 8k → 200k → 1M, 可以塞整個小型 codebase); (b) tool ecosystem 成熟 (Next.js App Router、shadcn/ui、Tailwind v4 等 AI-friendly 框架同時崛起); (c) execution sandbox 成熟 (WebContainers 讓 LLM 能在瀏覽器裡跑 npm install 並 preview)。三者加乘才催生第三代。(基於既有知識的延伸寫作)

10.2.2 二、Design system 與 component library 在 AI 時代的角色重定位

過去 design system (設計系統) 是大公司的奢侈品 — 自己維護一套 token、一套 component、一套 documentation site。AI 時代這個資源結構逆轉: design system 從「成本」變「prompt 槓桿」。當你跟 v0 說「給我一個 dashboard」, 它需要某個 default 來生成 — 那個 default 就是它讀過、訓練過、知道怎麼穩定產出的 component library。

這就是為什麼 shadcn/ui 在 2024-2025 年大爆發。它不是傳統意義上的 component library (不是 npm 包), 而是一套 copy-paste-able 的 React component source code, 建立在 Radix UI primitives + Tailwind CSS 之上。對 AI 友善的三個關鍵設計:

1. **Source 在你 repo:** 不是 black-box 套件, 每個 component 的 code 全在你 codebase 裡, AI 讀 code 直接知道結構, 不用猜 props
2. **Tailwind utility class:** 所有 styling 是 inline class, AI 改 styling 不用追到外部 CSS 檔
3. **Composition over configuration:** 複合元件用 `<Dialog><DialogTrigger /><DialogContent /></Dialog>` 結構, AI 對這種 declarative tree 的 reasoning 比 prop-heavy API 強

對應的,Tailwind CSS 也是 AI-native: 所有 design decision 直接在 markup 裡,沒有「跳到另一個 .css 檔」的 context switch 成本。v0、Cursor Composer、Claude Artifacts 預設都跑 Tailwind + shadcn/ui, 幾乎是業界共識。(基於既有知識的延伸寫作)

10.2.3 三、Prompt-to-app 五大工具的職能差異

把 2025-2026 主要的「自然語言生 app」工具放在同一張地圖上比較, 差異不在「會不會生 code」, 在它替你做了多少 infra 決策:

工具	設計哲學	強項	弱項
v0 (Vercel)	UI-first、generate React/Next.js	元件品質頂、整合 Vercel 部署一鍵到位、shadcn/ui 原生	後端要自己接、不適合非 Next.js 棧
Lovable	Design-first、做 SaaS 起手式	包整套 frontend + Supabase backend、迭代體驗順	棧鎖定(Supabase + React)、客製度有限
Bolt (StackBlitz)	In-browser full-stack IDE	WebContainers 跑 Node.js / npm 完全在 browser、瞬間 preview	Browser 能跑就能寫, 無法做 native / heavy compute
Replit Agent	All-in-one: 寫 + 跑 + 部署	一站完整、適合 0 設定 prototyping、有 cloud DB	Replit 平台鎖定、產出搬到別處要重整
Cursor Composer	IDE 內 multi-file edit	直接在你 local repo 操作、跨 file 改動最強、用熟的工具	沒 preview / 沒 deploy, 純 coding 段

選用直覺:**Demo / prototype** 趕時間 → Lovable 或 Bolt (一句話到能 share 的 link); 自己長期 **side project** → Cursor Composer + v0 (v0 生 UI、Cursor 改 code、Vercel 部署); 全棧 + 不想碰 **infra** → Replit Agent。

第二類差異是「iteration loop 的閉合速度」— 從你說一句話到看到結果的時間。Bolt / Lovable 在 30 秒內, v0 約 1-2 分鐘, Cursor Composer 看你 repo 大小但通常即時。loop 越快, prompt iteration 的次數越多, 最終品質越高。這就是為什麼 v0 / Bolt 把 preview 做進產品本體 — 它們把 iteration loop 內建化。(基於既有知識的延伸寫作)

10.2.4 四、為什麼 Vercel 的全棧押注 (v0 + AI SDK + AI Gateway) 值得單獨理解

Vercel 在這波 prompt-to-app 賽道裡是少數同時做產品(v0) + infrastructure (Vercel platform) + developer SDK (AI SDK) + AI infra (AI Gateway) 四層的玩家。理解這個 stack 才看得懂 Gaspar Garcia 演講背後的策略:

- **v0**: 自然語言 → React/Next.js component / app (前端 UI 層)
- **Vercel AI SDK**: TypeScript SDK, 讓你在自己 app 裡接各家 LLM (OpenAI / Anthropic / Google) 的統一介面, 提供 streaming、tool use、structured output、generative UI 等高階 abstraction
- **AI Gateway**: 跨 LLM provider 的統一 endpoint + observability + cost control + rate limiting, 讓 production app 能在 model 之間 hot-swap
- **Vercel Platform**: edge runtime、serverless function、CDN、preview deployment, 是 v0 生成的 app 部署的 default 目的地

這個 stack 的整合點是 **edge-first generative UI**: v0 生 component → AI SDK 在 edge runtime 串 LLM → AI Gateway 做 multi-provider routing → Vercel 部署。對 vibe coder 的 takeaway: 選 Next.js + Vercel 棧的最大紅利不是 framework 本身, 是這條完整的 AI-native infrastructure pipeline, 每一段都減少你的工程負擔。(基於既有知識的延伸寫作)

10.3 Monday Lecture (11/10): End-to-end apps with a single prompt

- Slides: [Google Slides 公開連結](#)
- 講者: Mihail Eric

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

Mihail 用一句自我吐槽開場: 「 **This is not the vibe coding class** — 因為我是 software development purist. ...except today 」。今天破例專講「 prompt-to-app 」現象。

1. **Why** — 現代 SaaS 已二十多年聚焦 **web app** — Salesforce 1999 通常被視為第一個 SaaS 平台。
2. **The Old Days** — 經典技術棧巡禮 —
 - **LAMP**: Linux + Apache + MySQL + PHP (dynamic site / 傳統 web app)
 - **MERN**: MongoDB + Express + React + Node.js (modern interactive SPA, client-side / server-side / database 三層)
 - **MEVN**: MongoDB + Express + Vue + Node.js (用 Vue 替代 React, 輕量靈活)
 - **JAM stack**: JavaScript(dynamic 功能) + APIs(連後端 service) + Markup(pre-built static HTML)。常見工具 Gatsby、Next.js、Nuxt + headless CMS 像 Contentful、Sanity
 - **Serverless**: frontend (React / Vue / Svelte) + backend (AWS Lambda / Google Cloud Functions) + DB (DynamoDB / Firebase)
 - **AWS stack**: 跨 service combo
3. **Take #1: Visual Low-code / No-code** — Wix、Squarespace、Webflow (pre-AI 時代非工程師建站方案)。
4. **The new AI world** — 第二波生成式 app builder: **Lovable**、**Replit**、**Vercel v0**、**Base44**、**Cursor** / **Claude Code**。
5. **What has changed** —
 - 一句 prompt 生出完整 app
 - 工程師跨職能化 (design、product management)
 - 任何人都能建 app
 - 比第一代 no-code (Webflow、Wix、Squarespace) 更 user-friendly
6. **Live demo: Let's make an app right now** — 用 Bolt 建 frontend + backend, 做一個「 salsa partner finder 」。初次 generation 後再做 iteration: 更新 color palette (給三個 hex code: #EF8354 / #660000 / #EFC408)、改 iconography、改 typography。展示自然語言 iteration 的 fluidity。
7. **App builder architecture** — **WebContainer** — Bolt 等工具的核心是 WebContainer: 在瀏覽器內跑任意 generated code 的 sandbox (Node.js / npm 在瀏覽器跑), 瞬間 preview 不必 deploy。
8. **App builder system prompt 解析** — Mihail 引用兩個 [外洩 Bolt system prompt](#):
 - 明確要求 LLM 只用 established / well-known technology / framework (避免 LLM 用偏門 lib)
 - 用自訂 `<boltartifact>` / `<boltaction>` 標籤標示「在 WebContainer 裡該發生什麼動作(建檔、跑檔等)」
9. **Limitations** —
 - **Everything is awesome** 直到出事就回到原點 — debug 不順時 prompt-driven UI 無法救
 - **Prompt** 是建議不是指令 — 不是每個 user 都懂這個
 - **Security** 已經是個問題 (過去事件)
 - 如何避免所有 app 看起來一樣?
 - 這些 app 真的能撐多複雜?

Key takeaway: Prompt-to-app 的工程本質是 **LLM × component library × execution sandbox (WebContainer) × system prompt 紀律** 的整合。Bolt / Lovable / v0 看起來像魔法, 背後是精心設計的 system prompt (強迫用主流 framework) + 沙盒 (瀏覽器內跑 npm) + 即時 preview。Demo 級別容易、production 級還有距離 — 知道邊界 (complex business logic / 真實 auth / payment / 一致 design) 在哪, 比追新工具更重要。

10.4 Friday Lecture (11/14): Gaspar Garcia (Vercel Head of AI Research)

- **Speaker:** [Gaspar Garcia](#), Head of AI Research at [Vercel](#)
- **Slides:** [Google Slides](#) (需 Stanford 帳號)

Slides 需 Stanford 帳號, 依 Vercel 公開資料 + Gaspar Garcia 角色 best-effort 重建:

Garcia 在 Vercel 領導 AI research, 這場演講預期會橫跨 v0 / AI SDK / AI Gateway 三條產品線:

1. **v0 的設計哲學** – 為什麼 v0 不只生 code、要做 chat-driven iteration loop? 為什麼 default 用 shadcn/ui? v0 V1 → V3 的 product evolution 故事
2. **Generative UI 的範式轉移** – 傳統 UI 是「設計師畫 → 工程師實作 → 用戶看」, generative UI 是「LLM 即時 generate component 給每個用戶看」這不只是 v0 的內部技術, 也是 AI SDK 給 developer 的 abstraction (`generateUI` API、streaming React components)
3. **Vercel AI SDK 的 design decision** – 為什麼選 TypeScript first、為什麼 streaming 是 first-class、為什麼 tool use 跟 generative UI 同一套 API。對比 LangChain / OpenAI SDK 的取捨
4. **AI Gateway 的 production reality** – 為什麼大家最終需要 multi-provider abstraction (cost、rate limit、availability、quality drift)。AI Gateway 怎麼解: unified endpoint + observability + caching + fallback chain
5. **Vercel infrastructure 怎麼支撐 vibe coding** – Edge runtime、preview deployment、Postgres / KV / Blob storage 三件套怎麼讓 v0 生的 app 能 zero-config 上 production
6. **What's next** – Garcia 應該會給 Vercel 對 generative UI / agent infra 的下一步想法 (agent-as-a-service? multi-tenant generative app?)

Key takeaway: v0 不是孤立的「畫 UI 工具」, 是 Vercel 整條 AI-native stack 的 entry point。Vibe coder 用 v0 + AI SDK + Vercel 部署的最大紅利是「從 prompt 到 production 的單一供應商整合」, 把工程負擔壓到趨近於零。

10.5 Reading

本週原 syllabus 沒列 reading list。本講義延伸推薦資源:

- [v0 by Vercel – Official Docs](#)
- [Vercel AI SDK Docs](#)
- [shadcn/ui – The component library AI loves](#)
- [Lovable.dev](#) / [bolt.new](#) / [replit.com/agent](#) (產品比較)

10.6 Assignment: Multi-stack Web App Builds

- **Source:** github.com/mihail911/modern-software-dev-assignments/tree/master/week8
- **任務描述:** 用至少三種不同工具 (建議 v0 + Lovable + Cursor Composer, 或 Bolt + Replit Agent + v0) 各自從同一個 prompt 生成同一款小型 web app (例: todo app with auth、blog with markdown editor、minimal SaaS landing page), 比較產出的 (a) UI 品質、(b) code 結構乾淨度、(c) 從 prompt 到 deploy 的時間、(d) iteration 第二輪的穩定度。最終寫一份 comparison report, 給未來的 vibe coder 同學一份「該用哪個工具」決策指南。
- **自學者可行性:** ★★★★★ 完全可做且強烈推薦做。所有工具都有 free tier: v0 個人 free quota、Lovable 試用、Bolt 在 stackblitz.com 直接用、Replit 個人免費、Cursor 14 天試用。預估 6-10 hr – 這是整門課最動手的 assignment 之一, 做完你會對 prompt-to-app 工具的真实能力邊界有 first-hand 直覺, 而不是 demo 影片裡的感覺。

💡 **加速 tip:** 三個工具不要分開三天做，同一天並行做最有效 — 你能 side-by-side 看到同一 prompt 產出三個截然不同的結果，差異最明顯。比如同一個下午開三個瀏覽器分頁，每個 paste 同一段 prompt，等 30 秒看誰先生完、誰生得對。

10.7 對 Vibe Coder 的應用

W8 是把 vibe coder 從「能寫 component」升級到「能一個人 ship 完整 app」的關鍵一週。實戰建議：

1. 建立 **v0 + Cursor + Vercel** 三件套 workflow — 這是目前最成熟、最便宜（個人 free tier 可走完整 pipeline）、最少 vendor lock-in 的組合。流程：(a) 在 v0 生第一版 UI（用 chat 迭代到滿意）→ (b) 把 v0 產出 export 到本地 repo（v0 有“Open in Cursor”/“Download Code”按鈕）→ (c) Cursor Composer 接手做業務邏輯、API 接、跨 file 改動 → (d) git push 自動觸發 Vercel preview deployment → (e) merge 到 main 自動 production deploy
2. **shadcn/ui** 是 **default** 不要懷疑 — 任何新 React/Next.js 專案開頭直接 `npx shadcn-ui@latest init`。理由 W8 核心概念已說明：source 在你 repo、Tailwind utility、composition pattern 三件事讓 AI 寫起來最穩。其他 component library（MUI、Chakra、Mantine）對 AI 都沒這麼友善
3. **Prompt** 永遠帶 **design system context** — 跟 v0 / Cursor 對話時把 design system 寫進 prompt：「使用 shadcn/ui Card / Dialog / Form 元件，配色用 zinc + emerald，圓角 lg，間距 6/8/12 rhythm」。這比寫「做個漂亮的 dashboard」效果差 10 倍。沒 design system 直覺的話先去 [shadcn/ui blocks](#) 抓一個現成 layout 當 reference，把 reference 截圖貼進 prompt
4. **Iteration 鐵律**：第一版用 **v0 / Lovable** 做粗、第二版進 **Cursor** 做細 — Prompt-to-app 工具在 first generation 是天才、second iteration 開始崩（會回頭刪你已經改好的東西）。穩定的 workflow 是第一版做 **dirty draft** → export 到 **git repo** → 之後所有改動在 **Cursor** 做，把 v0 當「美術初稿生成器」而非「長期 IDE」
5. 不要在 **Replit / Bolt** 養大型專案 — 它們是 prototype 殺手 app，不是長期 dev environment。Demo / hackathon / 一日 MVP 用得很爽，但你 side project 想養三個月以上一定要在 git repo + local IDE + 自選 deployment 平台。Vendor lock-in 越早脫離越好
6. **Design system prompting** 進階技巧 — 把你常用的 design 偏好寫進 **CLAUDE.md** / **.cursorrules** / **v0 system instructions**：「色票限定 zinc + emerald」「字體 Inter」「圓角統一 lg」「動畫用 framer-motion」「icon 用 lucide-react」「不要用 gradient 除非我明說」。每次新 prompt 不必重述，產出一致性會跳一個 level

💡 **vibe coder 的 Day-1 Quick Win:** 今天 30 分鐘做這件事 — 開 [v0.dev](#)，prompt 「Build a clean dashboard with sidebar navigation, top header with user avatar, main area showing 4 stat cards and a data table. Use shadcn/ui, Tailwind, zinc + emerald color scheme, lucide-react icons」。等 1 分鐘看產出。然後點“Open in Cursor”，把產出拉到本地 repo。再 prompt Cursor 「把這個 dashboard 接到 mock data，加一個 dark mode toggle」。最後 [vercel deploy](#) 上線。從零到 production URL 不到一小時 — 這就是你 W8 之後常駐的 baseline workflow。

上一週: [W7 Modern Software Support](#) | 下一週: [W9 Agents Post-Deployment](#)

11. Week 9: Agents Post-Deployment

本週你會學到什麼：把 AI agent 推上 production 之後的世界 — observability、incident response、SRE (Site Reliability Engineering)。當你的服務出事, AI agent 怎麼自動 triage、debug、甚至自動修復? Friday 由 Resolve.ai 的 CTO Mayank Agarwal + Technical Staff Milind Ganjoo 開講(Resolve 在做 AI-native on-call)。

💡 對非資工背景讀者：本週很多概念來自 production engineering 世界 (Kubernetes、observability、SRE)，如果你只做 side project 沒上 production，可以挑「核心概念導讀」+「對 vibe coder 的應用」讀就好。本週會大量用醫療類比 (observability $\approx$ 病歷 + 生命徵象 + 影像、incident response $\approx$ 急救流程、postmortem $\approx$ M&M conference、on-call $\approx$ night float) 幫你建立直覺。

學習目標

11.1

完成本週後，你應該能：

1. 解釋 SRE 的核心三角 (SLI/SLO/SLA) 以及為什麼 observability $\neq$ monitoring
2. 辨識 AI agent 的特殊 observability 需求 (token usage、tool call latency、hallucination rate、context drift)
3. 設計一個 incident response workflow 含 AI auto-triage step
4. 評估 Resolve.ai / PagerDuty AI / Datadog AI agent 在 on-call 場景的能力差異

11.2 核心概念導讀

11.2.1 一、SRE 入門：把營運當軟體工程問題解

W1-W8 講的都是「把 code 寫出來」但 production system 的真正挑戰不在寫，而在「上線之後」— 7×24 小時持續運轉、流量會起伏、依賴會壞、客戶會抱怨、半夜會被 page。在這個世界裡，Google 提出的 SRE (Site Reliability Engineering，網站可靠度工程) 是業界共識的解法。SRE 的核心不是「找一群運維」，而是「找軟體工程師來設計營運團隊，把 manual operation 當成 bug 來自動化掉」
[Google SRE Book](#) 列了三個基礎紀律，所有 production system 都適用：

1. **SLI / SLO / SLA** (**service level indicator / objective / agreement**) — SLI 是測量值 (如 p99 latency = 187 ms)、SLO 是內部目標 (99.9% 請求 < 200 ms)、SLA 是對客戶的合約承諾 (達不到要賠錢)。三者依序由內而外、由緊而鬆。
2. **Error budget** (錯誤預算) — 既然 100% availability 既不可能也無價值 (每加一個 9 成本指數成長)，就明訂 SLO (例：99.99%)，剩下 0.01% 是「可以拿來冒險的預算」。新功能上線會吃預算，預算用完就停止 release。
3. **50% rule** — 工程師花在 toil (重複瑣事) 的時間不得超過一半，另一半必須做工程改善。違反這條 SRE 團隊就會隨流量線性膨脹，跟傳統 sysadmin 一樣。

💡 譯解(醫療類比)：SLO 像住院病人的「目標生命徵象」— 不是要把 BP 維持在 120/80 不准動，是「BP 維持在 100-140/60-90 之間就算 ok」。Error budget 就是「下緣到目標之間的容忍空間」— 病人 BP 在 105 還是 ok，可以做 challenging 的事 (早期 ambulation、調藥)，如果預算用光 (BP 已經在 100)，就先別動。Blame-free postmortem 就像 M&M (morbidity & mortality conference) — 不是要找哪個住院醫師簽錯醫囑，是要找系統性漏洞 (為什麼 EHR 沒擋下這個 dose? 為什麼交班沒講到 allergy?)。

把 SRE mindset 套到 LLM agent: 你的 agent 也會壞 — hallucination、tool call timeout、context overflow、prompt injection。關鍵是先量化什麼叫「ok」 (例：hallucination rate < 0.5%、p95 response time <

5 s、tool success rate > 95%), 然後用 error budget 管理 release 風險(這個 prompt 改完 hallucination rate 飆到 1%, 立刻 rollback)。

11.2.2 二、Observability 三柱: logs / metrics / traces

Observability Basics 釐清一個常被混用的概念: **monitoring** (監控) 只告訴你「什麼壞了」(CPU 高、5xx 多); **observability** (可觀測性) 要回答「為什麼壞」。在 microservices (微服務) 架構下, 一個 user request 可能跨 10+ 個 service, 光看單一 service 的 metrics 完全不知道哪一段慢。Observability 的三柱各司其職, 缺一不可:

柱	是什麼	醫療類比	LLM agent 場景例
Logs (日誌)	離散文字事件記錄 (誰、何時、做了什麼)	病歷紀錄 – 每個診間、每次處置的逐筆紀錄	“User asked X, agent called tool Y, got result Z”
Metrics (指標)	時間序列數值 (QPS、 p99 latency、 error rate)	生命徵象 – HR / BP / SpO ₂ 持續監測	“average tokens per request”、 “tool call success rate”
Traces (追蹤)	一個 request 從進入到回應的完整路徑	影像檢查 (CT / MRI) – 一次性看全身結構與問題位置	“user query → LLM call → MCP tool → DB → 第二次 LLM call” 整條路徑

Trace 由多個 **span** (區段) 組成, 每個 span 是一個工作單位 (一次 DB query、一次 RPC call、一次 LLM call), spans 用 parent-child 關係組成階層。關鍵是 **context propagation** (上下文傳播) – trace ID 必須跨 service 傳遞才能串成完整路徑。**OpenTelemetry** (OTel) 是業界中立的 instrumentation 標準, 所有大廠 (Datadog、New Relic、Honeycomb、Grafana) 都支援, 意味著你 instrument 一次、可以隨時換 vendor。

💡 譯解: 你想 debug 一個 LLM agent 「怎麼會回這個怪答案」 – 看 logs 像翻一頁頁病歷找線索 (耗時、容易漏)、看 metrics 像看 vital sign trend chart (知道某時段惡化但不知為何)、看 traces 像直接拉出那次 request 的「全身 CT」 – 從 user query 進來到 response 出去的每一步、每一步的 input / output / 耗時都看得到。Production agent 沒有 traces 等於 debug 黑箱。

三柱必須相互關聯才有用 – trace ID 要寫進 log line、metric 要帶 trace exemplar, 這樣你看到 metric spike 時能一鍵跳到那個時段的 traces, 看到 trace 異常時能一鍵看相關 logs。

11.2.3 三、AI agent 的特殊 observability 需求

傳統 observability 三柱足夠看 web service。但 LLM agent 多了幾個維度, 傳統工具看不到:

1. **Token usage** (token 消耗) – 每個 request 用了多少 input / output token、累計成本多少。沒看會被 OpenAI / Anthropic 帳單嚇到。
2. **Tool call latency** (工具呼叫延遲) – Agent 一次任務可能 chain 5-10 個 tool call, 每個 tool 的耗時、成功率、failure mode 必須個別 instrument。某個慢 tool 會拖垮整個 agent 的 p95。
3. **Hallucination rate** (幻覺率) – 這個最難量化但最重要。常見做法: (a) 對 critical claim 跑 fact-check pipeline、(b) 抽樣人工審核、(c) 用 LLM-as-judge (另一個 model 當審查員)。
4. **Context drift** (上下文漂移) – 長對話 / 多輪 agent loop 中, context window 累積到後期 model 會偏離 system prompt 的指示 (俗稱 “agent 走鐘”)。需 instrument 每輪的 prompt 大小、context 中各部分占比。
5. **Tool selection accuracy** (工具選擇準確率) – Agent 有 N 個 tool 可用時, 它選對了嗎? 常見指標: 選對率、平均嘗試次數、wrong tool 後恢復率。

💡 譯解: 傳統 web service 的 observability 像加護病房的 monitor — 量 BP / HR / SpO₂ / EtCO₂ 就大致知道病人狀況。LLM agent 多了「精神狀態」這條軸 — 即使 vital sign 都正常, 他可能 confused / disoriented / hallucinating。你需要額外的 GCS (Glasgow Coma Scale)、CAM-ICU (confusion assessment method) 那種專門 instrument 才測得到。Hallucination rate 與 context drift 就是 LLM agent 的「精神狀態評估」。

工具上 [Langfuse](#)、[Helicone](#)、[Arize Phoenix](#) 是專做 LLM observability 的開源/商用方案, 本質上把 trace 模型擴充支援 LLM call 與 tool call, 再加上 token / cost / hallucination eval 維度。

11.2.4 四、Multi-agent system 在 SRE: MDT 會議模式

[Multi-Agent Systems for AI-Native Engineering](#) 提出一個關鍵 observation: **single agent** 在 **production debugging** 是 **sequential bottleneck** (序列瓶頸) — 它一次只能調查一個假設, 但 production incident 的時間壓力要求並行假設驗證。

解法是 multi-agent 架構, 讓專業 agent 各司其職並行運作。一個典型的 incident response multi-agent system 長這樣:

Agent	專長	輸出
Trace agent	追跨 service 的 distributed trace	「 latency spike 出現在 service B 的 DB call 」
DB agent	分析 DB performance、slow query	「 過去 10 分鐘有一支 N+1 query 」
Deployment agent	review 最近 CI/CD 部署紀錄	「 30 分鐘前 service B 上了 commit abc123 」
Code diff agent	看那個 commit 的 code change	「 abc123 移除了 query 的 index hint 」
Customer impact agent	估算影響範圍	「 過去 10 分鐘有 12,000 user 受影響 」
Coordinator agent	merge 上述發現給結論	「 root cause: abc123 commit 的 query regression, 建議 rollback 」

💡 譯解(MDT 會議類比): 這個架構就是醫院 MDT (multidisciplinary team) 會議的軟體版 — 一個 lung cancer 病人的 case, 心臟科評估手術風險、腫瘤科出 chemo 方案、放射科出 RT 計畫、病理科確認 diagnosis、社工評估家庭支持, 最後 lead physician (coordinator) 綜合所有意見開會 merge 結論。每個專家獨立、並行作業,最後一次 merge — 比一個 generalist 順序問每個專家快得多。Production incident 也是多 domain 問題, multi-agent 並行調查比 single agent sequential 快數倍。

但並行的代價是需要 **formal coordination protocol** 來避免 race condition 與 deadlock — 這也是 MDT 會議要有 chair 主持、有 agenda、有 timing 規則的原因。把這套作出 production-ready 需要罕見的雙重專業: 深度 production domain knowledge + sophisticated AI architecture, 缺一邊就會做出「會調查但調查錯方向」的系統。Resolve.ai 的賣點就是兩邊都有。

11.3 Monday Lecture (11/17): Incident response and DevOps

- Slides: [Google Slides 公開連結](#)
- 講者: Mihail Eric

以下基於 Google Slides 公開內容 (TXT export) 整理的繁中摘要:

Mihail 開場直接給出 framing 數據「Coding represents just 30% of engineering time. 難搞的 70% 是把 code 跑在 production 裡 – 那裡 complexity / tool silo / knowledge gap / interdependency 全 collide」。

1. **The old world – SRE 的痛點清單 –**
 - Operational monitoring: on-call、troubleshoot、infrastructure management、security
 - Incident resolution 需要從多 source / 多 team 拼湊資訊
 - 維護常常已過時的 runbook
 - Cloud-native + Kubernetes + 容器化把 data / dependency / complexity 推到新高度
 - SRE 因 on-call shift 普遍 burnout
 - **estimates that downtime and service degradation cost the Global 2000 about \$400 billion annually**
2. **Infrastructure & DevOps 的核心紀律: Four Golden Signals of monitoring**
 - **Latency:** 分開追蹤 successful 與 failed request (避免 HTTP 500 失敗讓平均失真, 慢 error 特別需要關注)
 - **Traffic:** system demand, 通常 req/sec、依系統決定 (streaming 算 session、DB 算 transaction)
 - **Errors:** 失敗率, 包含明確 (HTTP 500)、隱含 (200 OK 但 wrong content)、policy-based (超過 SLA)。需多層 monitoring 抓不同 failure
 - **Saturation:** 「系統滿到什麼程度」, 追蹤 CPU / memory / I/O。System 通常在 100% 之前就慢, 要訂 safe threshold
 - 加上: monitor production trace
3. **凌晨 3:12 的實境演練 – PagerDuty ping 你 DB query 出現 500 spike, 怎麼辦? Mihail 給一份 8 步驟 incident playbook:**
 - **Acknowledge & assess:** 在 PagerDuty acknowledge、確認嚴重度、看 app + DB dashboard、判斷 partial / full outage
 - **Check Golden Signals (DB first):** connection 是否爆 / 卡 / 突然掉? P95 latency spike? Slow query? timeout / refused / aborted transaction? CPU / IOPS / lock / memory / replication lag?
 - **Look for what changed:** 最近 deploy? DB migration? Config / feature flag 改? autoscaling 事件? – 有 correlation 立刻 rollback / revert
 - **Localize failure:** 所有 query 都壞 vs 特定 query? read 還是 write? primary 還是 replica? 單 shard / instance 還是全部? App-side (pool / timeout) 還是 DB-side (load / lock)?
 - **Apply fast mitigation:** connection issue → restart pod / 降 concurrency; DB saturation → 關 heavy job / throttle traffic / read 路由到 replica; bad slow query → 關該 feature / 開 cached degraded mode; replica lag → read 路由到 primary / 重啟 replica; unhealthy node → 只重啟 replica, primary 要 escalate
 - **Stabilize & monitor:** 看 500 rate / DB latency / traffic 回正、確認沒 retry storm 與 cascading failure、health check 恢復
 - **Communicate:** 每 10-15 分鐘一次簡短 update (issue → action → status)
 - **Close out:** 時間軸、root cause、follow-up (query fix、indexing、scaling、retry tuning、capacity review)
4. **Metrics tracked – MTTR (mean time to repair)、被拉進 incident 的 engineer 數量、對 customer 的 reported SLA。**
5. **The new AI world – Resolve AI (W9 Friday guest)、Datadog Bits AI Agent、Splunk Observability Assistant。**
6. **AI SRE 的特徵 –**
 - 動態維護 knowledge graph
 - 跨 observability stack 與 cloud 的 agentic system
 - 即時生成「現在發生什麼」的 narrative、pinpoint likely root cause + 證據、給 prescriptive remediation
 - **Heavy emphasis on explainability and auditability**
7. **What has changed – AI 把 organizational / service-level knowledge scale 出來 – 那些 undocumented dependency、brittle legacy service、只會在 high-stakes incident 浮現的 quirk, 不再被孤立在某幾個資深 engineer 腦中。**

8. **AI SRE in action** 預覽 — observability、working theory、span info、heavily evidence-based、chat 介面做動態查詢。
9. **Limitations** —
 - 能處理 incident 的複雜度有限
 - 現代 production stack 異質性
 - 從偵測到實際 remediate code 還很遠 (all provider 都先做 root cause analysis 起)
 - 好的 RCA 需要好的 monitoring 「園藝」(gardening) — 工具雜草不除 AI 也救不了你
 - Security 可能變新 attack vector

Key takeaway: SRE 工作流 (detect → check golden signals → look for change → localize → mitigate → stabilize → communicate → close out) 是個成熟 8 段 playbook。AI agent 不是要重新發明它，是要在前面 5 段大幅縮短時間 — 從凌晨 3:12 page 到 4:00 找到 root cause，從原本的「多 engineer + tool silo + 拼湊資訊」變成「single agent + dynamic knowledge graph + evidence-first」。但 remediation 邊界還在，blast radius 大的動作必須 human authorize。

11.4 Friday Lecture (11/21): Mayank Agarwal + Milind Ganjoo (Resolve)

- **Speakers:**
 - [Mayank Agarwal](#), Founder & CTO of [Resolve AI](#) (**OpenTelemetry creator**)
 - [Milind Ganjoo](#), Member of Technical Staff at Resolve AI (**ex-DeepMind Staff MLE**)
- **Slides:** [Drive 連結 \(公開\)](#)

以下基於 Drive PDF 公開內容 (pdftotext 抽出) 整理的繁中摘要。演講題目「**Agentic AI for software in production**」:

段 1: 軟體工程的真實樣貌 (不是 idealized)

軟體工程比想像複雜: 跨 systems (code、AI、telemetry、cloud、knowledge、security)、跨 teams (application、infra、networking)、跨 workflows (development、deployment、on-call、cost management、compliance、security vulnerability、documentation)。Software engineer 花 70+% 時間在 **grunt work**: building context、working across tools、evidence gathering、log queries、coordination、application、documentation、compliance、on-call、optimization、deployment、problem solving — 只有少部分時間在 design decision、trade-off、creative work。

段 2: On-call engineer 在凌晨 3:04 被 page 後到底發生什麼

走 timeline: 03:04 AM page → 03:20 AM L1 support 接手 → 04:00 AM multiple team escalation (infra / app & product / DB / engineering manager / incident commander) → 04:45 AM 工程師終於把問題 mitigate → ? AM postmortem。整段過程要靠 runbook、observability、code、infra 拼湊，**nearly all manual effort** — incident commander 協調人、L1 看 infra、app & product team 看 application、director / comms manager 對外溝通。

段 3: 什麼讓 production 對人類(與 model) 來說都很難?

1. 跨多 system / tool 的 data silo — 1000s of service across 100s of team、複雜瞬息的 infra (DB、messaging service...)、低層工具看 log / metric / dashboard / feature flag / CI-CD 各自 query language、access mechanism、operational behavior
2. 跨 team / 跨 expertise 的協調 — application engineer、platform engineer、SRE、IT ops、security engineer、support engineer 各有專長，但 context 通常 fragmented and undocumented
3. 直接影響營收與成本 — incident 拖好幾小時或好幾天才解掉、tool 維護成本高、incident regularly 牽涉 20+ engineer、change 難做又會 trigger issue、infrastructure 開支不斷漲、客戶常常先發現問題、只有少數工程師真的懂 production 全貌、新人 onboard 要 3-6 個月、AI 生成的 code 又把問題放大

段 4: AI 要怎麼幫工程師管 production?

Resolve 的 Agent-first approach 結合三件事: (1) understand and operate all your production tools, (2) capture tribal knowledge of your unique system, (3) combine expertise of all your engineers across team. Mayank 把這拆成 3 個設計原則對應到 3 個產品 capability:

設計原則 1: **Production** 系統複雜且持續變化 → AI 要深度理解 production - 接到 code、infra、tool、knowledge - 為「你的系統」建模(不是 generic SRE, 是你公司的 architecture) - 在 graph 內導航到對的 node 蒐證據 - 像專家一樣操作每個 tool / system - AI 連接 alerts / metric / dashboards / traces / logs / runbooks / change events 等多源訊號

設計原則 2: **Knowledge fragmented or undocumented** → AI 要 capture tribal knowledge 並逐次變聰明 - Capture 公司與 team 的知識 - 記得 in-the-loop 給的 feedback / teaching - Retrieve context-specific information

設計原則 3: 調查需要跨 team 專長 → AI 要結合所有 engineer 的專長 - Create investigation plan - 並行 pursue 多個 hypothesis (multi-agent 並行查不同方向) - 持續 refine plan 到 root cause - 讓多人跨 org 邊界協作得到答案

段 5: Lessons learned building AI for prod

1. 這不只是 **model** 問題 - production 導航需要大量 domain expertise, 這些必須 hard-code 進 architecture, 單靠 **prompt engineering** 蓋不出 **production AI**
2. **Context window** 有限, **production context** 是無限的 - 10M log line 不可能塞進任何 context window. Intelligence 是「**knowing WHAT to query, WHEN, and HOW to filter**」based on production understanding
3. 跟 **tool** 工作是 **non-trivial** 問題 - Raw API 不可用: response 太大、output 雜、為人類設計。必須建 AI system 能 filter noise、回 structured summary、graceful 處理 error、parallel 工作
4. **Eval** 跟 **product** 一樣難 - 建 eval 要重現 production complexity (service、dependency 等), 沒 eval 就不能信任 output

段 6: AI is changing software engineering

Mayank 預測「By next year software engineering will look fundamentally different」。Three eras 的 grunt work / creative work 比例: - **Models era** (純 model API): grunt work 主導 - **Agents era**: grunt work 還是大頭,但 creative work 占比上升 - **Closed-loop agents era**: grunt work 大幅縮減, creative work 占主導

Key takeaway: Resolve 對 vibe coder 的最大 lesson 不是「怎麼用 AI agent」, 是「**production AI** 為什麼不能靠 **prompt engineering** 做出來」。它需要 (a) hard-code domain expertise into architecture, (b) 知道何時 / 怎麼 query / filter 而非塞 raw data, (c) 把 messy raw API 包成 AI-friendly tool, (d) eval 跟 product 一樣難建。對未來 AI vertical startup 的 founder 是必看的工程現實 check - production engineering 是 LLM agent 的 killer app, 但門檻比一般 chat agent 高一個量級。對自己的 side project, 至少先學會這套「接 alert → 檢查 golden signal → 看最近改了什麼 → localize → mitigate」的 8 步驟 playbook, 自己當自己的 SRE。

11.5 Reading 摘要

篇名	來源	一句話重點
Introduction to SRE	Google SRE Book	SRE = 把營運當軟體工程問題; error budget 量化 dev/ops 衝突, blame-free postmortem 找系統漏洞
Observability Basics	last9.io	Observability 三柱: logs (病歷) + metrics (生命徵象) + traces (影像); context propagation 是關鍵
Kubernetes Troubleshooting with AI	resolve.ai	K8s 三大痛點 (alert fatigue、ephemeral context、observability fragmentation) 由 AI agent + knowledge graph 解

篇名	來源	一句話重點
Your New Autonomous Teammate	resolve.ai	Resolve 產品深度導覽: dynamic knowledge graph + just-in-time runbook + 1 分鐘 root cause + 自動 postmortem
Multi-Agent Systems for AI-Native Engineering	resolve.ai	Single agent 是 sequential bottleneck; multi-agent 並行查 root cause 是 AI-native 工程的核心
Top 5 Benefits of Agentic AI in On-call	resolve.ai	五大好處: 消 alert fatigue、活知識、調查一致性、證據式協作、主動找潛在問題

閱讀優先順序: 先讀 [Introduction to SRE](#) (建立 mental model) → [Observability Basics](#) (基礎工具觀念) → [Multi-Agent Systems](#) (agent 架構) → [Your New Autonomous Teammate](#) (具體商業案例), 時間有限的话前三篇必讀。

11.6 Assignment

本週原 syllabus 沒列 weekly assignment。建議自學者用 Sentry 或 Better Stack 給自己的 side project 設定 observability 當練習: (1) 開一個帳號(兩家都有 free tier) → (2) 接到自己的 side project (npm/pip 裝 SDK, 5 分鐘) → (3) 故意製造一個 error 看 dashboard 抓得到嗎 → (4) 設一個 alert (例: error rate > 1%) 發到 Discord / email → (5) 觸發後跑一次完整 incident response 流程 (看 trace 找 root cause → fix → 寫 postmortem)。預估 2-3 hr, 會建立完整 production-ready 的肌肉記憶。

11.7 對 Vibe Coder 的應用

W9 是 vibe coder 最容易跳過、但跳過後悔最大的一週。多數人 ship side project 時根本沒 observability, 等到出 bug 才發現自己什麼都看不到、只能憑直覺猜。這週的概念套到日常工作流:

1. 第一個 **production project** 立刻接 **Sentry** — Sentry 對 vibe coder 是 P1 夯。Free tier 給每月 5K event, 足夠 hobby project 用。npm install 後 3 行 code 就接好, 自動抓 unhandled exception、含 stack trace、含 source map (看到原 TypeScript 而不是 minified JS)。沒有 Sentry 等於沒儀表板開車
2. 三選一: **Sentry / Better Stack / Vercel Analytics** —
 - **Sentry** 主攻 error tracking + performance traces, all-rounder。最 P1, 預設選它
 - **Better Stack** (原 Logtail + Better Uptime) 主攻 log aggregation + uptime monitoring, 介面漂亮、SQL-like log query 強。如果你 log 量大選它
 - **Vercel Analytics** 只在 deploy 到 Vercel 時用、focus 在 web vitals + page view, 是「最低限觀測」。配 Sentry 用不衝突
3. **Day 1** 就加 **structured logging** — 別用 `console.log("user clicked")`, 用 JSON log 含 request ID / user ID / timestamp / event: `logger.info({ user_id, action: "click", item: "checkout" })`。Sentry / Better Stack 都會自動 parse JSON log 的 field 變成可篩選的 metadata。改寫 0 成本、回收高得驚人
4. **LLM app** 加 **Langfuse / Helicone** — 如果你 side project 用了 OpenAI / Anthropic API, 標準 observability 看不到 token usage / hallucination。Langfuse 開源、self-host 免費、5 分鐘接 SDK, 給你完整的 prompt + response + tool call + cost 紀錄。一個月後回頭看, 會發現 80% 的 cost 來自你想不到的 5% request — 沒這個資料無法 optimize
5. **Production-ready** 的觸發點不是「有用戶」是「你會半夜被 page」— 多數 vibe project 永遠不會到那個點, 所以別過度工程化。只要 (a) error tracking、(b) basic uptime check (Better Stack free tier 給 10 個 monitor), 就涵蓋 90% 的 production-ready 需求。Kubernetes、Datadog、PagerDuty 全是 over-engineering — 等到你的 side project 真的有 paying customer 再上
6. **Postmortem** 文化套到自己 side project — 每次 production bug 寫 30 字 postmortem (root cause + fix + lesson) 存進 repo 的 `POSTMORTEMS.md`。半年後看會發現自己一直在重蹈覆轍同類 bug, 這個 doc 就是個人版的 dynamic knowledge graph

💡 **vibe coder 的 Day-1 Quick Win:** 今天去 sentry.io 開帳號(30 秒)、選你最在意的那個 side project、跟著 5 分鐘 setup wizard 接好 SDK、deploy。下次 bug 不必再從 user 抱怨「我點了沒反應」開始 debug — Sentry 會直接 email 你 stack trace + 出錯時的 user / request context, 可能還沒等 user 抱怨就已經修好。這個 ROI 是所有 dev tool 中最高的之一。

上一週: [W8 Automated UI and App Building](#) | 下一週: [W10 What's Next](#)

12. Week 10: What's Next for AI Software Engineering

本週你會學到什麼：10 年後軟體工程師長什麼樣？「junior dev」會被 AI 取代嗎？emerging paradigm 有哪些（agentic CI/CD、formal spec、autonomous open-source contribution）？本週由 a16z（Andreessen Horowitz，矽谷指標 VC）的 General Partner Martin Casado 開講，從投資人視角看產業 10 年。

💡 本週特別說明：W10 原 syllabus 沒有 reading list、Mon lecture 也沒公開 slides — 內容偏 forward-looking speculation + 講者個人 thesis。本講義以 W1-W9 學到的 9 週素材為基礎，結合 a16z 公開 essays 與業界共識做延伸寫作，所有預測都標註（基於既有知識的延伸寫作）並避免聲稱「Stanford 課堂這樣講」。

學習目標

12.1

完成本週後，你應該能：

1. 辨識 software dev role 在 AI 時代的 5 種可能演化路徑（augmentation / specialization / abstraction / replacement / new roles）
2. 解釋 emerging paradigms（spec-driven dev、formal verification × AI、autonomous repo maintenance）的技術前提
3. 評估自己的 skill stack 在未來 5 年的相對價值（哪些 skill 會 commoditize、哪些會更值錢）
4. 預測 AI dev tool 市場的下一個 winner pattern（platform vs vertical、open vs closed）

12.2 核心概念導讀

12.2.1 一、Software Dev Role 的 5 種演化路徑

「AI 會不會取代軟體工程師？」這個問題問了 3 年，答案從來不是 yes/no — 是 role 會分裂成幾種典型形態。從 W1-W9 的素材抽絲剝繭，可以看到 5 條互不互斥的演化路徑：

1. **Augmentation**（增強）— 大多數人會走的路 — 不換工作，把 Claude Code / Cursor / Warp 整合進現有工作流，產出 / 時間提升 30-100%。需要的新 skill：context engineering、agent management（W4）、prompt-driven debugging（W6/W7）。這條路最 boring 但 ROI 最高
2. **Specialization**（專業化）— 變成「某類 AI tool 的世界級用戶」— 例：成為 Claude Code subagent / skill 系統的專家，或 xG prompting 的世界級操作者、或 Vercel AI SDK 的 community contributor。這條路有「先發優勢」但會被後來者追平
3. **Abstraction Up**（抽象上層）— 從 **implementer** 轉 **architect** / **agent manager** — 不再寫 code，全心做 spec design、acceptance criteria、verification 設計、人機分工策略。對應 W3 的 spec-driven 命題與 W4 的 agent manager mindset
4. **Replacement**（取代）— 真實但範圍有限 — 不是取代「軟體工程師」這個 role，是取代「寫 boilerplate / 補測試 / 改格式 / 做 routine refactor」這些 task。受影響最大：junior dev 在 boilerplate-heavy 公司、外包 contract programmer。受影響最小：產品經理、tech lead、staff+ engineer
5. **New roles**（新角色）— 已經出現 — AI Engineer（W2-W4 的 agent / MCP 操作者）、Forward Deployed Engineer（在客戶端配合 AI 解決方案的工程師）、AI Reliability Engineer（W9 的 AI-native SRE）、Prompt Architect（資深 prompt 設計師）、Eval Engineer（評測 AI system 的專家）

對使用者個人 strategic（醫學生 + RA + vibe coding hobby）的對應：路徑 1 是基本盤（Claude Code 整合進醫學研究 workflow）、路徑 2 可選（成為「醫學 + AI tool」的早期專家）、路徑 5 跨領域（醫學 AI engineer、clinical AI ops 都是 emerging niche）。（基於既有知識的延伸寫作）

12.2.2 二、Emerging Paradigms: Spec-driven Dev / Formal Verification × AI / Autonomous Repo Maintenance

W3 已經提了 spec-driven dev 的概念，W10 把它放進更大的 paradigm shift 圖裡。3 個正在浮現的範式變化值得注意：

1. **Spec-driven Development**（規格驅動開發）走向主流 - W3 的 [Specs Are the New Source Code](#) 是先驅論文。實際走勢：未來 5 年大公司（Google、Meta）會把 PRD / design doc 轉成可被 agent 直接 implement 的 formal spec - 副作用：PM 跟 designer 的工作會變硬 - 寫得糊的 spec 直接沒結果，逼大家寫得精準 - Vibe coder 對應：把寫 PRD 當 first-class skill 練（不是 nice-to-have）

2. **Formal Verification × AI** 的二度興起 - 2010 年代 formal verification 因為手寫太難失敗。2025-2030 年 LLM 把證明重擔自動化，formal methods 在 safety-critical（醫療、金融、自駕、aerospace）領域會大爆發 - W6 的 [Vulnerability Prompt Analysis with O3](#) 是早期信號 - LLM 已經能做 formal-ish vulnerability analysis - 對醫療 AI 的長尾影響：FDA 過審 AI 醫材會強制要 formal verification step

3. **Autonomous Open-source Maintenance** - 已有早期跡象：Dependabot、Renovate（dependency 自動 upgrade）、CodeRabbit / Diamond（W7 PR review）、Devin（Cognition 的自動 PR 機器人）。下一步：autonomous bug fix、autonomous feature backport、autonomous release management - 風險：低品質 / spam PR 暴增，maintainer 變成「reviewer of reviewers」 - Vibe coder 對應：你的 side project repo 提早設定 AI reviewer + auto upgrade，用得越早越熟（基於既有知識的延伸寫作）

12.2.3 三、AI Dev Tool 市場的 Winner Pattern

W2-W9 串連起來看，AI dev tool 市場已經分成 4 個 layer，每層的 winner pattern 不同：

Layer	範例(2026 強者)	Winner Pattern
Foundation models	Anthropic (Claude)、OpenAI (GPT)、Google (Gemini)	寡占 - 訓練成本高、moat 在 model 與 evals
Coding agents / IDE	Claude Code、Cursor、Devin、Warp	寡占走向碎片化 - 每個 niche(CLI / IDE / autonomous) 會有 1-2 強者
Vertical tools	Graphite (W7 code review)、Resolve (W9 SRE)、v0 (W8 UI)	Vertical SaaS - 每個 vertical 1-2 個專業玩家
Infrastructure	Vercel (AI SDK + Gateway)、Cloudflare、Modal	Platform-led - 跟 cloud provider 整合越深越強

幾個跨層 trend： - **Vertical** 比 **horizontal** 賺錢 - 一個 code review 公司（Graphite）、一個 on-call 公司（Resolve）、一個 UI 公司（v0）能各自撐 unicorn valuation；做「all-in-one AI dev tool」的反而難變現 - **Open source** 是漲粉、不是商業模式 - Claude Code / Cursor 都不開源核心，但開放擴充（subagent、skill、extension）；MCP 協定開源（吸引 ecosystem），客戶端產品閉源（保 monetization） - **Foundation model** 是 **commoditizing** - Anthropic / OpenAI / Google 三家差距在縮小，wrapper 產品（Cursor、Devin）會越來越用 router 在三家間切換以降本（W6 的 Context Rot 也說明 model 之間差距已經不大）

對使用者個人決策的 takeaway：別把學習投資押在單一 model（Claude vs GPT 之爭已經 boring），押在跨 **model abstraction** 層（MCP、Vercel AI SDK、LangChain）跟 **vertical workflow**（你的醫療研究 pipeline、阿摩錯題迴圈、IRB 自動化） - 這兩層的價值不會被 model 變遷洗掉。（基於既有知識的延伸寫作）

12.2.4 四、a16z 的 AI Dev Tool 投資 Thesis

Martin Casado 在 a16z 主導 American Dynamism + Infrastructure / AI 的投資組合。從他公開 essays（“The End of Cloud Computing”、“AI 50” series）和 a16z 對 Cognition（Devin）、Cursor（Anysphere）、各 AI dev tool 公司的投資紀錄，可以推出他的 4 條 thesis：

1. **Infrastructure layer** 比 **application layer** 大 — Cloud computing 的 winner 是 AWS / GCP / Azure 而非 SaaS 應用。同理 AI 時代的 winner 不會是 chatbot 應用,是 Vercel / Cloudflare / Modal 這類 AI infra
2. **Agentic platform** = 下一個 **unicorn factory** — 不只是「AI 寫 code」, 是「AI 自主完成 task 並付費」。Devin (autonomous coding)、Resolve (autonomous on-call) 都在這條軸線上
3. **Vertical AI** > **horizontal AI** — 醫療 AI、法律 AI、code review AI、SRE AI 各自會出 unicorn, 比通用 chatbot 更穩
4. **Talent geography** 大洗牌 — AI 解放 talent 不必住 SF Bay。a16z 的 American Dynamism 主題押注是「全美都會有 AI startup」。對台灣 / 亞洲:英文良好的 AI engineer 在 remote / async 模式下能直接接美國公司 contract, 門檻變低

對使用者長期 career bet 的對應: - 押 **vertical AI** (醫學 + AI 的交集) 而非 generic AI - 押 **infra layer** 工具 (會用 MCP / Vercel AI SDK / observability 三件套) 而非單一 wrapper 產品 - 遠端 / 自由 contracting 比 traditional FTE 路線在 AI 時代 ROI 更高
(基於既有知識的延伸寫作)

12.3 Monday Lecture (12/1): Software development in 10 years

- Slides: 未公開
- 講者: Mihail Eric

Slides 未公開, 依 lecture title + W1-W9 累積的素材 best-effort 推測:

這節是課程收尾的個人 prediction 演講。預期內容:

1. **Recap**: W1-W9 串成一條線 — 從「LLM 是什麼」(W1) 到「LLM 在 production 怎麼跑」(W9) 的 9 週技術疊加,最後落到「10 年後是什麼樣」
2. 3 個 **ten-year prediction** — Mihail 應該會給幾個具體預測(例:「10 年後 80% 的 PR 是 AI 開的」 「software engineer 平均薪水降但人數不變」 「formal verification 變必修」,每個附 reasoning chain
3. **What stays valuable** — 哪些 skill 不會被 commoditize: spec design、agent management、cross-domain knowledge、taste / judgment、organizational influence
4. **What gets cheap** — 哪些 skill 會被 AI 接手: boilerplate generation、test coverage、style enforcement、documentation、basic refactoring
5. 教育與 **onboarding** 的衝擊 — 沒有 boilerplate 練習,junior dev 怎麼從 zero 上手? 大學 CS 教育要怎麼改?
6. **Open Q&A** — 這類課程末尾的 reflection lecture 通常會留大量時間給學生問問題

Key takeaway: 10 年後的軟體工程不是「沒有工程師」, 是「工程師的工作 mix 變了」— 寫 code 比例下降、設計 / 審 / 管理 agent 比例上升。早一年 adapt 比晚一年值很多。(基於既有知識的延伸寫作)

12.4 Friday Lecture (12/5): Martin Casado (a16z General Partner)

- Speaker: [Martin Casado](#), General Partner at [a16z](#)
- Slides: 未公開

Slides 未公開, 依 Casado 公開 essays + a16z 投資組合 best-effort 重建:

Casado 的研究背景(Lawrence Livermore National Lab, Stanford CS PhD, Nicira 創辦人,後賣給 VMware \$1.26B) 讓他擅長把工程深度與 VC thesis 串接。預期演講:

1. **Cloud** → **AI** 的 **historical analogy** — 把 2007-2015 的 cloud computing winner pattern 套到 2023-2030 的 AI dev tool。哪些教訓直接適用、哪些不適用
2. **Why infrastructure wins** — 為什麼 AWS / GCP / Azure 在 cloud 是 winner? 為什麼 Casado 認為 AI infra (Vercel / Modal / Replicate / Cloudflare) 會是下一波 winner, 而非 application 層

3. **Devin / Cursor / Anysphere 投資 case** — a16z 投了 Cognition (Devin), Anysphere (Cursor), 其他 AI dev tool 公司。每個 case 的 thesis (為什麼這個 team / 這個 timing / 這個市場)
4. **Agentic AI 的 trillion-dollar opportunity** — Agent 不只是工具, 是「能自主完成 task 並付費」的 economic actor。這個轉變創造的市場是傳統 SaaS 的 10 倍
5. **What founders should build now** — 給 Stanford 學生的建議: 哪些 niche 還空著、哪些 problem 真的值得解、什麼樣的 founder background 在這波會贏
6. **a16z 招募 founder 的訊號** — Casado 應該會 implicit 招手 — 這類演講常常是「you should build, talk to me」

Key takeaway: AI 不是泡沫也不是炒作, 是 cloud-scale 的 platform shift。這次的 platform 是 agent, winner 是有深度技術 + 工程 craft 的人。對 vibe coder 個人: 把現在當成 1995 年 (網際網路啟動) 或 2007 年 (cloud computing 啟動) — 早一年 commit 後面差很多。(基於既有知識的延伸寫作)

12.5 Reading

本週原 syllabus 沒列 reading list。本講義延伸推薦資源:

- [Martin Casado on a16z author page](#) — 看他過去 essays, 特別是 “The End of Cloud Computing” 與 AI infra 系列
- [a16z American Dynamism / AI portfolio](#) — 看 Cognition, Cursor, Anysphere 等 AI dev tool 投資
- [Gartner Hype Cycle for AI 2025](#) — 對照 emerging paradigm 的 hype vs reality
- [Stack Overflow Developer Survey 2025](#) — 看 dev 對 AI tool 的 adoption / sentiment

12.6 Assignment

本週原 syllabus 沒列 weekly assignment (Final Project 應該已進入 demo 階段)。

12.7 對 Vibe Coder 的應用

W10 不是 actionable week, 是 strategic week — 思考「3 年後我要在哪」而不是「今天我做什麼」。給自己的問題:

1. 挑 3-year skill bet — 哪 3 個 skill 你會 deeply 投資 1000+ 小時? 建議: (a) **context engineering** (W3/W4 的 spec / CLAUDE.md / subagent 紀律 — 跨 model 都受用), (b) **vertical workflow design** (你的醫學研究 / 阿摩 / IRB 自動化 pipeline — 別人複製不了的領域知識), (c) **agent observability** (W9 的 LLM eval / hallucination 監控 — 任何 production AI 都需要的)
2. 挑 3 件不再學的事 — 隨意: (a) 手寫 boilerplate (讓 v0 / Claude Code 處理), (b) 追每個新 framework (鎖定 1-2 套深用就好), (c) 跟 model 比快 (你比 LLM 慢但比 LLM 有 taste)
3. **Side project future-proofing** — 你的 vibe coding side project 要做到「3 年後不會被 AI tool 進化淘汰」原則: (a) 不做純技術 demo (會被新 model 立刻超越), (b) 要做「跟你個人 taste / 領域知識 / niche 結合」的東西, (c) 接 MCP / 走標準 protocol 而非 vendor-locked
4. **Portfolio 差異化** — 純會用 Cursor / Claude Code 已經不是差異化 (人人會)。差異化來自: deep cross-domain (醫學 + AI), open source contribution (社群影響力), production-grade product (不是 demo) 三個方向選 1-2 個押
5. **Geographic optionality** — Casado 的 American Dynamism thesis 對台灣有解放性影響: 英文良好 + 遠端 + AI fluent 的 engineer 能直接接美國 contract / remote FTE。把英文寫作 / 同步溝通練好, 是 ROI 最高的 non-technical skill
6. **時程管理**: 每年 review 一次 — Tech 變化太快, 3-year plan 會過時。建議每年生日或新年做一次「skill bet 復盤」砍掉不對的 bet、加新興 bet

💡 **vibe coder 的 Day-1 Quick Win:** 今天花 30 分鐘做兩件事: (a) 寫一份「2029 年的我長什麼樣」的 1-page note, 列 5 個你想擁有的能力 / portfolio item / 影響力 KPI; (b) 反推回今天, 列

3 個本週可以做的小事讓你接近那個未來。寫下來,半年後再看一次。這個 exercise 讓 W10 不只是 reading week, 而是 strategic week。

上一週: [W9 Agents Post-Deployment](#) | 總目錄: [00_index.md](#)

II

	Know	257
95	Observability Basics You Should Know	259
95.1	核心論點 (150-200 字繁中)	259
95.2	關鍵概念	259
95.3	對 CS146S 的意義	259
95.4	對 Vibe Coder 的 Takeaway	259
95.5	原文連結	259
96	Kubernetes Troubleshooting with AI	261
97	Kubernetes Troubleshooting with AI	263
97.1	核心論點 (150-200 字繁中)	263
97.2	關鍵概念	263
97.3	對 CS146S 的意義	263
97.4	對 Vibe Coder 的 Takeaway	263
97.5	原文連結	263
98	Your New Autonomous Teammate	265
99	Your New Autonomous Teammate	267
99.1	核心論點 (150-200 字繁中)	267
99.2	關鍵概念	267
99.3	對 CS146S 的意義	267
99.4	對 Vibe Coder 的 Takeaway	267
99.5	原文連結	267
100	Role of Multi Agent Systems in Making Software Engineers AI-native	269
101	Role of Multi Agent Systems in Making Software Engineers AI-native	271
101.1	核心論點 (150-200 字繁中)	271
101.2	關鍵概念	271
101.3	對 CS146S 的意義	271
101.4	對 Vibe Coder 的 Takeaway	271
101.5	原文連結	272
102	Top 5 Benefits of Agentic AI in On-call Engineering	273
103	Top 5 Benefits of Agentic AI in On-call Engineering	275
103.1	核心論點 (150-200 字繁中)	275
103.2	關鍵概念	275
103.3	對 CS146S 的意義	275
103.4	對 Vibe Coder 的 Takeaway	275
103.5	原文連結	275
104	9 Weekly Assignments – 自學者可 行性評估	277
104.1	推薦做題順序 (給自學者)	277
104.2	Final Project 建議	278

13. Reading 摘要總目錄

CS146S 的 9 週共 45 篇 reading (W8、W10 原 syllabus 沒列 reading)。每篇 200-400 字繁中摘要，含核心論點 / 關鍵概念 / 對 CS146S 的意義 / 對 vibe coder 的 takeaway / 原文連結。

統計

13.1

狀態	篇數	說明
✅ Complete	43	抓到原文後做的繁中摘要
🟡 Best-effort	1	原文取得受限 (access code), 用主題脈絡做高層摘要
🔴 Failed	1	原文無法抓取 (X/Twitter 需登入), 只放連結 + disclaimer

來源類型分布

13.2

類型	篇數	範例
Blog post	33	Anthropic, Stytl, Cloudflare, Splunk, OWASP, ...
GitHub README	6	MCP servers, MCP SDK, Awesome Claude Agents, Super Claude, ACE-FCA, O3 prompt
YouTube	3	Karpathy Deep Dive, Anthropic Prompt Engineering, AI Code Review Lessons
PDF	2	OpenAI Codex use cases, Anthropic uses Claude Code
arXiv	1	AI-Assisted Assessment of Coding Practices
Twitter/X	1	How FAANG Vibe Codes (failed)

13.3 Week 1: Introduction to Coding LLMs and AI Development

#	標題	來源類型	狀態
1	Deep Dive into LLMs	YouTube	🟡 高層摘要
2	Prompt Engineering Overview	Blog (Google Cloud)	✅
3	Prompt Engineering Guide	Blog (promptingguide.ai)	✅

#	標題	來源類型	狀態
4	AI Prompt Engineering: A Deep Dive	YouTube (Anthropic)	🟡 高層摘要
5	How OpenAI Uses Codex	PDF	✅

13.4 Week 2: The Anatomy of Coding Agents (MCP)

#	標題	來源類型	狀態
6	MCP Introduction	Blog (Stytch)	✅
7	Sample MCP Server Implementations	GitHub	✅
8	MCP Server Authentication	Blog (Cloudflare)	✅
9	MCP Server SDK (TypeScript)	GitHub	✅
10	MCP Registry	Blog	✅
11	MCP Food-for-Thought	Blog	✅

13.5 Week 3: The AI IDE

#	標題	來源類型	狀態
12	Specs Are the New Source Code	Blog (Ravi Mehta)	✅
13	How Long Contexts Fail	Blog	✅
14	Devin: Coding Agents 101	Blog (Cognition)	✅
15	Getting AI to Work In Complex Codebases (ACE-FCA)	GitHub	✅
16	How FAANG Vibe Codes	Twitter/X	🔴 無法抓取
17	Writing Effective Tools for Agents	Blog (Anthropic)	✅

13.6 Week 4: Coding Agent Patterns (Claude Code)

#	標題	來源類型	狀態
18	How Anthropic Uses Claude Code	PDF	✅

#	標題	來源類型	狀態
19	Claude Code Best Practices	Blog (Anthropic)	✓
20	Awesome Claude Agents	GitHub	✓
21	Super Claude Framework	GitHub	✓
22	Good Context, Good Code	Blog (StockApp)	🟡 高層摘要
23	Peeking Under the Hood of Claude Code	Medium	✓

13.7 Week 5: The Modern Terminal (Warp)

#	標題	來源類型	狀態
24	Warp University	Blog	✓
25	Warp vs Claude Code	Blog	✓
26	How Warp Uses Warp to Build Warp	Notion	✓

13.8 Week 6: AI Testing and Security

#	標題	來源類型	狀態
27	SAST vs DAST	Blog (Splunk)	✓
28	Copilot RCE via Prompt Injection	Blog	✓
29	Finding Vulnerabilities Using Claude Code and OpenAI Codex	Blog (Semgrep)	✓
30	Agentic AI Threats: Identity Spoofing	Blog (Palo Alto Unit 42)	✓
31	OWASP Top Ten	Blog (OWASP)	✓
32	Context Rot: Degradation in AI Context Windows	Blog (Chroma)	✓
33	Vulnerability Prompt Analysis with O3 (CVE-2025-37899)	GitHub	✓

13.9 Week 7: Modern Software Support (AI Code Review)

#	標題	來源類型	狀態
34	Code Reviews: Just Do It	Blog (Coding Horror)	✓
35	How to Review Code Effectively (GitHub Staff Engineer)	Blog (GitHub)	✓
36	AI-Assisted Assessment of Coding Practices in Modern Code Review	arXiv	✓
37	AI Code Review Implementation Best Practices	Blog (Graphite)	✓
38	Code Review Essentials for Software Teams	Blog	✓
39	Lessons from millions of AI code reviews	YouTube (Graphite)	✓

13.10 Week 8: Automated UI and App Building

本週原 syllabus 沒列 reading list。延伸推薦資源見 [W8 講義](#)。

13.11 Week 9: Agents Post-Deployment (SRE / Observability)

#	標題	來源類型	狀態
40	Introduction to Site Reliability Engineering	Blog (Google SRE Book)	✓
41	Observability Basics You Should Know	Blog (last9.io)	✓
42	Kubernetes Troubleshooting with AI	Blog (Resolve)	✓
43	Your New Autonomous Teammate	Blog (Resolve)	✓
44	Role of Multi Agent Systems in Making Software Engineers AI-native	Blog (Resolve)	✓
45	Top 5 Benefits of Agentic AI in On-call Engineering	Blog (Resolve)	✓

13.12 Week 10: What's Next for AI Software Engineering

本週原 syllabus 沒列 reading list。延伸推薦資源見 [W10 講義](#)。

13.13 高優先閱讀清單（給時間有限的讀者）

如果只有時間讀 5-10 篇，建議這份精選：

1. [How OpenAI Uses Codex](#) – 最 actionable 的 production case study
2. [MCP Introduction](#) – vibe coder 必須建立的 MCP mental model
3. [MCP Food-for-Thought](#) – 寫 MCP server 的設計品味
4. [Specs Are the New Source Code](#) – 理解 PRD 在 agent 時代的角色
5. [Writing Effective Tools for Agents](#) – Anthropic 官方的 tool design 心法
6. [How Anthropic Uses Claude Code](#) – Anthropic 內部使用真實樣貌
7. [Claude Code Best Practices](#) – Anthropic 官方 best practice
8. [How Long Contexts Fail](#) – 4 種 context failure mode + fix
9. [Lessons from millions of AI code reviews](#) – Graphite Diamond 的 52% action rate 定義 AI reviewer 達成 human parity
10. [Introduction to Site Reliability Engineering](#) – Google SRE Book 經典開頭

13.14 延伸主題群

把跨週相關 reading 整在一起：

- **MCP 完整系列**（W2 全 6 篇）– 從 protocol 介紹 → server 實作 → auth → registry → 設計反思
- **Claude Code 系列**（W4 全 6 篇 + W2 #6 MCP servers）– Claude Code 的內部實作、用法、生態系
- **AI security 系列**（W6 全 7 篇）– 從 SAST/DAST 基礎 → prompt injection 攻擊 → 真實 CVE 案例 → 防禦
- **Agent observability 系列**（W9 全 6 篇）– 從 SRE 入門 → observability 三柱 → Resolve.ai 的 AI-native incident response
- **Code review 系列**（W7 全 6 篇）– 從人類 review 哲學 → AI reviewer 實證 → Graphite Diamond

14. Deep Dive into LLMs

15. Deep Dive into LLMs

一句話摘要: (基於既有知識的高層摘要, 未抓取逐字稿) Karpathy 用 3.5 小時把 LLM 從 raw text 變成可用 chatbot 的整條 pipeline 拆給非專家看, 重點是「LLM 是 token-level autocomplete + 後天 RLHF 微調出來的 helpful assistant」這個世界觀。

15.1 核心論點 (150-200 字繁中)

Karpathy 主張要真正用好 LLM, 必須先理解它怎麼被造出來的。整部影片把 ChatGPT 級別模型的生成過程拆成三段: (1) **pre-training** (預訓練)——抓網路 scale 的文本 (FineWeb 等 dataset) 做 next-token prediction, 這階段產出的是 base model, 本質上是「網路文件補全器」; (2) **post-training** (後訓練)——supervised fine-tuning (SFT) 用人工標註的 conversation data 把 base model 變成會「對話」的 assistant; (3) **RLHF** (reinforcement learning from human feedback, 人類回饋強化學習)——用 reward model 進一步把回答品質拉上來。理解這個 stack 後, hallucination (幻覺)、knowledge cutoff (知識截止)、tokenization 怪象、為什麼 LLM 不會數字母都變得可預測。Karpathy 也強調 LLM 在 capability spectrum 上分布極不均勻: 某些任務遠超人類、某些任務 (簡單算術、字元計數) 卻離譜地差。

關鍵概念

15.2

1. **Pre-training** (預訓練) — 對網路 scale 的文本資料做 next-token prediction, 產出 base model
2. **Post-training** (後訓練) — 用人工 conversation data 做 SFT, 把 base model 轉成 assistant
3. **RLHF** (reinforcement learning from human feedback, 人類回饋強化學習) — 用 reward model 進一步優化回答品質
4. **Hallucination** (幻覺) — Model 編造看似合理但不存在的內容, 是 statistical pattern matching 的副產物
5. **Tokenization** — 把文字切成 token 的步驟, 造成「數字母」「拼字」這類任務反常困難
6. **Capability spectrum** (能力光譜) — LLM 在不同任務上能力分布極不均勻, 不能假設整體強就每件事都強

15.3 對 CS146S 的意義

W1 用這部影片建立 LLM 的 mental model — 後續所有 prompt engineering 技巧都建立在「LLM = pre-trained autocomplete + RLHF aligned assistant」這個底層認知上。沒有這層基礎, 後面 prompt 技巧只會淪為 cargo cult。

15.4 對 Vibe Coder 的 Takeaway

知道 LLM 是 token-level next-token predictor 後, 能解釋很多 weirdness: 為什麼長 context 後段會掉資訊, 為什麼 prompt 順序影響輸出, 為什麼要 few-shot examples。寫 prompt 時把它想成「補全網路上會出現的下一段文字」比想成「跟人對話」更準確。

15.5 原文連結

[Deep Dive into LLMs](#)

16. Prompt Engineering Overview

17. Prompt Engineering Overview

一句話摘要: Google Cloud 的 prompt engineering 入門頁, 定義 prompt 為「給 AI 的輸入」, prompt engineering 為「設計與優化 prompt 引導 LLM 產出期望結果的藝術與科學」, 並列出 prompt 設計四要素與五種常見 prompt 類型。

17.1 核心論點 (150-200 字繁中)

這篇 Google Cloud 文章把 prompt engineering 定位為「為 AI 設計 roadmap」——透過精心設計 context、instructions、examples 來引導 LLM 產出期望輸出。文章主張有效 prompt 取決於四個要素: (1) **prompt format** (格式)——自然語言問句、直接指令或結構化輸入, 須配合 model 偏好; (2) **context and examples** (情境與範例)——提供任務背景與少量示範, 能顯著提升輸出品質; (3) **fine-tuning and adapting** (微調與適應)——根據 model 回饋持續優化 prompt; (4) **multi-turn conversations** (多輪對話)——設計能保持 context-aware 的連續對話。文章把 prompt 分成 zero-shot (直接問), one/few/multi-shot (給範例), chain-of-thought (要求逐步推理), zero-shot CoT (CoT + 直接問的混合) 四大類, 並用 creative writing、summarization、translation、dialogue、open-ended Q&A 等具體 use case 示範如何套用。

關鍵概念

17.2

1. **Prompt** — 給 AI model 的輸入 (問題、指令、範例都算); prompt 品質直接決定 output 品質
2. **Zero-shot prompting** (零樣本提示) — 直接給指令、不給範例, 靠 model 內建知識回答
3. **Few-shot prompting** (少樣本提示) — 在 prompt 內塞 1 ~ 多個 input-output pair 當示範
4. **Chain-of-Thought prompting** (思維鏈提示, CoT) — 引導 model 把複雜推理拆成中間步驟
5. **Zero-shot CoT** — 不給範例但加「let's think step by step」之類指示, 混合兩者優點
6. **Multi-turn conversations** — 設計能維持 context 的連續對話結構

17.3 對 CS146S 的意義

W1 用這篇建立 prompt engineering 的標準詞彙表 — zero-shot / few-shot / CoT 是後續所有討論的基本構件, 也讓沒接觸過的學生對「prompt 是可被工程化的物件」有初步認知。

17.4 對 Vibe Coder 的 Takeaway

寫 Claude / Gemini prompt 時心裡先過四個 checklist: format 對不對、有沒有給 context 與 example、需不需要 CoT 拆步驟、是否多輪。光是養成「給範例 + 拆步驟」的習慣, 產出品質就會明顯提升。

17.5 原文連結

[What is prompt engineering?](#)

18. Prompt Engineering Guide (Techniques)

19. Prompt Engineering Guide (Techniques)

一句話摘要: DAIR.AI 維護的 Prompt Engineering Guide 「Techniques」總覽頁,把當前 18 種主流 prompting 技巧整理成單一 catalog,從基本的 zero-shot 一路涵蓋到 ReAct、Reflexion、Graph Prompting 等 agent 級別技巧。

19.1 核心論點 (150-200 字繁中)

這頁不是線性教學,而是一份「prompting 技巧分類目錄」——把社群與學界發表的 18 種技巧並列,定位為「進階 prompting 技巧,能完成更複雜任務、提升 LLM 可靠性與效能」。整體分布大致可分四層: (1) 基礎層——zero-shot、few-shot; (2) 推理層——Chain-of-Thought、Self-Consistency、Tree of Thoughts、Generate Knowledge Prompting; (3) 工具與外部資源層——Retrieval Augmented Generation (RAG)、Automatic Reasoning and Tool-use (ART)、Program-Aided Language Models (PAL)、ReAct; (4) 自動化與 meta 層——Automatic Prompt Engineer (APE)、Active-Prompt、Directional Stimulus Prompting (DSP)、Reflexion、Meta Prompting、Prompt Chaining、Multimodal CoT、Graph Prompting。讀者用法是「想解某類問題 → 翻 catalog 找對應技巧 → 點進子頁看 paper 與範例」等同於 prompting 的 reference manual。

關鍵概念

19.2

1. **Chain-of-Thought (CoT, 思維鏈)** — 要求 model 逐步推理而非直接給答案
2. **Self-Consistency (自洽)** — 同 prompt 跑多次取多數票,降低推理 variance
3. **Tree of Thoughts (ToT, 思維樹)** — 把推理擴展成樹狀搜索,允許回溯
4. **ReAct** — Reasoning + Acting 交錯,讓 LLM 邊推理邊呼叫外部工具
5. **Reflexion** — Agent 對自己的 failure 做 self-critique,下一輪改進
6. **RAG (Retrieval-Augmented Generation, 檢索增強生成)** — 先檢索外部文件再生成,降低 hallucination
7. **PAL (Program-Aided Language Models)** — 把 reasoning 委託給 generated code,避開 LLM 算術弱點
8. **Meta Prompting** — 用 prompt 去產生或優化另一個 prompt

19.3 對 CS146S 的意義

W1 用這頁當「prompting 技巧 map」,幫學生建立後續課程查表用的 mental index。後面幾週深入講解的技巧都能對應回這個 catalog。

19.4 對 Vibe Coder 的 Takeaway

把這頁 bookmark 起來當 cheat sheet — 遇到 model 答不好時先掃一遍 18 種技巧、挑一個最像對症的試。實務上 zero-shot → few-shot → CoT → RAG 這條升級路徑能解 80% 的問題。

19.5 原文連結

[Prompt Engineering Guide — Techniques](#)

20. AI Prompt Engineering: A Deep Dive

21. AI Prompt Engineering: A Deep Dive

一句話摘要: (基於既有知識的高層摘要, 未抓取逐字稿) Anthropic 內部 prompt engineer 圓桌討論, 主張寫 prompt 的本質是「清楚思考並把任務說清楚給一個沒有上下文的聰明同事」, 並透過實戰故事拆解 system prompt、example、structured output、edge case 等實務細節。

21.1 核心論點 (150-200 字繁中)

Anthropic 這場圓桌(Alex Albert 主持, Amanda Askell、David Hershey、Zack Witten、Sander Schulhoff 參與)把 prompt engineering 重新框成「精確思考與技術寫作的混合作」——好 prompt 的關鍵不是奇技淫巧, 而是把任務要求講清楚到「一個聰明但完全不認識你公司、不知道你資料、沒看過你產品的 contractor」也能照做。實務上他們強調幾個重點: (1) 靠近 model 的直覺——多跟 model 對話、看它怎麼讀你的 prompt; (2) **read the model output as data**——把 model 失敗 case 當 debug 訊號而非 bug; (3) **system prompt vs user prompt 分工**——persistent context、規則放 system, task-specific 放 user; (4) **few-shot** 不只是給格式——examples 同時定義「你期望什麼樣的推理路徑」; (5) **enterprise prompting** 與 **research prompting** 不同——前者要處理 edge case 與 long tail, 後者追究極限能力; (6) **prompt** 要為未來的 model 設計——把「現在 model 蠢所以加 hack」與「描述任務本

關鍵概念

21.2

1. **System prompt** — 跨對話 persistent 的角色與規則設定
2. **Multi-shot / few-shot examples** — 用範例同時傳遞格式與推理風格
3. **Structured output** — 用 XML tag、JSON schema 等強制 model 產出可被下游解析的格式
4. **Honest failure mode** — 與其逼 model 假裝會, 不如教它在不確定時明說
5. **Edge case enumeration** — Enterprise 場景必須窮舉 long-tail 異常輸入
6. **Chain-of-Thought scratchpad** — 讓 model 在 `<thinking>` 區塊先推理再給答案
7. **Prompt as documentation** — 寫 prompt 等於寫「如何完成這個任務」的 SOP

21.3 對 CS146S 的意義

W1 用這部當「業界一線 prompt engineer 怎麼想 prompt」的視角, 補足 Karpathy 偏 model 內部的視角, 讓學生看到 prompt engineering 在工業實務中是什麼樣子。

21.4 對 Vibe Coder 的 Takeaway

寫 prompt 卡住時, 問自己「如果今天請一個聰明 contractor 做這事, 我會怎麼寫 brief?」答案通常就是好 prompt。把 system / user / examples / structured output 四個槽位填好再去調語氣。

21.5 原文連結

[AI Prompt Engineering: A Deep Dive](#)

22. How OpenAI Uses Codex

23. How OpenAI Uses Codex

一句話摘要: OpenAI 內部訪談 + 使用數據彙整, 分享 Security / Product Engineering / Frontend / API / Infrastructure / Performance 等團隊每天怎麼用 Codex 加速軟體開發, 並歸納出 7 個 use case 與 5 條 best practice。

23.1 核心論點 (150-200 字繁中)

這份 12 頁文件把 Codex 從「demo 級 AI coding assistant」拉到「OpenAI 內部跨團隊日常工具」的真實使用樣貌。整理出 7 個 use case: (1) **code understanding** (程式理解) ——快速摸熟陌生 repo、追資料流、on-call incident response; (2) **refactoring and migrations** (重構與遷移) ——跨多個檔案的一致性改動 (例: 把 callback 全換成 async/await); (3) **performance optimization** (效能優化) ——找 hot path、批次化 DB query; (4) **improving test coverage** (補測試) ——對低覆蓋模組產 unit/integration test; (5) **increasing development velocity** (提升開發速度) ——scaffold boilerplate、收尾 last-mile; (6) **staying in flow** (保持心流) ——在 meeting / on-call 碎片時間 fire-and-forget 任務; (7) **exploration and ideation** (探索與發想) ——找替代方案、辨識潛在 regression。Best practice 強調: 先 Ask Mode 出計畫再切 Code Mode、用 GitHub Issue 風格寫 prompt、靠 AGENTS.md 提供持久 context、Best-of-N 探索多解、把 task queue 當 backlog。

關鍵概念

23.2

1. **Ask Mode vs Code Mode** — 先 Ask 產 implementation plan, 再切 Code 落地, 避免 model 跳過設計
2. **AGENTS.md** — Repo 內 persistent context 檔, 放命名慣例、業務邏輯、quirks
3. **Best-of-N** — 同 task 同時產多個版本, 挑或合併最佳結果
4. **Task queue as backlog** — 把 drive-by fix、雜想丟進 queue, 不必當下完成 PR
5. **Well-scoped task** — Codex 最佳適用範圍: 人類約 1 小時 / 數百行程式碼
6. **Environment iteration** — Startup script、env var、internet access 設好能顯著降低 error rate
7. **GitHub Issue-style prompt** — 含檔案路徑、模組名、diff、doc snippet 的 prompt 效果最好

23.3 對 CS146S 的意義

W1 用這篇當「LLM 在真實工程組織怎麼被使用」的 case study — 不是理論技巧、不是 demo, 而是 OpenAI 自己怎麼把 LLM 嵌進日常 workflow, 給學生一個 production-grade 的對標。

23.4 對 Vibe Coder 的 Takeaway

把這 7 個 use case 當 vibe-coding playbook 直接抄: 寫新功能前先 Ask 產 plan、repo 加 AGENTS.md (或 CLAUDE.md)、把碎想丟 task queue、把 prompt 當 GitHub Issue 寫。這幾招套上去 Claude Code 用起來會立刻不一樣。

23.5 原文連結

[How OpenAI Uses Codex \(PDF\)](#)

24. MCP Introduction

25. MCP Introduction

一句話摘要: Model Context Protocol (MCP, 模型上下文協定) 是一個基於 JSON-RPC 2.0 的開放標準, 讓 LLM 應用透過統一介面接上外部工具與資料源, 取代過去每個 API 都要客製整合的做法。

25.1 核心論點 (150-200 字繁中)

Stytch 這篇 blog 把 MCP 定位為「AI 應用與外部系統之間的 universal adapter (通用轉接頭)」, 傳統上, 每個 LLM 應用要接 Slack、Gmail、internal database 都得寫一份客製整合, 代碼重複且難以維護。MCP 用 client-server 架構解決這問題: MCP client 嵌在 AI 應用裡(chatbot、IDE assistant), MCP server 用標準化的 JSON-RPC 訊息暴露能力; client 透過 handshake → discovery → invocation 五步驟流程呼叫 server 端的 tool 或讀取 resource。MCP 比 ChatGPT plugin 更開放 (非專屬)、比 LangChain 更標準 (formal protocol), 且近期加入 OAuth 2.0 + Dynamic Client Registration 解決多 user 環境的認證問題。

關鍵概念

25.2

1. **MCP client** (協定客戶端) – 嵌在 AI 應用內, 負責與 server 對話、把 tool/resource 暴露給 LLM。
2. **MCP server** (協定伺服器) – 對外暴露 tools、resources、prompts 三類 capability, 回應 client 的 JSON-RPC 請求。
3. **Tools** – 可被 LLM 呼叫執行動作的函式 (如「發送 email」「查詢資料庫」)。
4. **Resources** – 模型可讀取的 context 資料或文件 (檔案、DB 紀錄)。
5. **Prompts** – 預先定義的任務模板, 讓 server 提供建議的對話起手式。
6. **JSON-RPC 2.0** – 訊息格式標準, 固定的 request/response schema 確保跨 server 一致性。
7. **Transport** – 通訊管道, 分 local (stdio) 與 remote (HTTP/SSE stream) 兩類。
8. **Dynamic Client Registration (DCR, 動態客戶端註冊)** – OAuth 2.0 擴充, 讓新 client 自動註冊不需手動設定。

25.3 對 CS146S 的意義

這篇是 Week 2 MCP 主題的入門 baseline: 建立「為什麼需要協定、protocol 解決什麼整合問題」的心智模型, 後續的 server SDK、authentication、registry 各篇都建立在這個架構上。

25.4 對 Vibe Coder 的 Takeaway

要把自己的 app (例如資料分析 dashboard、阿摩錯題解析器) 接到 Claude, 與其手刻 plugin, 不如包成 MCP server – 一次寫完, Claude Desktop、Cursor、其他支援 MCP 的 client 都能直接用。Local stdio transport 開發成本最低, 原型階段不必處理 OAuth。

25.5 原文連結

[MCP Introduction](#)

26. Sample MCP Server Implementations

27. Sample MCP Server Implementations

一句話摘要: `modelcontextprotocol/servers` repo 是官方維護的 MCP server reference implementation 集合, 用多種程式語言示範如何把 LLM 安全、可控地接到外部工具與資料源。

27.1 核心論點 (150-200 字繁中)

這個 GitHub repo 是 MCP 生態系最重要的「教學樣本庫」。Repo 把 server 分兩大類: (1) **reference servers**, 由 MCP steering group 維護, 做為各 SDK 的標準範例; (2) **archived servers**, 過去由官方維護但已轉交社群的整合 (如 GitHub、PostgreSQL)。Reference 範例涵蓋日常開發者最會遇到的場景 – Filesystem (檔案讀寫)、Git (repo 操作)、Memory (knowledge graph 持久記憶)、Sequential Thinking (多步推理輔助)、Fetch (HTTP 抓取)、Time (時區換算)、Everything (功能展示用 demo)。Repo 強調這些是「reference implementation」而非 production 解決方案, 使用者部署前必須自行評估安全需求。除了官方 server 外, repo README 還大量索引社群框架 (FastMCP、EasyMCP、Spring AI MCP)、client implementation、與第三方 management platform。

關鍵概念

27.2

1. **Reference server** (參考實作) – 由 MCP 官方維護的標準範例, 目的是教學而非生產用。
2. **Filesystem server** – 提供「configurable access controls 的安全檔案操作」, 可限定可讀寫的路徑範圍。
3. **Memory server** – knowledge graph-based 的持久記憶系統, 讓 LLM 跨對話保留結構化記憶。
4. **Sequential Thinking server** – 引導模型做多步推理的輔助 tool。
5. **FastMCP / EasyMCP** – 社群開發的高階框架, 封裝 SDK 細節讓 server 開發更快。
6. **Servers-archived repo** – 過去官方整合 (GitHub、GitLab、Google Drive、Brave Search、PostgreSQL) 的歸檔位置, 現由社群接手。

27.3 對 CS146S 的意義

這個 repo 是動手做 MCP server 的最佳起點 – 上課作業要實作 server 時, 先 clone 一個 reference (如 Filesystem 或 Memory), 對照 SDK 文件改成自己的需求, 比從零寫快非常多。

27.4 對 Vibe Coder 的 Takeaway

想把家教題庫、研究資料庫、或自己寫的 R script 包成 Claude 可呼叫的 tool? 先讀 Memory server 的 source code (最短、最完整、TypeScript 寫的), 抄它的結構就能跑出第一個 prototype。Production 部署再考慮 access control 與 audit log。

27.5 原文連結

[Sample MCP Server Implementations](#)

28. MCP Server Authentication

29. MCP Server Authentication

一句話摘要: Cloudflare 這篇文件示範如何在 remote MCP server 加上 OAuth 2.0 認證,讓 server 從「人人可用」升級成「user 登入後才能呼叫 tool」的多租戶服務。

29.1 核心論點 (150-200 字繁中)

Remote MCP server (部署在 Cloudflare Workers 上的 HTTP server) 一旦對外開放, 就需要回答「誰能呼叫我的 tool?」這個問題。Cloudflare 提出兩條路: (1) **Cloudflare Access OAuth**, 把 GitHub / Google 等 identity provider 的驗證委託給 Cloudflare 平台, policy 在 request 層執行;(2) 第三方 **OAuth provider** (文中以 GitHub 為例),讓開發者自己整合任何 OAuth 2.0 服務。整個架構由 **OAuthProvider** class 統籌,配置四個關鍵 endpoint: API route (/mcp), authorization endpoint, token endpoint, client registration endpoint。 **defaultHandler** 負責把未認證的請求導向 OAuth flow, 認證通過才能進入 tool 呼叫。實作上需建 OAuth app、設環境變數、用 Wrangler CLI 存 secret、設 KV namespace 管 session; 測試則靠 MCP Inspector 走完 OAuth flow 驗證 token 有效性。

關鍵概念

29.2

1. **OAuth 2.0** — 業界標準的授權協定,讓 user 不交密碼也能授權第三方 app 存取資源。
2. **OAuthProvider** — Cloudflare Workers 提供的 class, 集中管理 authorization / token / client registration endpoint。
3. **defaultHandler** — 攔截未認證請求並導向 OAuth flow 的 gateway 角色。
4. **Cloudflare Access** — Cloudflare 的 identity aggregation 平台, 可串接 GitHub / Google 等 IdP (identity provider, 身分提供者)。
5. **Wrangler CLI** — Cloudflare Workers 的部署工具, 用來把 OAuth secret 安全存進 production 環境。
6. **KV namespace** — Cloudflare 的 key-value 儲存,用來持久化 OAuth session 與 token。
7. **MCP Inspector** — 官方測試工具, 提供 GUI 走完 OAuth flow 並驗證 authenticated tool 呼叫。

29.3 對 CS146S 的意義

這篇補上了「MCP server 從 hobby project 走向 production」的關鍵環節。Local studio server 不用考慮 auth, 但只要做 remote server, OAuth 就是繞不開的議題。CS146S 課程要設計 multi-user MCP 服務的話, 這套 pattern 是基準。

29.4 對 Vibe Coder 的 Takeaway

如果你的 vibe coding 作品想讓朋友/同事用, 又不想全世界都能呼叫 — 用 Cloudflare Workers + OAuthProvider 是門檻最低的 hosting 方案。免費 tier 夠跑大部分 prototype, 且 GitHub OAuth 30 分鐘就能設好。不要把 secret 寫進 source code, 永遠用 Wrangler secret put 注入。

29.5 原文連結

[MCP Server Authentication](#)

30. MCP Server SDK (TypeScript)

31. MCP Server SDK (TypeScript)

一句話摘要: MCP TypeScript SDK 用 `McpServer` class 把 server 開發收斂成「初始化 → 註冊 tool/resource/prompt → 選 transport → 啟動」四步驟, 搭配 `Zod` 等 schema 庫做 type-safe 的輸入驗證。

31.1 核心論點 (150-200 字繁中)

TypeScript SDK 是官方維護的 MCP server 開發 SDK 之一, README 的 Server 章節展示最小可運行範例。核心 API 是 `McpServer` class — 開發者初始化時帶 server 名稱與版本 (這些 metadata 之後會回給 client 做 discovery), 然後用 `registerTool()` 之類的 method 把功能掛上去。每個 tool 註冊需要三個元件: tool name、設定物件 (含 description 與 inputSchema)、async handler function。SDK 採用 Standard Schema 介面, 可自由換 `Zod v4`、`Valibot`、`ArkType` 等驗證庫。Transport 層支援 stdio (local 開發最常用) 與 streamable HTTP (remote 部署); 後者另有 `Express / Hono / Node HTTP middleware adapter` 方便整合既有 web 框架。文件附 weather server quickstart、完整 server guide、與 [examples/](#) 資料夾。值得注意: v2 SDK 為 pre-alpha 但官方推薦新專案直接用, v1 receive 6 個月維護期。

關鍵概念

31.2

1. **McpServer** — TypeScript SDK 的核心 class, server 實例的 entry point。
2. **registerTool()** — 註冊一個可被 LLM 呼叫的工具, 三參數為 name、config (含 description + inputSchema)、async handler。
3. **Standard Schema** — 跨 schema 庫的統一介面, 讓 SDK 不綁定特定驗證實作。
4. **Zod** — 最常見的 TypeScript schema 庫, 用 chainable API 描述 input 結構並自動產生 type。
5. **stdio transport** — 透過標準輸入輸出與 client 通訊, 最適合 local development 與 Claude Desktop 整合。
6. **Streamable HTTP transport** — Web-based 通訊, 適合 remote server 部署, 可搭配 `Express/Hono` middleware。
7. **Server-initiated request** — Server 主動向 client 發請求的能力 (如 sampling, 請 client 端 LLM 幫 server 做 inference)。

31.3 對 CS146S 的意義

這是寫 MCP server 作業的實作層 reference。比起讀 spec 文件, 直接抄 SDK README 的 weather server 範例改成作業需求, 能在最短時間跑出 working prototype。

31.4 對 Vibe Coder 的 Takeaway

TypeScript 是 vibe coding 寫 MCP server 性價比最高的選擇 — type system 抓 bug、Zod schema 自動生 JSON schema、stdio transport 直接接 Claude Desktop 不用部署。第一個 server 只要 50 行: `new McpServer({...}) → registerTool(...) → server.connect(new StdioServerTransport())`。Python SDK 也有同等 API 但 type checking 較弱。

31.5 原文連結

[MCP Server SDK \(TypeScript\)](#)

32. MCP Registry

33. MCP Registry

一句話摘要: MCP Registry 是 2025 年 9 月推出的官方 server discovery 中央目錄, 採開源 + sub-registry 模式, 避免單一 app store 壟斷又能維持跨生態系一致性。

33.1 核心論點 (150-200 字繁中)

Registry 解決的是 MCP 生態系成長後最迫切的問題: 「想用 server 的人找不到、寫 server 的人沒地方掛」。在沒有官方目錄前, server 散在 GitHub repo、社群清單、各 client 自家 directory, 缺一個 single source of truth。MCP Registry (registry.modelcontextprotocol.io) 以 open API + OpenAPI spec 形式提供, server 維護者照 quickstart 提交, client 維護者透過 documented API 抓資料。Registry 刻意不做 app store — 不做 pre-screening、不收費、不壟斷分發, 反而鼓勵 sub-registry 架構: MCP client 可以在 upstream registry 之上做 curated marketplace (精選清單)、企業可建 private sub-registry 維持 internal compliance。Moderation 採社群驅動: 使用者 flag 違規 → registry maintainer 審核後 denylist。這個 federation-friendly 設計保留多元解讀空間, 又靠共享 schema 維持一致性。

關鍵概念

33.2

1. **MCP Registry** — registry.modelcontextprotocol.io, 官方中央目錄, 2025-09-08 進入 preview。
2. **Sub-registry** (子目錄) — 基於 upstream registry 的衍生目錄, 可 public (curated marketplace) 或 private (企業內網)。
3. **OpenAPI spec** — Registry 對外開放的 API schema, 任何人可基於此做 compatible 工具。
4. **Single source of truth** (單一事實來源) — Registry 的設計目標: 所有 server metadata 以此為準。
5. **Community moderation** — 不做 pre-screening, 靠 user 提 issue → maintainer denylist 的 reactive 模式。
6. **Namespace** — Server 在 registry 裡的命名空間, 避免名稱衝突並標示維護者。
7. **Federation 模式** — 不把所有人都收進中央, 而是讓 sub-registry 分擔 curation 與 hosting。

33.3 對 CS146S 的意義

理解 MCP 的 distribution layer。Server 寫得再好, 沒人找得到也沒用 — Registry 是 ecosystem 從「protocol」走到「marketplace」的關鍵基礎建設。CS146S 討論 platform design 時, 這個 federation vs centralization 對比是經典案例。

33.4 對 Vibe Coder 的 Takeaway

寫完 MCP server 第一件事: 到 Registry 提交。免費、無審核延遲、提交完別人就能在 Claude Desktop 之類的 client 內搜到。但記得: Registry 是 metadata catalog, 不是 hosting — server 還是要自己跑 (local stdio 或 Cloudflare Workers 之類的 remote)。

33.5 原文連結

[MCP Registry](#)

34. MCP Food-for-Thought (APIs don't make good MCP tools)

35. MCP Food-for-Thought (APIs don't make good MCP tools)

一句話摘要：把現有 REST API 機械式包成 MCP tool 雖然技術上可行，但會撞上 tool proliferation、context window 浪費、與錯失 agent 原生能力三大問題；好的 MCP tool 必須為 agent 重新設計，而非單純轉接。

35.1 核心論點（150-200 字繁中）

Reilly Wood 提出對 MCP 開發的 contrarian take：別把 OpenAPI spec 自動轉成 MCP tool。理由四點：(1) **Tool proliferation** – agent 在 tool 數量遠未達到 128 之前就會 call 錯，每多一個 tool 就吃 context window 額度；多數 API 設計時根本沒考慮這限制。(2) **Context window 浪費** – API 回 wide record 含一堆 field，agent 不需要的也都吃 token；JSON 又是低效格式，作者實測 CSV 同樣資料只用 JSON 一半的 token。(3) **錯失 agent 原生能力** – 為 agent 量身打造的 tool 可以回 free-form text、做 layered tool chaining、混搭 structured 與 suggestive output；機械轉換 API 全失。(4) **冗餘** – Claude Code 之類的現代 agent 本來就會寫 code 直接 call API，多一層 MCP wrapper 反而是負擔。建議：把多支相關 API 合併成一個彈性 tool、加 field projection 與 result filtering、用 CSV/YAML 取代 JSON、在 response 裡指引下一步。

關鍵概念

35.2

1. **Tool proliferation**（工具擴增）– Tool 數量越多，agent 選錯的機率越高；實務上 128 之前就明顯惡化。
2. **Context window** – LLM 一次能處理的 token 數上限，每個 tool 定義與每筆 response 都吃額度。
3. **Field projection**（欄位投影）– 讓 caller 指定只回傳需要的欄位，借自 SQL `SELECT` 概念。
4. **Layered tool chaining**（分層工具串接）– 設計 tool 時考慮「呼叫完這個之後 agent 通常會 call 什麼」，在 response 引導 next step。
5. **Free-form response** – 不被嚴格 schema 綁住的回應，給 agent 用自然語言推理空間。
6. **Token efficiency** – 同樣資訊用 CSV / YAML 比 JSON 省 token，因為不需重複 key 名稱。
7. **API-vs-agent design mismatch** – API 為 code consumer 設計（精確、完整、結構化）；agent 需要的是 contextual reasoning（精簡、引導、可解釋）。

35.3 對 CS146S 的意義

這是 Week 2 的「反思題」– 前 5 篇教你怎麼寫 MCP server，這篇逼你想該不該這樣寫。對課程作業設計層面的批判性討論很有用：別只看 server 能不能跑，要看 agent 用起來順不順。

35.4 對 Vibe Coder 的 Takeaway

Vibe coding 最容易踩這個雷 – 拿到 OpenAPI spec 用 generator 一鍵變 MCP server，看似 productive 其實做出 agent 用不順手的東西。實務原則：(1) 先想清楚 agent 用這個 tool 會問什麼問題，再設計 tool 介面；(2) 永遠加 `fields` 參數讓 agent 自選欄位；(3) response 用 markdown table 或 CSV，別 dump 整坨 JSON；(4) 在 description 裡寫「這個 tool 適合在 X 情境用，後續通常呼叫 Y」，幫 agent 做 routing。

35.5 原文連結

[MCP Food-for-Thought \(APIs don't make good MCP tools\)](#)

36. Specs Are the New Source Code

37. Specs Are the New Source Code

一句話摘要：當 AI 把實作速度推到極致，真正的「source of truth」不再是 code，而是描述意圖的 specification（規格書），spec-writing 因此成為 PM 與工程師最關鍵的技能。

37.1 核心論點（150-200 字繁中）

Ravi Mehta 借 Sean Grove 的話比喻：傳統世界是 source code（人讀）→ binary（機器讀），AI 時代等於「把 source 撕碎，反而謹慎 version control 那個 binary」— 因為 code 只是 intent 的 lossy projection（有損投射），spec 才是完整需求。AI 把 implementation（實作）的時間從週縮到分鐘，但「理解 customer 需要什麼」仍是人類速度的工作，這個 gap 讓 spec 變成最有價值的 artifact（產物）。Spec 的功能是雙向的：對人，是團隊對齊與辯論的依據；對 AI agent，是執行的精確指令。Workflow 也跟著反轉 — 過去是 idea → spec → prototype → 痛苦修改；現在是先 rapid prototype 拿真實 customer feedback，再回頭把學到的東西寫成精確 spec。寫得糊的 spec 會產生糊的 code，這個關係比以前更直接、更殘酷。

關鍵概念

37.2

1. **Specification**（規格書）— 描述 user-facing 行為與意圖的 artifact，AI 時代的真正 source of truth；code 只是 spec 的有損投射。
2. **Lossy Projection**（有損投射）— Code 相對於 intent 是壓縮過的結果，丟失了「為什麼這樣做」的完整 context；spec 保留完整需求才能讓 AI 重現相同產出。
3. **Spec-Driven Alignment**（規格驅動對齊）— Spec 同時對齊人與 AI 系統，是 PM、工程師、designer 共同辯論的 single source。
4. **Prototype-Then-Spec**（先 prototype 再 spec）— Workflow 反轉：用便宜的 prototype 收集 customer feedback，再凝結成 crystal-clear spec，避免「想像中的需求」被直接打成 code。
5. **Non-Technical Contribution**（非工程師也能 contribute）— 清楚的 spec 讓 PM、designer 不寫 code 也能 directly 影響 codebase（透過 AI assistant 把 spec 翻成 code）。

37.3 對 CS146S 的意義

這篇定義了整個 Week 3 的概念基底：The AI IDE 的核心不是「補完更快」，而是「intent → executable」的鏈條被重組。CS146S 教 PRD（product requirement document）、context engineering、spec-driven development 三件事的共通底層 — spec 才是 AI 時代的 contract，PRD 是 spec 的一個變體。

37.4 對 Vibe Coder 的 Takeaway

醫學生做 vibe coding 時，最常見的失敗是「腦袋裡有畫面但沒寫下來」就直接叫 AI 寫 code，結果改 N 次都不對。先寫 1 頁 spec（user 是誰、要做什麼、成功長怎樣），再丟給 AI agent — 5 分鐘 spec 換 2 小時 debug 不值得跳過。把 spec 當 artifact 維護，下次想加功能直接改 spec 重生 code，比逐行改 code 划算。

37.5 原文連結

[Specs Are the New Source Code](#)

38. How Long Contexts Fail (and How to Fix Them)

39. How Long Contexts Fail (and How to Fix Them)

一句話摘要: 百萬 token context window 不等於更好的回答 — context 過載會引發四種特定 failure mode (poisoning / distraction / confusion / clash), agentic 系統尤其受傷。

39.1 核心論點 (150-200 字繁中)

Drew Breunig 直接打破「context 越大越好」這個假設。frontier model 雖然支援 1M+ token context, 但實際使用時 context 越塞越多反而讓 agent 行為崩壞。他歸納出四種 failure mode, 每一種都跟 agentic loop (多輪、長 context) 強相關:(1) **context poisoning** — hallucination (幻覺) 一旦寫進 context 就會被反覆引用, 越錯越深; (2) **context distraction** — context 超過某個 threshold (小模型 ~32k、大模型 ~100k) 後, 模型會偏向「重複歷史 action」而非用 train 出來的 reasoning 重新規劃; (3) **context confusion** — 跟當前任務無關的資訊 (例如過多 tool definition) 也會干擾決策, Berkeley Function-Calling Leaderboard 顯示 8B 模型給 19 個 tool 還能用、46 個就崩; (4) **context clash** — 新資訊跟舊 context 衝突時 reasoning 會壞掉, 多輪對話「拐錯一個彎就回不來」(Microsoft/Salesforce 研究顯示分輪餵相同資訊比一次性餵下降 39%)。修復方向: dynamic tool loading + context quarantine (隔離區)。

關鍵概念

39.2

1. **Context Poisoning** (脈絡中毒) — Hallucination 被寫入 context 後反覆 self-reference, agent 朝不可能的目標前進 (Gemini Pokémon 例)。
2. **Context Distraction** (脈絡分心) — Context 過長時模型過度依賴歷史, 不再用 reasoning, 傾向 repeat past action。
3. **Context Confusion** (脈絡混淆) — 不相關資訊 (多餘 tool、無關文件) 干擾選擇; tool 越多 function-calling accuracy 越差。
4. **Context Clash** (脈絡衝突) — 新舊資訊矛盾時 reasoning 崩潰; 多輪對話一旦走偏無法恢復。
5. **Context Quarantine** (脈絡隔離區) — 把易污染或一次性的 context 隔到 sub-agent / 獨立 window, 避免污染主 reasoning。

39.3 對 CS146S 的意義

這篇是 context engineering 的「failure mode catalog」。Week 3 的 PRD / spec / tool design 都是在處理同一件事: 主動管理 context, 而不是把所有東西丟進去讓 model 自己挑。理解四種 failure 才能設計對應 mitigation (subagent、tool 精簡、compaction)。

39.4 對 Vibe Coder 的 Takeaway

長 session 越改越壞通常不是 model 變笨, 是 context 中毒了。三個 quick fix: (1) Claude Code `/clear` 或開新 session; (2) 一個任務只給必要的 file, 不要整個 repo 全 attach; (3) 發現 agent 重複犯同一個錯時, 不要繼續解釋, 直接重啟並把正確結論寫進 spec / CLAUDE.md。對寫 statistics analysis script 也同理 — 一個 dataset 一個 session, 不要混著做。

39.5 原文連結

[How Long Contexts Fail \(and How to Fix Them\)](#)

40. Devin: Coding Agents 101

41. Devin: Coding Agents 101

一句話摘要: Coding agent 跟 autocomplete / copilot / chatbot 是不同物種 — 它能 end-to-end 從 description 走到 PR, 工程師的角色從「寫 code」變成「engineering manager 委派並驗收」。

41.1 核心論點 (150-200 字繁中)

Devin 團隊把開發者 AI tool 的演化分成四代: 10 年前的 autocomplete (method 級), 4 年前的 copilot (多行), 2 年前的 generative chatbot (檔案級), 今天的 autonomous agent (end-to-end task)。agent 跟 assistant 的關鍵差異不是「更會寫 code」, 而是三件事: (1) **autonomy** — 對 test、linter、CI/CD 自我 iterate; (2) **error recovery** — 自己 debug 自己的錯, 不需要人類每步介入; (3) **multi-task** — 工程師可同時委派多個 agent, 自己變 engineering manager。但作者明確降溫: 對大型任務 expect ~80% 時間節省, 不是完全自動化, 仍需多輪 feedback 與人類 verify before merge。Prompt strategy 的重點是 architecture-first — 講 how 不只是 what、提供 starting point (檔案路徑、文件)、defensive prompting 預期 agent 會困惑的地方、給 agent 強 feedback system (type、test、lint)。

關鍵概念

41.2

1. **Autonomous Coding Agent (自主編程 agent)** — 能從 description 跑到 PR、自我 iterate 對 test 與 CI、不需逐步指令的 AI 系統。
2. **Levels of Autonomy (自主層級)** — autocomplete → copilot → chatbot → agent 的能力光譜; autonomy 越高人類介入頻率越低、單次任務 surface 越大。
3. **Engineering Manager Mode (工程主管模式)** — 工程師同時 supervise 多個 agent, 自己不寫多數 code, 而是定義任務、review plan、merge PR。
4. **Architecture-First Prompting (架構優先提示)** — Prompt 重點在 how (指定設計方向、檔案位置、約束) 而非只給 what (功能描述)。
5. **Defensive Prompting (防禦性提示)** — 預先告訴 agent 哪些地方容易誤解、哪些 path 不要碰、哪些假設要 challenge。
6. **Feedback Loop Quality (回饋迴圈品質)** — Agent 表現上限取決於環境給它的 type / test / lint signal 是否準確即時。

41.3 對 CS146S 的意義

這是 Week 3 的 autonomy spectrum 教材。理解 agent vs assistant 的分界, 才能在 Week 4+ 的 PRD / spec 設計時知道「這個 spec 要寫到多細」— 給 assistant 的 prompt 跟給 agent 的 spec 顆粒度差很多。

41.4 對 Vibe Coder 的 Takeaway

醫學生用 Claude Code 已經是在用 agent, 不是 chatbot — 對應的工作習慣要跟著換: (1) 不要逐行盯 agent 寫 code, 要 review plan 跟 final diff; (2) build feedback loop (type check、test command) 讓 agent 自己 iterate, 不要當人肉 linter; (3) 接受 80% 自動化、20% 人工驗收這個 ratio, 期望 100% 會完美。

41.5 原文連結

[Devin: Coding Agents 101](#)

42. Getting AI to Work In Complex Codebases (ACE-FCA)

43. Getting AI to Work In Complex Codebases (ACE-FCA)

一句話摘要: AI 在 production codebase 表現差不是 model 不夠強, 是 context engineering 不夠好; 用 research → plan → implement 三段式 + frequent intentional compaction, 才能在 brownfield 專案 ship 真正的 feature。

43.1 核心論點 (150-200 字繁中)

humanlayer 團隊提出 ACE-FCA (Advanced Context Engineering for Coding Agents) 的核心命題: 「 context window 的內容是你影響 output 品質的唯一槓桿 」(agent 是 stateless function)。優先順序是 correctness → completeness → size → trajectory。框架核心是 **Frequent Intentional Compaction (FIC, 頻繁主動壓縮)** 三段式 workflow: (1) **Research phase** – 主 agent 不寫 code, 先了解 codebase、依賴關係、潛在 failure point, 產出結構化 markdown; (2) **Plan phase** – file-level 精度的 implementation step、verification 策略、可平行產生多版 plan 比較 trade-off; (3) **Implement phase** – 照 plan 執行、每階段間 compact、context 維持 40-60% 使用率、在 isolated git worktree 工作。Subagent 用來隔離 expensive 操作(grep、檔案搜尋), 避免污染主 reasoning。最關鍵的是 **high-leverage human review**: 人類審查重心放在 research 與 plan 階段, 不是 **code review** – 因為 「 plan 一行壞 = code 100 行壞 」。實際成果: 300k LOC 不熟的 Rust repo 幾小時內 fix bug、35k LOC feature 在 7 小時 pairing 完成。

關鍵概念

43.2

1. **Context Engineering (脈絡工程)** – 主動 curate 進入 agent context window 的內容, 把 context 當 first-class artifact 管理而非副作用。
2. **Frequent Intentional Compaction (FIC, 頻繁主動壓縮)** – 在每個 phase 邊界把過程資訊壓成結構化 summary, 丟掉 raw exploration 的雜訊。
3. **Research → Plan → Implement** – 三段式 workflow; 前兩段不寫 code, 只有最後一段才產 code。
4. **Subagent Context Isolation (子代理隔離)** – Grep、檔案搜尋這類會產生大量 token noise 的操作派 subagent, 主 agent 只接收 summary。
5. **High-Leverage Human Review (高槓桿人類審查)** – 人力放在 research / plan 而非 code, 因為錯誤在前段被攔截 cost 最低。
6. **Spec-First Thinking (規格優先思維)** – 跟 Sean Grove / Ravi Mehta 同源: spec 是 source code, 不是探索的副產品。

43.3 對 CS146S 的意義

這篇是 Week 3 最 actionable 的 reading – 把 「 context engineering 」 從口號落地成可執行 workflow。其他 reading 講為什麼 (Specs Are New Source, Long Contexts Fail), ACE-FCA 講怎麼做。是貫穿 PRD / spec / tool design 三主題的橋樑。

43.4 對 Vibe Coder 的 Takeaway

醫學生做 vibe coding 最常掉的坑是 「跳過 research / plan 直接讓 Claude Code 開寫」遇到 brownfield (已有 codebase) 就崩。三個落地動作: (1) 任何 non-trivial task 強迫先產 plan.md, 自己 review 完才 implement; (2) grep / 找檔案派 subagent 跑, 不要在主 session 做; (3) Claude Code session context 用到 60% 就 compact, 不要硬撐到爆。對統計分析也適用 – 先寫 analysis plan、review、再讓 agent 跑 R script。

43.5 原文連結

[Getting AI to Work In Complex Codebases \(ACE-FCA\)](#)

44. How FAANG Vibe Codes

45. How FAANG Vibe Codes

一句話摘要: 原文無法抓取(X/Twitter 對未登入存取回 HTTP 402); 以下為 best-effort 的 context 推測, 不是原文摘要, 請以原推文為準。

45.1 抓取狀態說明

WebFetch 對 x.com/rohanpaul_ai/status/1959414096589422619 回傳 HTTP 402 (Payment Required, X 對未登入 / 非付費 API 的標準擋牆)。MCP 沒有 X-authenticated 工具可用。這篇不放原文摘要, 僅依 (a) Rohan Paul 的公開內容主題(AI / ML engineering popularizer), (b) “How FAANG Vibe Codes” 這個標題、(c) Week 3 的 reading list 上下文, 做出有限度的推測。實際內容請使用者自行登入 X 查看。

45.2 Best-Effort 推測 (基於標題與 context, 非原文)

「Vibe coding」一詞由 Andrej Karpathy 在 2025 年初推紅, 指「跟著感覺寫、把想法丟給 AI 自動實作、不細看每行 code」的 workflow。「How FAANG Vibe Codes」這個標題套用在大公司情境, 常見論點走向有兩類: (1) FAANG 工程師正在用 Cursor / Claude Code / Devin 等 agent 工具大量產 code, 個人產出倍增、code review 文化承壓; (2) FAANG 對 vibe coding 的態度反而保守 — 因為大型 codebase 的 maintenance、security、code review 紀律不容許「跑得通就 ship」, 他們把 AI tool 限制在 well-scoped 範圍。這兩種敘事在 2025 下半年的 X discourse 都很常見, 無法從標題本身判斷此篇取哪個角度。

45.3 關鍵概念(基於 vibe coding 領域共識)

1. **Vibe Coding**(隨感覺編程) — Karpathy 提出, 指高度依賴 AI agent、人類不細審 code 的 workflow; 爭議在於 maintainability。
2. **Tool Adoption Asymmetry**(工具採用不對稱) — 個人開發者 / startup 與 FAANG 採用 AI coding tool 的速度與紀律差異很大。
3. **Code Review Bottleneck**(code review 瓶頸) — AI 加速 code 產出後, review capacity 變成新的瓶頸。

45.4 對 CS146S 的意義

即使無法讀到原文, 標題本身點出 Week 3 的一個重要 tension: context engineering / spec-driven 工作流在「個人 vibe coding」與「企業 production」兩種情境下的張力。CS146S 學生畢業多半進大公司, 理解這個 gap 比照搬 indie hacker workflow 重要。

45.5 對 Vibe Coder 的 Takeaway

不要只 follow 個人開發者的 vibe coding tutorial — 在大型 codebase 那套會崩。建議實際登入 X 看原推文確認作者主張, 再決定要不要採納。

45.6 原文連結

[How FAANG Vibe Codes](#) (需登入 X 才能瀏覽)

46. Writing Effective Tools for Agents

47. Writing Effective Tools for Agents

一句話摘要：給 AI agent 的 tool 不是傳統 API — agent 是 non-deterministic 的 caller，tool 設計要少而精、命名清楚、output 給 agent 看不是給開發者看，並用 evaluation-driven 方式 iterate。

47.1 核心論點（150-200 字繁中）

Anthropic engineering team 把 tool design 視為「deterministic 系統與 non-deterministic agent 之間的橋樑」— 同樣的輸入 agent 可能給不同回應，所以 tool 設計哲學跟傳統 API 不同。五個核心原則：(1) **Strategic Tool Selection** — 不要把每個 API endpoint 都 wrap 成 tool，agent context 有限，tool 越多反而 confuse（呼應 Drew Breunig 的 context confusion）；ex. 與其給 `list_users` + `list_events` + `create_event` 三個 tool，不如給一個 `schedule_event` 直接處理 availability check + scheduling。(2) **Meaningful Naming** — 用 namespace prefix (`asana_search` / `asana_projects_search`) 讓 agent 容易選對 tool。(3) **Contextual Response Design** — output 不要塞 UUID / MIME type 這類 agent 看不懂的 token，換成 semantic 名稱；提供 `response_format=concise|detailed` 讓 agent 自選詳簡。(4) **Token-Efficient Implementation** — pagination、filter、truncation 要有合理 default，截斷時告訴 agent 怎麼拿剩下的。(5) **Descriptive Specifications** — tool description「像在跟新進員工解釋」，把 implicit knowledge 寫明 (query format、術語、resource 關係)；用 `user_id` 而非 `user`。Error handling 不要丟 opaque error code，要給 actionable 改善建議。最後是 **evaluation-driven development**：用真實 case 測、跑 eval、看 transcript、用 Claude Code 自己迭代 tool implementation。

關鍵概念

47.2

1. **Tool（工具）** — 給 agent 用的 callable function，需考慮 non-deterministic caller 的特性，不能照搬 API design。
2. **Strategic Tool Selection（策略性 tool 選擇）** — 少而精、聚焦 high-impact workflow；tool 數量過多會引發 context confusion。
3. **Tool Consolidation（tool 合併）** — 把多個 low-level operation 合併成一個 high-level tool，減少 agent 的 orchestration 負擔。
4. **Semantic Output（語意化輸出）** — Tool output 用 agent 看得懂的 semantic 名稱取代 UUID / MIME type；token 用在 signal 不是 noise。
5. **Response Format Parameter** — 讓 agent 用 `concise` / `detailed` 自選 verbosity，避免不必要的 token 消耗。
6. **Actionable Error Message（可行動錯誤訊息）** — Error 要告訴 agent 怎麼修，不是丟 error code 然後期望 agent 自己 google。
7. **Evaluation-Driven Tool Development** — 用真實 task + agent transcript 衡量 tool 品質，再用 agent 本身迭代 tool 設計。

47.3 對 CS146S 的意義

Week 3 三主題（PRD / context engineering / tool design）裡，tool design 是最 concrete 的工程議題。這篇直接給工程 best practice，是 Week 4+ 學生自己實作 agent / MCP server 時的 reference。連結到 Drew Breunig 的 context confusion failure mode — 給 19 個 tool agent 還能用、46 個就崩，這篇就是教怎麼避免變成那 46 個。

47.4 對 Vibe Coder 的 Takeaway

寫自己的 MCP server / Claude skill 時三個立刻能用的原則：(1) 不要每個 function 都 export 成 tool，先想 user 的真正 workflow，合併成 1-3 個高層 tool；(2) tool description 寫到「我新進的 RA

看完就會用」的程度，不要假設 Claude 比 RA 聰明;(3) error 訊息給 fix instruction，不要丟 stack trace。對醫療 statistics tool 也適用 — 與其給 `load_data` + `clean_data` + `run_cox` 三個 tool，給一個 `cox_regression(data_path, time_col, event_col, ...)` 對 agent 友善太多。

47.5 原文連結

[Writing Effective Tools for Agents](#)

48. How Anthropic Uses Claude Code

49. How Anthropic Uses Claude Code

一句話摘要: Anthropic 內部 10 個團隊(含非工程部門如 legal、growth marketing、design) 都把 Claude Code 當作日常工作流的核心,技術門檻被大幅壓低,跨領域協作的瓶頸從「能不能寫 code」轉成「能不能描述清楚需求」。

49.1 核心論點 (150-200 字繁中)

這份 PDF 是 Anthropic 內部 power users 的訪談合輯,涵蓋 data infrastructure、product development、security engineering、inference、data science、API、growth marketing、product design、RL engineering、legal 共 10 個團隊。共通模式有三個:(1) 非技術人員大量使用 — finance、legal、design 透過 plain text workflow 把例行流程交給 Claude Code 自動執行,不必再請工程師寫 script; (2) **debugging** 透過螢幕截圖 + 自然語言描述就能完成 — Kubernetes pod IP 耗盡、Google Cloud UI 導覽都能靠 Claude Code 一步步診斷; (3) 跨技能 gap 被填平 — 工程師可以 vibe-code 出設計稿、設計師可以改 production code、legal 團隊自動生成法規 mapping。文件強調 Anthropic 把 Claude Code 視為「team multiplier」而非「coding assistant」。

關鍵概念

49.2

1. **Plain text workflow** — 把流程用自然語言寫成 .txt 檔,丟給 Claude Code 執行,等同無 code 的 automation。
2. **Screenshot-driven debugging** — 把 dashboard / error UI 截圖丟給 Claude Code,由它逐步引導排查。
3. **Skill-gap bridging** — 非工程師 (finance、legal、design) 能完成過去需委外給工程師的 task。
4. **Agentic autonomy** — Claude Code 不只回答問題,而是自主探索、執行命令、迭代修正。
5. **Cross-functional adoption** — 採用率不限於 R&D, growth marketing、legal 等部門也是 power users。

49.3 對 CS146S 的意義

這篇是 Week 4「Coding Agent Patterns」的 first-hand evidence: Anthropic 自己怎麼吃自己的 dog food。對課程而言,它具體展示了 coding agent 不只是 productivity tool,而是組織內部 capability 重新分配的槓桿 — 工程資源從「寫 boilerplate」釋放到「設計 system」,非工程資源從「等工程師排程」變成「自己用 Claude Code 跑」。

49.4 對 Vibe Coder 的 Takeaway

非資工背景的人 (如使用者本人) 可以直接從這份 PDF 抓到三個 actionable pattern: (1) 把重複性流程寫成 plain text spec 而非學 Python; (2) 遇到 infra / debugging 卡關直接截圖丟 Claude Code; (3) 別預設「這超出我的技術範圍」— 試了再說。對醫學生 / RA 而言,把研究 pipeline、資料清理、文獻整理當成 plain text workflow 是最低門檻的切入點。

49.5 原文連結

[How Anthropic Uses Claude Code](#)

50. Claude Code Best Practices

51. Claude Code Best Practices

一句話摘要: Claude Code 的所有 best practice 都圍繞同一個核心約束 — context window 會被填滿、performance 會隨 context 增加而下降, 所以使用者的工作就是主動管理 context、提供可驗證的 success criteria, 並善用 subagent / `/clear` / Plan Mode 來保持上下文乾淨。

51.1 核心論點 (150-200 字繁中)

Anthropic 官方總結出的 Claude Code 高效使用模式分四大類。第一類是「給 Claude 驗證自己的方法」— 提供 test、screenshot、expected output, 讓 agent 自己檢查而非靠人類做唯一 feedback loop。第二類是「Explore → Plan → Implement → Commit」四階段工作流 — 用 Plan Mode 隔離研究與執行, 避免直接 code 解錯問題。第三類是 environment 配置 — 寫精簡的 CLAUDE.md (每行都問「刪掉會不會出錯」)、用 `/init` 起手、設定 permission allowlist 或 auto mode、連 MCP server、用 hook 強制必跑步驟、用 skill 與 subagent 補充 domain knowledge。第四類是 session 管理 — `/clear` 切 task、`/rewind` 回退、`claude --continue` 接續、用 subagent 探索以保持主對話乾淨。最後也教 non-interactive mode (`claude -p`) 做 CI 整合與 fan-out 批次處理。

關鍵概念

51.2

1. **CLAUDE.md** — 每次對話自動載入的 persistent context; 要短、可審、可 prune; 過長會讓重要規則被淹沒。
2. **Skills** — 放在 `.claude/skills/` 的 SKILL.md, Claude 按需載入 domain knowledge / workflow, 不污染預設 context。
3. **Subagents** — 在獨立 context window 跑的特化 agent, 研究類任務的首選工具, 避免主對話被 file read 灌爆。
4. **Hooks** — 寫在 `.claude/settings.json` 的 deterministic script, 「必須每次發生」的動作 (如 lint、block migrations 寫入) 走 hook 而非 CLAUDE.md。
5. **Plan Mode** — 純讀檔規劃、不做修改的 mode, 作為 Explore-then-Code 的安全閘。
6. **Auto mode** — 用 classifier 自動審核 command, 僅擋 risky 操作以減少 permission 中斷。
7. **Verification loop** — test、screenshot、Bash check 作為 agent 自我驗證的 ground truth。

51.3 對 CS146S 的意義

這篇是 Week 4 的「正規教科書」。它把分散的 feature (CLAUDE.md / skills / hooks / subagents / MCP) 整合成一套使用紀律, 並提出 context engineering 是使用者最重要的技能。對課程設計而言, 這篇定義了「會用 Claude Code」的行為基準 — 不是會打字, 而是會管 context、會寫 verification、會分工到 subagent。

51.4 對 Vibe Coder 的 Takeaway

非資工背景使用者最直接的兩個操作改變: (1) 永遠提供 verification (醫學研究就是 unit test: 「這段 SQL 跑出來該有 X 筆」這個 figure 該有 Y 個 group); (2) 任務切換就 `/clear`, 不要省那幾秒。第三個是 CLAUDE.md 要當 code 維護 — 出錯時先檢查 CLAUDE.md 有沒有衝突或過長。對使用者既有的醫學 / 統計 workflow, 把可重複流程做成 skill 而非塞進 CLAUDE.md, 是長期可規模化的關鍵。

51.5 原文連結

[Claude Code Best Practices](#)

52. *Awesome Claude Agents* (curated list)

53. Awesome Claude Agents (curated list)

一句話摘要：把 24 個 specialized subagent 編成「虛擬開發團隊」－由 orchestrator 分派工作、framework specialist 處理特定技術棧、universal expert 跨棧支援、core team 做 QA 與 documentation－比起單一 generalist Claude，分工 + 自動配置能輸出更高品質的 code。

53.1 核心論點（150-200 字繁中）

這個 repo 不是 framework，而是 curated list－把 24 個預製好的 Claude subagent 分成四個層級：(1) **Orchestrators** (3 個，含 Tech Lead Orchestrator、Project Analyst、Team Configurator) 負責分析專案、路由任務；(2) **Framework Specialists** (13 個) 是特定技術棧的深度專家，含 Laravel / Django / Rails / React / Vue 的 backend / API / ORM 三組；(3) **Universal Experts** (4 個) 是跨 framework 的 polyglot；(4) **Core Team** (4 個) 做 code review、performance 優化、documentation、code archaeology。核心命題是「specialized expertise + real collaboration + tailored solutions 勝過 solo AI」－把 Claude Code 的 subagent 機制當組織學的 division of labor 來操作。內建 auto-configurator 會偵測專案技術棧並自動派任合適 specialist。

關鍵概念

53.2

1. **Orchestrator pattern**－senior 級 agent 不寫 code，只分析任務、路由給 specialist，模擬 tech lead 角色。
2. **Framework Specialist**－對單一技術棧(如 Laravel) 做深度 prompt engineering，比 generalist 更熟慣例。
3. **Universal Expert**－跨棧通才，處理 framework specialist 涵蓋不到的 cross-cutting 問題。
4. **Core Team (QA layer)**－code reviewer、performance optimizer、documentation specialist、code archaeologist，作為品質防線。
5. **Auto-configuration**－掃描 repo 偵測 stack (package.json、composer.json 等) → 自動激活對應 specialist。

53.3 對 CS146S 的意義

對 Week 4 課程而言，這 repo 是「subagent ecosystem」具體化的 case study。它把 best practice 文件中抽象的 subagent 概念具體化成「24 個角色 + 路由規則」，是課堂討論「agent specialization vs generalization」trade-off 的現成素材。也展示了社群如何把 Claude Code 的 primitive 包裝成 reusable agent library。

53.4 對 Vibe Coder 的 Takeaway

非工程背景使用者的啟發：與其想自己寫 24 個 agent，不如把這個 list 當 inspiration，挑出與自己 workflow 對應的 2-3 個直接複製到 `.claude/agents/`。對醫學研究情境，可以類比成「自己編一支研究團隊」－biostatistician agent、manuscript reviewer agent、literature search agent，各管一段，主對話只負責協調。但要注意：agent 多 ≠ 效果好，沒實際使用會變成維護成本。

53.5 原文連結

[Awesome Claude Agents](#)

54. Super Claude Framework

55. Super Claude Framework

一句話摘要: SuperClaude 是 Claude Code 的「meta-programming configuration framework」— 透過 30 個 slash command、20 個 specialized agent、7 個 behavioral mode、8 個 MCP server 整合, 把通用 Claude Code 升級成有結構化開發生命週期支援的平台。

55.1 核心論點 (150-200 字繁中)

SuperClaude 解決的問題是「Claude Code 預設 generic、缺 domain-specific workflow」。解方是 behavioral instruction injection + component orchestration: 在 Claude Code 上層疊加四類元件。**30 個 slash command** 涵蓋 ideation 到 deployment 全週期 — `/brainstorm`、`/design`、`/implement`、`/test`、`/document`、`/git`、`/spawn` 等。**20 個 specialized agent** 是 domain expert persona (PM Agent、Deep Research Agent、Security Engineer、Frontend Architect), 按 context 自動激活。**7 個 behavioral mode** 是 adaptive operating context — Brainstorming、Business Panel、Deep Research、Orchestration、Token-Efficiency、Task Management、Introspection。**8 個 MCP server** 整合外部能力 — Tavily (web search)、Context7 (doc)、Sequential-Thinking、Serena (memory)、Playwright、Magic (UI gen)、Morphllm-Fast-Apply、Chrome DevTools。設計哲學強調「framework footprint 要小, 把 context window 留給 project code」。安裝走 `pipx install superclaude + superclaude install`。

關鍵概念

55.2

1. **Behavioral Instruction Injection** — 在 Claude 行為層注入 prompt, 不改 model 也不改 Claude Code 本體。
2. **Component Orchestration** — 多個 agent + tool 協同的調度層。
3. **Behavioral Mode** — 不是 agent 也不是 command, 是「整段對話的操作脈絡」(如 Brainstorming Mode 會讓所有後續回應變成 Q&A 探索風格)。
4. **MCP Server Bundle** — 把多個 MCP 預打包進 framework, 使用者一次裝齊。
5. **Plugin Marketplace (v5.0 開發中)** — 未來支援 TypeScript plugin 可擴充。

55.3 對 CS146S 的意義

對 Week 4 而言, SuperClaude 是「community framework on top of Claude Code」最完整的範例。它示範了如何把 Anthropic 提供的 primitive (slash command / agent / hook / MCP) 系統化包裝成上層產品。可作為討論「Claude Code 的可擴充性 ceiling 在哪」、以及「framework 與 primitive 之間的取捨」的教案。

55.4 對 Vibe Coder 的 Takeaway

對非資工使用者要小心 SuperClaude 是強大但 opinionated 的 framework, 會在你的 Claude Code 上注入大量 behavioral instruction, 可能與已有 CLAUDE.md / skill 衝突。建議先用 `superclaude doctor` 驗證、選擇性 install component, 而非無腦 full install。對使用者已有的 medical / biostat skill ecosystem, SuperClaude 的 mode 概念 (Brainstorming / Deep Research) 是值得學習的 pattern — 把「對話脈絡」當成可切換的 first-class concept。

55.5 原文連結

[SuperClaude Framework](#)

56. Good Context, Good Code

57. Good Context, Good Code

一句話摘要：標題即論點 — AI coding agent 的輸出品質與「給的 context 品質」線性相關，所以「context engineering」是使用者端最高槓桿的技能，而非 prompt engineering 或 model selection。

57.1 核心論點（150-200 字繁中）

Note: 原文網址（StockApp Engineering blog）需要 access code 才能讀全文，本檔以標題、URL slug、與課程脈絡（Week 4: Coding Agent Patterns）做高層摘要，未直接抽取原文。讀者建議：以 access code 取得原文後再核對細節。

依標題與 Week 4 主題推論，這篇文章的核心命題與 Anthropic best practices 一致：coding agent（如 Claude Code）能不能寫出可用的 code，瓶頸不在 model 而在 context。「Good Context, Good Code」是 Garbage In, Garbage Out 的 agent 版本 — 給 agent 的 file reference、CLAUDE.md、example pattern、verification criteria 越完整、越具體，輸出品質越高。對應的反向命題是：當 Claude 寫出爛 code，第一個該檢查的不是 prompt 措辭，而是「我有沒有給它看對的 file、有沒有提供可執行的 test」。文章預期論及的 context 來源：existing codebase pattern、style guide、test fixture、past PR convention，以及 user 端用 @file reference、screenshot、CLI tool output 餵進去的 rich context。

關鍵概念

57.2

1. **Context engineering** — 比 prompt engineering 高一階的技能，重點在「餵什麼進 context」而非「怎麼措辭」。
2. **Code quality $\propto$ context quality** — agent 不會無中生有；輸出風格 / 正確性都是 context 中既有素材的重組。
3. **Verification as context** — test case、screenshot、expected output 也是 context 的一種，而且是最高槓桿的一種。
4. **Reference patterns** — 在 prompt 裡指出「跟這個 file 一樣的 pattern」比抽象描述更可靠。
5. **Garbage in, garbage out (agent 版)** — 不完整的 context 不只是減速，會主動誘發 hallucination。

57.3 對 CS146S 的意義

對 Week 4 而言，這篇（依標題判讀）是把「context engineering」抬升為論述主軸的代表。與 Anthropic 官方 best practices、Peeking Under the Hood 形成同一論述軸線：coding agent 的 alpha 在於 context discipline，不在 model size 或 prompt trick。

57.4 對 Vibe Coder 的 Takeaway

對非工程使用者最 actionable 的訊息是：當 Claude 寫的 code 不對，先別罵 model 也別重寫 prompt — 先問三個問題：(1) 我有給它 reference file 嗎？(2) 我有給它 verification (test / expected output) 嗎？(3) CLAUDE.md / skill 裡有沒有相關的 domain knowledge？三個都做了還錯，再考慮其他原因。這跟臨床醫學的 history-taking 同構：問診（context）做不好，鑑別診斷（output）一定不會準。

57.5 原文連結

[Good Context, Good Code](#)

58. Peeking Under the Hood of Claude Code

59. Peeking Under the Hood of Claude Code

一句話摘要: OutSight AI 透過 LiteLLM proxy 監聽 Claude Code 的 API call, 逆向工程出四個核心 pattern — context front-loading、`<system-reminder>` 標籤、embedded safety、conditional sub-agent — 結論是 Claude Code 的「魔法」不在 model, 而在系統化的 context scaffolding。

59.1 核心論點 (150-200 字繁中)

OutSight AI 用 LiteLLM proxy 攔截 Claude Code 真實送出的 API call, 揭開四個 pattern。(1) **Context Front-Loading**: session 一開始就先把對話濃縮成 < 50 字元 title、判斷是否新主題, 建立清晰的 task framing。(2) **`<system-reminder>` Tag 滲透**: 最重要的發現 — 這個特殊 tag 散布在 system prompt、user message、tool result 各層, 「tiny reminders, at the right time, change agent behavior」, 用以對抗 long-context drift。(3) **Embedded Safety**: 權限不是 hardcode 規則, 而是用專門的 prompt 偵測 command injection — 抽 command prefix 比對 policy spec、標記 backtick injection / pipe 外洩等可疑 pattern。(4) **Conditional Sub-Agent**: Task tool spawn 出的 subagent 預設不帶 TodoWrite reminder (保持 focus), 但若 agent 一段時間沒用 tool, 系統會 conditionally 注入 reminder — 在 specialization 與 capability awareness 之間做動態平衡。結論: Claude Code 的成功來自 systematic context scaffolding, 不是隱藏的 model superiority, 這個 pattern 可移植到其他 agentic system。

關鍵概念

59.2

1. **`<system-reminder>` tag** — 散布在多層 prompt 裡的微提醒, 用來抵抗 context drift; 最關鍵的逆向工程發現。
2. **Context front-loading** — session 開頭主動濃縮對話、判斷主題切換, 建立 framing。
3. **Prompt-based safety** — command injection 偵測用專門 prompt 而非 hardcoded rule, policy 是可演化的 spec。
4. **Conditional reminder injection** — subagent 預設無 reminder, 行為偏離才注入, 動態維持 focus 與 capability。
5. **Adaptive complexity** — sub-agent 依任務複雜度收到不同 instruction set。
6. **Tool result as context** — tool 回傳值不是 plain output, 會被包進 system reminder, 維持跨對話 focus。

59.3 對 CS146S 的意義

對 Week 4 而言, 這篇是唯一從「實作層」反推 Claude Code 設計哲學的文章, 補足官方 best practice 文件留白的 implementation detail。對課堂討論「為什麼 Claude Code 比同期 agent 框架穩定」提供 concrete answer: 是 context engineering pattern 而非 model 優勢。也呼應 Good Context, Good Code 的論點 — 連 Anthropic 自己都用 context scaffolding 撐起 agent, 更不用說 user 端。

59.4 對 Vibe Coder 的 Takeaway

兩個可移植的 pattern: (1) 自己寫長 workflow 時, 模仿 `<system-reminder>` 概念 — 在關鍵步驟前重述當前目標, 比期待 model 「自己記得」可靠; (2) 對 long session, 主動 framing (「我們現在在處理什麼」) 能顯著降低 drift。對使用者醫學研究 workflow 的啟發: 在多階段 pipeline (如 meta-analysis 9 stages) 裡, 每個 stage 開頭明寫「我們在 stage N, input 是 X, output 是 Y」, 效果接近 Claude Code 的 system reminder 機制。

59.5 原文連結

[Peeking Under the Hood of Claude Code](#)

60. Warp University (Getting Started with Warp and Oz)

61. Warp University (Getting Started with Warp and Oz)

一句話摘要: Warp 把自己定義為 Agentic Development Environment (代理式開發環境) — 由 modern terminal「Warp」加上 cloud agent orchestration platform「Oz」組成的雙產品架構,讓 local interactive coding 與 cloud-side automation 共用同一套 agent / context / rules。

61.1 核心論點 (150-200 字繁中)

Warp 不再把自己定位成「比較好看的 terminal」, 而是直接喊出 Agentic Development Environment 這個新類別。產品被切成兩塊: **Warp** 是使用者面前的 modern terminal, 內建 block-based UI、code editor、LSP、code review, 並能直接跑 third-party CLI agent (Claude Code、Codex、OpenCode); **Oz** 則是背後的 cloud agent orchestration platform, 負責驅動所有 AI 功能。Oz 同時支援 local agent (在 app 裡即時 pair coding) 與 cloud agent (在 Warp 機房或自家機房背景跑、靠 trigger / schedule / parallelism 處理 PR review、issue triage、dependency update 等不需即時關注的任務)。兩邊共用同一個 agent core、同一套 Warp Drive / Rules / MCP context, 所以可以「雲端開工、local 接手」無縫切換。整個 client 在 AGPL v3 開源,並標榜 SOC 2 + Zero Data Retention 的合規承諾。

關鍵概念

61.2

1. **Agentic Development Environment** (代理式開發環境) — 把 IDE + terminal + agent runtime 合成單一 first-class 產品類別, 不是把 chatbot 黏在 editor 旁邊。
2. **Oz (Orchestration Platform)** — Warp 自家的 cloud agent 調度層,多模型架構,使用者可在不同任務挑不同 LLM。
3. **Local Agent vs Cloud Agent** — 前者 interactive、人在 loop 裡審 diff; 後者 background、被 webhook / cron 觸發、可大規模並行。
4. **Block-based Terminal** (區塊化終端機) — 每條 command + output 是一個可分享、可搜尋、可篩選的 block, 取代傳統 line-buffer。
5. **Warp Drive** — 跨 local/cloud agent 共享的 notebook / workflow / env var / rules 倉庫,是 agent 的 persistent context layer。

61.3 對 CS146S 的意義

這是「Modern Terminal」單元最直白的 reading — 它把 terminal 的下一個演進方向講得很白: terminal 不再只是 input/output stream, 而是 agent runtime。Block 概念、Warpify、AGENTS.md / WARP.md 對應 Claude Code 的 CLAUDE.md, 都是課程在談「人 + agent 在 CLI 共存」的具體 design pattern 範本。

61.4 對 Vibe Coder 的 Takeaway

對非資工背景、用 Claude Code 寫程式的使用者,Warp 提供兩條路: 要嘛把 Claude Code 當 first-class CLI agent 跑在 Warp 裡(拿到 rich input、agent notification、code review panel), 要嘛直接用 Warp 內建的 Agent Mode。Warp Drive 的 Rules / Prompts / env var 概念跟 CLAUDE.md import 結構非常像,學一邊就能類推。

61.5 原文連結

[Warp University \(Getting Started with Warp and Oz\)](#)

62. Migrate to Warp from Claude Code (Warp vs Claude Code)

63. Migrate to Warp from Claude Code (Warp vs Claude Code)

一句話摘要: Warp 不把 Claude Code 當對手,而是定位成「跑在 Warp terminal 裡的 first-class CLI agent」— 使用者可以選擇繼續用 Claude Code 享受 Warp 的 IDE-level wrapper, 或乾脆切到 Warp 內建 Agent Mode。

63.1 核心論點 (150-200 字繁中)

這篇 doc 開宗明義說: Claude Code 不是 terminal emulator, 而是「跑在任何 terminal 裡的 CLI agent」, Warp 則是 agentic IDE, 所以兩者不是競品而是 stack 的不同層。Warp 給使用者兩條 migration path: (1) 在 Warp 裡跑 Claude Code — 自動偵測 `claude` 指令後解鎖 rich input editor (Ctrl+G 多行 prompt + @ mention + 語音), agent notification、inline code review、vertical tabs with agent metadata、remote control 等 IDE 級功能; (2) 切到 Warp Agent Mode — `⌘+Enter` 進入後直接自然語言下指令; 最關鍵的 migration 提示是「把 `CLAUDE.md` 改名為 `AGENTS.md` 或 `WARP.md` 放專案 root, Warp 會自動讀取, 不必重寫」Codebase Context、Rules、Warp Drive、Agent Mode Context、MCP 五個 context source 共同組成 Warp agent 的「該讀什麼檔」決策。

關鍵概念

63.2

1. **Third-Party CLI Agent Integration** — Warp 對 Claude Code / Codex / OpenCode 提供 first-class wrapper, 不是放任它在 dumb terminal 裡跑。
2. **Rich Input Editor (Ctrl+G)** — 把 CLI agent 的 single-line prompt 升級成多行 + @ mention + voice + slash command 的 IDE-style 輸入。
3. **AGENTS.md / WARP.md** — Warp 版的 `CLAUDE.md`, repo root 會被自動拾取; rename 即可遷移。
4. **Agent Mode (⌘+Enter)** — Warp 內建 agent 模式, 用 `⌘+I` 在 terminal mode 與 agent mode 間切換。
5. **Bracketed Paste** — Warp 預設啟用, 多行 paste 進 Claude Code 不會誤觸發送。
6. **Codebase Context Indexing** — 開資料夾後 Warp 自動 index Git-tracked 檔案, agent 可直接 search, 不用人工貼 snippet。

63.3 對 CS146S 的意義

直接示範「terminal 是 agent 的 host」這個課程主軸 — 同一個 Claude Code agent 在不同 terminal 跑會有完全不同的 UX。Warp 把 IDE 級 affordance (notification、code review panel、tab metadata) 下沉到 terminal 層, 正是 modern terminal 的設計核心。`AGENTS.md` 的命名標準化也是社群在收斂 multi-agent ecosystem 的具體案例。

63.4 對 Vibe Coder 的 Takeaway

如果已經用 Claude Code 順手且有寫好的 `CLAUDE.md`, 遷移到 Warp 幾乎零成本: rename 成 `AGENTS.md` 即可。立刻能拿到的好處是 desktop notification (agent 等你輸入時彈通知)、Ctrl+G 多行 prompt 編輯、vertical tab 同時管多個 Claude Code session。不必選邊站 — Warp 把 Claude Code 包進來而非取代。

63.5 原文連結

[Migrate to Warp from Claude Code](#)

64. How Warp Uses Warp to Build Warp

65. How Warp Uses Warp to Build Warp

一句話摘要: Warp 公司內部的「Coding Mandate」— 每個 coding task 都必須先用 Warp prompt 起手, 連續卡關十分鐘才能 fallback 到別的工具, 這份 dogfooding 紀律才是 Warp 產品快速演進的真正引擎。

65.1 核心論點 (150-200 字繁中)

這是 Warp CEO 公開的內部規範。核心是一條 mandate: 每個 coding task 都從在 Warp 裡 prompt 開始; 若十分鐘內無進展, 必須先到 `#feedback`-channel 回報 prompt + conversation id (給內部建 eval), 然後才允許試 Cursor / Claude / 手寫。Mandate 的「why」有三: (1) prompt-driven coding 對許多任務真的更快, 且能 multi-thread 開發、加速摸熟陌生 codebase; (2) dogfooding — 既然要客戶這樣工作、要拿這個押公司未來, 自己就得相信到願意用; (3) 用競品建立對市場上限的直覺。配套指南包含: 送審的 code 要像自己手寫的一樣負責 (不能用「AI 寫的」當 bug 藉口), 告訴 agent 怎麼做不是只說做什麼 (指定 data model / API / 檔案位置 / 測試), 拆小步、別 one-shot、先要 plan 再要 diff、重複工作做成 Warp Drive prompt / rule、context 用 MCP 拉 (Sentry、Linear、Notion、Slack)。

關鍵概念

65.2

1. **Coding Mandate** — 強制 prompt-first 的內部紀律, 把 dogfooding 從口號變成 enforced workflow。
2. **Ten-Minute Escape Hatch** — 卡十分鐘必須先回報 feedback + 提供 conversation id, 再被允許 fallback。
3. **Eval-Driven Feedback Loop** — 內部第一直覺是把每個 bug report 轉成 eval, 不是直接 patch。
4. 「Tell the agent how, not just what」— 反 goal-seeking prompt、要 explicit engineering spec (data model / API / test)。
5. **Small-step over One-shot** — 大改用 incremental commit + frequent test 取代一發命中。
6. **Persistent Context (Rules / Prompts / WD objects / MCP)** — Warp Drive 是讓重複指令不必每次重打的關鍵層; MCP server 連 Sentry / Linear / Notion 餵 context。
7. **Multi-thread Local Setup** — 同時開多份 warp-internal / warp-server clone 平行跑多個 agent task。

65.3 對 CS146S 的意義

從「終端機 + AI = 怎麼真的拿來工作」這個角度看, 這份 mandate 比任何 marketing page 都誠實 — 它告訴你 prompt-driven coding 不是魔法, 而是需要工程紀律: 拆小步、明確規格、code review 不放水、把 context 寫成 reusable rule、把 bug 變 eval。對課程談「modern terminal 改變開發流程」是非常具體的 case study。

65.4 對 Vibe Coder 的 Takeaway

兩個立刻能套用的習慣: (1) 不要 one-shot 大功能 — 拆成小 commit、邊跑邊讓 agent 寫 test, 避免被「看似能跑但無法維護」的 code 淹沒; (2) 重複指令立刻變 rule / prompt — 看到自己第二次打同樣的「請用 X linter、import 順序這樣排」就該存進 Warp Drive 或 `CLAUDE.md`。「卡十分鐘要回報」這條對個人開發者太嚴格, 但精神是「不要靜默掙扎, 馬上換策略或記下來下次改」。

65.5 原文連結

[How Warp Uses Warp to Build Warp](#)

66. SAST vs DAST

67. SAST vs DAST

一句話摘要: SAST (白箱、看 source code) 與 DAST (黑箱、跑 runtime 攻擊) 是兩種互補的 application security testing 方法, 現代 pipeline 應整合兩者並可加入 RASP 形成多層防禦。

67.1 核心論點 (150-200 字繁中)

文章把 application security testing 分成兩大典範: **SAST (Static Application Security Testing, 靜態應用程式安全測試)** 是 white-box 方法, 在不執行程式的情況下掃描 source code 或 binary, 於 SDLC (software development lifecycle) 早期就可揪出 SQL injection、XSS (cross-site scripting, 跨站腳本攻擊)、buffer overflow (緩衝區溢位) 等 coding flaw; **DAST (Dynamic Application Security Testing, 動態應用程式安全測試)** 則為 black-box 方法, 對 running application 模擬真實外部攻擊, 可抓出 server misconfiguration、authentication (驗證) 漏洞等只在 runtime 才會浮現的問題。SAST 早期便宜但 false positive 率高, DAST 較貼近真實攻擊但發現晚、修復成本高。最佳實務是把兩者整合進 CI/CD pipeline, 並可進一步加入 RASP (Runtime Application Self-Protection, 執行期自我保護) 形成全生命週期的多層防禦。

關鍵概念

67.2

1. **SAST (Static Application Security Testing, 靜態應用程式安全測試)** – White-box; 不執行程式、直接掃描 source code 或 binary 找漏洞, 常用 pattern matching、data flow analysis、control flow analysis。
2. **DAST (Dynamic Application Security Testing, 動態應用程式安全測試)** – Black-box; 對 running application 從外部送入 malicious request、觀察回應, 模擬真實攻擊者視角。
3. **RASP (Runtime Application Self-Protection, 執行期自我保護)** – 嵌進 application server 內部, runtime 監控行為並即時 block 惡意動作, 是 SAST/DAST 的補充。
4. **False positive vs false negative** – SAST 常有大量 false positive 需人工 review; DAST false positive 少, 但漏抓深層 code-level bug 的 false negative 風險高。
5. **CI/CD integration** – SAST 接 IDE 與 commit hook 即時 feedback; DAST 在 staging 環境跑, 兩者皆可自動化進 pipeline。

67.3 對 CS146S 的意義

這篇是 W6 「AI Testing and Security」的入門背景, 建立 vulnerability (系統病灶) 的兩大檢測典範詞彙。後續所有 AI-assisted security tooling (Semgrep + Claude Code、O3 找 CVE) 都仍落在 SAST / DAST 框架的延伸上, 理解原本的 trade-off (早期/晚期、深度/廣度、白箱/黑箱) 才能看出 LLM 帶來什麼新東西。

67.4 對 Vibe Coder 的 Takeaway

別以為 commit 前用 IDE 跑一次 lint 就算「測過 security」。Push 之前至少跑一次 SAST tool (Semgrep、CodeQL、SonarQube 都有 free tier), deploy 後再用 DAST (OWASP ZAP) 對 staging 打一輪。Vibe coding 最容易踩的是把 user input 直接拼進 SQL 或 shell command, 這類 injection bug 是 SAST 的強項, 幾秒鐘就可掃出來。

67.5 原文連結

[SAST vs DAST](#)

68. GitHub Copilot Remote Code Execution via Prompt Injection

69. GitHub Copilot Remote Code Execution via Prompt Injection

一句話摘要: 研究員 wunderwuzzi 發現 CVE-2025-53773 — 攻擊者用 prompt injection 讓 Copilot 自行寫入 `.vscode/settings.json` 開啟 YOLO mode, 達成完全 RCE 並可橫向感染其他 repo。

69.1 核心論點 (150-200 字繁中)

研究員 Johann Rehberger (embracethered.com) 揭露 GitHub Copilot agent mode 的設計缺陷: agent 可在無使用者確認下自行修改 workspace 內任意檔案, 包括自身的 configuration `.vscode/settings.json`。攻擊鏈為: (1) 把 malicious instruction 藏在 source code、GitHub issue、網頁等 Copilot 會 ingest 的內容中; (2) injection 指示 Copilot 寫入 `"chat.tools.autoApprove": true`; (3) 該設定即 YOLO mode, 從此 Copilot 執行任何 shell command 都不再向使用者確認; (4) 達成 RCE (Remote Code Execution, 遠端任意程式執行)。研究員示範 Windows 與 macOS 上彈出 calculator, 並進一步指出此 attack surface 可被串成 botnet 招募 (ZombAIs), AI virus (malicious instruction 隨 git 散佈), supply-chain attack。Microsoft 於 2025-08 patch tuesday 修復, 編號 CVE-2025-53773。

關鍵概念

69.2

1. **Prompt injection** (提示詞注入) — 把惡意指令藏進 LLM 會讀進 context 的內容 (code comment, issue, web page), 覆蓋原本的 system instruction。
2. **RCE (Remote Code Execution**, 遠端任意程式執行) — 攻擊者可在受害者機器上執行任意 shell 指令, 等同完全接管。
3. **YOLO mode (`chat.tools.autoApprove`)** — VS Code 實驗性設定, 開啟後 Copilot 所有 tool call 都不向使用者確認。
4. **CVE-2025-53773** — 此次漏洞的 Common Vulnerabilities and Exposures 編號 (CVE ≈ 全球漏洞身份證)。
5. **AI virus / ZombAI** — 受感染機器被招進 botnet; malicious prompt 也可隨 commit push 上游, 散佈到其他 repo 的 Copilot session。

69.3 對 CS146S 的意義

這個 case study 是「 agentic AI 自我修改自身權限 → privilege escalation 」的經典範例。Threat model 必須在設計時就把「 agent 能不能改 agent 自己的設定 」列為 critical question。對 W6 而言, 它示範 prompt injection 不只是「讓 chatbot 講髒話」而是真正能造成 RCE 等同傳統 OWASP 等級的 system compromise。

69.4 對 Vibe Coder 的 Takeaway

不要打開 Copilot / Cursor / Claude Code 的「全自動 approve」mode。`.vscode/settings.json` 進 git 時要 review; clone 別人 repo 第一件事是檢查 `.vscode/`、`.cursor/`、`.claude/` 等 agent config 資料夾。對外部 untrusted source (issue, web fetch, PDF) 抱「假設裡面有 prompt injection」的心態, 把 agent 的 file write / shell exec 權限收斂到專案資料夾內。

69.5 原文連結

[GitHub Copilot Remote Code Execution via Prompt Injection](#)

70. Finding Vulnerabilities in Modern Web Apps Using Claude Code and OpenAI Codex

71. Finding Vulnerabilities in Modern Web Apps Using Claude Code and OpenAI Codex

一句話摘要: Semgrep 對 11 個真實 Python web app (>800k LoC) 跑 Claude Code 與 OpenAI Codex 找漏洞, 找到真正 vulnerability 但 false positive 率 80%+, 且結果 non-deterministic, 結論是 LLM 不是 silver bullet, 要與傳統 SAST 混合用。

71.1 核心論點 (150-200 字繁中)

Semgrep 團隊用 Anthropic Claude Code (Sonnet 4) 與 OpenAI Codex (o4-mini) 對 11 個 actively maintained 的開源 Django/Flask/FastAPI 專案(共 >800,000 LoC) 跑 security audit, 鎖定 6 類漏洞: authentication bypass, IDOR, path traversal, SQL injection, SSRF, XSS。結果 Claude Code 找到 46 個真實漏洞, Codex 找到 21 個,但兩者 false positive 率分別高達 86% 與 82%。Claude 強在 IDOR (22% TPR), Codex 反而強在 path traversal (47% TPR), 但兩者都嚴重缺乏 inter-procedural taint flow tracking 能力, injection 類漏洞偵測極差。最警訊的是 non-determinism — 同一份 code 跑三次, 發現 3、6、11 個不同 bug, 源於 LLM 的 lossy context compaction。結論:LLM 已經能找真實 bug, 但需與傳統 rule-based SAST tool 結合,不能單獨用。

關鍵概念

71.2

1. **IDOR (Insecure Direct Object Reference, 不安全直接物件參照)** — API 沒檢查 user 權限就回傳指定 ID 的資料, 攻擊者改 ID 即可看別人的資料。
2. **SSRF (Server-Side Request Forgery, 伺服器端請求偽造)** — 騙 server 對內部網路或 metadata service 發 HTTP request。
3. **Inter-procedural taint flow** — 追蹤 user input (被「污染」的資料) 跨多個 function、檔案流動到 dangerous sink 的能力, 是傳統 SAST 強項、LLM 弱項。
4. **False positive rate** — 報告為 vulnerability 但實際不是的比例; 80%+ 表示十個警報只有兩個是真。
5. **Context rot / non-determinism** — LLM 處理大 codebase 時做 lossy summarization, 同 prompt 跑多次結果不一致。

71.3 對 CS146S 的意義

這是一篇實證打臉「LLM 取代 SAST」狂熱的實驗。它清楚示範 LLM 在哪類任務有用 (contextual reasoning、跨檔案 architectural pattern)、在哪類任務拉胯 (taint tracking、deterministic coverage), 對 W6 的 AI security tooling 討論提供關鍵的 baseline data。也順帶引出下一篇 context rot 的延伸閱讀。

71.4 對 Vibe Coder 的 Takeaway

請 Claude Code 「幫我 audit 這個 repo」可能找到真 bug, 但別以為跑一次就安全。實務上: (1) 跑多次 (≥3) 取聯集; (2) 把警報當「嫌疑名單」而非結論, 每條都人工 verify; (3) 仍要跑 Semgrep/CodeQL 等 deterministic tool 補強 injection 類盲區; (4) 把 prompt 鎖定特定 vulnerability class 而非「找所有漏洞」可降 false positive。

71.5 原文連結

[Finding Vulnerabilities in Modern Web Apps Using Claude Code and OpenAI Codex](#)

72. Agentic AI Threats: Identity Spoofing and Impersonation Risks

73. Agentic AI Threats: Identity Spoofing and Impersonation Risks

一句話摘要: Palo Alto Unit 42 示範 agentic AI 的 identity spoofing 攻擊 – 攻擊者可透過 prompt injection 騙 agent 從 cloud metadata service 偷 service account token、或藉 BOLA 越權讀他人資料; 漏洞 framework-agnostic, 源於 insecure design 而非 framework 本身。

73.1 核心論點 (150-200 字繁中)

Unit 42 研究指出 agentic AI (autonomous 軟體 agent + LLM + 外部工具/API/DB) 大幅擴張 attack surface, identity spoofing 成為新興 critical risk。研究員示範兩條主要攻擊路徑: (1) **Service account token exfiltration** – 攻擊者用惡意 prompt 操控 agent 的 code interpreter 去 query GCP metadata endpoint, 把 service account access token 偷出來, 等同拿到 cloud infra 的萬用鑰匙; (2) **BOLA (Broken Object Level Authorization, 物件層級授權失敗)** – 直接請 agent 「給我 user X 的交易紀錄」, underlying tool 沒檢查權限就回傳, 達成跨 user 越權讀取。關鍵發現: 以 CrewAI 與 AutoGen 兩個不同 framework 實作同樣 app, 漏洞同樣存在 – 證明問題出在 insecure design pattern、misconfiguration、unsafe tool integration, 而非 framework 本身。建議 layered defense: prompt hardening、code executor sandboxing、tool input sanitization、runtime content filtering。

關鍵概念

73.2

1. **Agentic AI (代理型 AI)** – 自主收集資料、做決策、呼叫外部 tool 達成多步驟目標的 AI 系統, 與只回答 query 的 chatbot 不同。
2. **Identity spoofing / impersonation (身份偽冒)** – 攻擊者讓 agent 以 legitimate user 或 system 的身份發動操作, 常透過偷取 credential 達成。
3. **BOLA (Broken Object Level Authorization)** – OWASP API Top 10 第一名漏洞; server 沒檢查使用者是否有權存取特定 object (如 user_id=123 的訂單)。
4. **Service account token / cloud metadata service** – 雲端 VM 內可從特定 IP (如 169.254.169.254) 抓到自身 service account 的 access token, 被偷走等同拿到該 service account 全部權限。
5. **Defense in depth (縱深防禦)** – Prompt hardening + sandbox + input sanitization + runtime filtering 多層疊加, 沒有單一 mitigation 夠用。

73.3 對 CS146S 的意義

把 W6 的討論從「LLM 本身會講錯話」推進到「LLM agent 操作 enterprise infra 會被劫持」, 它把 OWASP 等傳統 web security 概念 (BOLA、SSRF) 移植到 agentic AI 場景, 提供具體 threat model 與 defense layering, 是設計企業級 AI agent 必讀的 case。

73.4 對 Vibe Coder 的 Takeaway

自己寫 agent app 時: (1) tool 的 input 一律要 sanitize、authorization check 別假設 LLM 會幫你做; (2) code interpreter / shell tool 要關 outbound network 或 whitelist domain, 特別封掉 metadata IP (169.254.169.254); (3) system prompt 明確禁止 disclose credential; (4) 別把 production 級 token 放進 dev agent 的 env var。

73.5 原文連結

[Agentic AI Threats: Identity Spoofing and Impersonation Risks](#)

74. OWASP Top Ten (2021)

75. OWASP Top Ten (2021)

一句話摘要: OWASP Top Ten 是社群維護、約四年更新一次的 web application 十大風險清單, 2021 版以 incidence rate (發生率)取代 raw frequency 作為排序基準, 是 secure coding 的入門共識。

75.1 核心論點 (150-200 字繁中)

OWASP (Open Web Application Security Project, 開放網頁應用程式安全計畫)是非營利組織,旗下最具影響力的產出就是 **OWASP Top Ten** — 一份「standard awareness document for developers and web application security」, 被全球視為 secure coding 的第一步。2021 版 (最新已 finalized 的版本, 2025 版進行中) 已被翻譯成 9 種語言,包含繁體與簡體中文。方法學上,OWASP 不再只看 raw vulnerability frequency, 而是改用 **incidence rate** (每個 app 含至少一個某類 CWE 的機率), 並區分「Human-assisted Tools」(高量、來自工具) 與「Tool-assisted Human」(低量、來自人工 pen-test) 兩類資料來源,搭配 CVSS (Common Vulnerability Scoring System) 做嚴重度權重。資料來源涵蓋 security vendor、consultancy、bug bounty platform, accept verified 與 unverified contribution 但清楚標示。其價值在於提供開發團隊一個共同詞彙與排序, 幫助 prioritize security effort。

關鍵概念

75.2

1. **OWASP (Open Web Application Security Project)** — 非營利社群組織, 全球數百個 chapter, 發布大量 open-source security 資源。
2. **OWASP Top Ten** — 約四年更新一次的 web app 十大風險清單, 2021 版十類為 A01 Broken Access Control、A02 Cryptographic Failures、A03 Injection、A04 Insecure Design、A05 Security Misconfiguration、A06 Vulnerable Components、A07 Identification & Authentication Failures、A08 Software & Data Integrity Failures、A09 Security Logging Failures、A10 SSRF。
3. **CWE (Common Weakness Enumeration, 共通弱點列表)** — MITRE 維護的弱點分類體系(如 CWE-79 = XSS), Top Ten 的每一類對應一組 CWE。
4. **CVSS (Common Vulnerability Scoring System)** — 0-10 分的漏洞嚴重度標準評分, 用於 weighting。
5. **Incidence rate vs frequency** — Incidence rate = 「至少含一個」的 app 比例, 避免少數高頻 app 灌水。

75.3 對 CS146S 的意義

OWASP Top Ten 是討論 application security 的 lingua franca。W6 後續所有 case study (Copilot RCE、agentic AI 攻擊、Simgrep 找的漏洞) 都可對應到 Top Ten 的某一類 — A01 (Broken Access Control = IDOR、BOLA), A03 (Injection = SQL injection、prompt injection 的類比), A10 (SSRF = agentic AI 偷 metadata token 的攻擊路徑)。掌握這份清單就有能力把零碎漏洞分類歸納。

75.4 對 Vibe Coder 的 Takeaway

寫 web app 之前花 30 分鐘讀完 Top Ten 每一類的範例。最常踩的三個是: A01 Broken Access Control (API 沒檢查 user 是否能看這筆資料), A03 Injection (user input 直接拼進 SQL/shell/HTML), A05 Security Misconfiguration (debug mode 上 production、CORS 全開、default password)。每寫一個 endpoint 自問「這條對 Top Ten 哪幾項有曝險?」。

75.5 原文連結

[OWASP Top Ten](#)

76. Context Rot: Understanding Degradation in AI Context Windows

77. Context Rot: Understanding Degradation in AI Context Windows

一句話摘要: Chroma 對 18 個 SOTA LLM 跑擴展版 NIAH benchmark 證實 context rot — 即使 context window 號稱百萬 token, 模型在第 10,000 token 的可靠度顯著低於第 100 token, 且 distractor、coherent narrative、low semantic similarity 都會加劇衰減。

77.1 核心論點 (150-200 字繁中)

Chroma Research 挑戰業界「context window 變大 = 模型一樣可靠」的假設, 對 GPT-4.1、Claude 4、Gemini 2.5 等 18 個 SOTA 模型跑系統化測試, 發現 **context rot** (上下文腐爛): input 越長, performance 越不穩, 即使是極簡任務也一樣。實驗在傳統 NIAH (Needle in a Haystack, 大海撈針) 基礎上加入四個新維度: (1) **Needle-question similarity** — 語意相似度低的 query/needle pair 衰減更快; (2) **Distractor impact** — 即使一個 distractor (干擾項) 就會降效, 且不同 distractor 影響不一致; (3) **Haystack structure** — 反直覺地, coherent 有邏輯的 haystack 反而比 shuffled 雜亂版讓模型表現更差, 暗示 attention 機制被結構性敘事干擾; (4) **Repeated words** — 超過 2,500 字的 exact replication task 開始出現拒答、亂寫、位置錯置。LongMemEval 對話實驗也顯示「給整段對話 history」比「只給相關片段」表現顯著差, 即使開 reasoning mode 也無法補。結論: context engineering (如何排版、置入相關資訊) 比 context length 本身更重要。

關鍵概念

77.2

1. **Context rot (上下文腐爛)** — LLM 在長 input 下 performance 隨 token 數量非均勻衰減的現象, 即使任務簡單也會發生。
2. **NIAH (Needle in a Haystack) benchmark** — 把一句已知 fact 「needle」藏進大段不相關文本「haystack」, 測模型能否在末尾正確回答關於 needle 的問題。
3. **Needle-question similarity** — Needle 與 query 的語意相似度; 低相似度需要 inferential reasoning, 是衰減重災區。
4. **Distractor** — 與 needle 相關但非答案的混淆項; 單一 distractor 就足以拉低 performance。
5. **Context engineering** — 透過 prompt 結構、資訊排序、片段選取主動降低 context rot 的工程實踐, 是 RAG (Retrieval-Augmented Generation) 系統的關鍵設計變數。

77.3 對 CS146S 的意義

這篇是上一篇 Semgrep 文中 non-determinism 現象的根因解釋。對 AI security testing 而言, context rot 直接威脅 audit 完整性 — 大 codebase 一次餵給 LLM, 後段被掃過的 file 可能根本沒被「真的看到」。設計 AI 安全 pipeline 必須假設 context 是 lossy 的, 要有 chunking 與多次採樣策略。

77.4 對 Vibe Coder 的 Takeaway

別把整個 repo 一次塞給 Claude / Cursor。實務原則: (1) 一個 prompt 只解一個 task、context 給最相關的 3-5 個檔案; (2) 對話越長越不可靠 — 進入 debug 階段考慮開新 session; (3) RAG 系統不是「retrieve 越多越好」, retrieve 5 個高相關片段勝過 retrieve 50 個包山包海; (4) 如果發現 model 開始亂答, 先懷疑 context 太長, clear 後重來。

77.5 原文連結

[Context Rot: Understanding Degradation in AI Context Windows](#)

78. Vulnerability Prompt Analysis with O3 (CVE-2025-37899 use-after-free)

79. Vulnerability Prompt Analysis with O3 (CVE-2025-37899 use-after-free)

一句話摘要: Sean Heelan 公開的 system prompt – 教 OpenAI o3 用三段式 (code path → 條件分析 → 矛盾自檢) 找 Linux kernel 的 use-after-free 漏洞; 嚴守「寧可不報、不報 false positive」原則, 實際成功發現 CVE-2025-37899。

79.1 核心論點 (150-200 字繁中)

這份 raw prompt 是 Sean Heelan 用來驅動 OpenAI o3 model 在 Linux kernel source code 中尋找 use-after-free (UAF, 釋放後使用) 漏洞的 system prompt 範本, 且實際產出 CVE-2025-37899 – 一個被官方確認的 kernel 漏洞。Prompt 設計核心是三階段方法論: (1) **Code path documentation** – 從 entry point 到 vulnerability 點, 逐步寫出完整 code path; (2) **Conditional analysis** – 對 path 上每個 conditional statement, 具體說明攻擊者要如何控制條件以達成所需 outcome, 把理論漏洞轉成可行的攻擊腳本; (3) **Contradiction detection** – 提交前自我審查推理是否有矛盾或未經驗證的假設, 等同 self peer-review。最關鍵的紀律: 「It is better to report no vulnerabilities than to report false positives or hypotheticals」– 寧可漏報也不願誤報。對於遺漏的 helper function, 分兩類處理: 應用層的請使用者補定義; 屬於 Linux kernel API 的可在有信心時自行假設。

關鍵概念

79.2

1. **Use-after-free (UAF, 釋放後使用)** – 程式 free 一塊記憶體後, 仍持有 dangling pointer 並去 read/write 它, 可導致 memory corruption、資訊外洩或 RCE。
2. **CVE-2025-37899** – 由此 prompt 驅動 o3 找出的 Linux kernel UAF 漏洞, 是首個由 LLM 獨立發現並獲官方 CVE 編號的 kernel 漏洞之一。
3. **Dangling pointer (懸空指標)** – 指向已釋放或失效記憶體的 pointer, 是 UAF 的物質載體。
4. **Code path / conditional analysis** – 從程式 entry 走到 vulnerable sink 的完整執行路徑與沿途每個 if/else 的可控性分析, 是 manual exploit development 的標準語言。
5. **No-hypotheticals principle** – Prompt 明確要求所有報告必須附具體 code 範例與 step-by-step trace, 禁止「理論上可能」的描述。

79.3 對 CS146S 的意義

這是 W6 中「LLM 真的找到 zero-day」最具體的證據。對照前面 Semgrep 那篇 80% false positive 的悲觀結論, 本案告訴我們: 只要 **prompt** 設計夠精確、**scope** 夠窄、紀律夠嚴 (寧可不報), LLM 也能在 Linux kernel 這種高度複雜 codebase 找出真實 vulnerability。它把 prompt engineering 從「對話技巧」升級為一種正式的 security research methodology。

79.4 對 Vibe Coder 的 Takeaway

請 LLM 找 bug 時: (1) 範圍要窄 (指定一類漏洞、一個檔案, 而非「全部」); (2) 強制三階段輸出 (path → 條件 → 矛盾自檢), 不要只要結論; (3) 明確說「找不到就回 no findings、別亂掰」– 這條紀律會大幅降低 false positive; (4) 對自家專案 critical path (auth、權限、payment) 可仿此 prompt 寫專屬 security review template, 每次大改後跑一次。

79.5 原文連結

[Vulnerability Prompt Analysis with O3 \(CVE-2025-37899 use-after-free\)](#)

80. Code Reviews: Just Do It

81. Code Reviews: Just Do It

一句話摘要: Jeff Atwood 引用 Steve McConnell 《Code Complete》的數據力證 peer code review 是性價比最高的 software quality practice, 呼籲團隊「別再爭辯做不做、直接開始做」。

81.1 核心論點 (150-200 字繁中)

Atwood 的核心主張很直接: peer code review (同儕程式碼審查) 是單一個能夠最有效提升 code quality (程式品質) 的實務做法, 沒做就等於把唾手可得的 ROI 丟在桌上。他援引 McConnell 的 industry data 指出 design + code inspection 平均能抓到 55-60% 的 defect (缺陷), 遠高於各種 testing method (25-45%)。實證案例包括: 某 maintenance 組織導入 review 後 error rate 從 55% 降到 2%; 另一團隊 error 從每 100 行 4.5 降至 0.82; AT&T 因 review 取得 14% productivity 增益與 90% defect 減少; JPL 的 inspection 每場省下約 \$25,000。Atwood 的結論是: review 形式 (over-the-shoulder、email pass-around、pair programming、tool-assisted、formal Fagan inspection) 並非關鍵, 有做就贏。他用一句話收尾: 「every minute you spend in a code review is paid back tenfold」。

關鍵概念

81.2

1. **Peer Code Review** (同儕程式碼審查) — 程式碼提交前由另一位 developer 檢視, 是 Atwood 主張影響最大的單一實務。
2. **Defect Detection Rate** (缺陷偵測率) — Inspection 55-60% vs. testing 25-45%, 是 review 優勢的量化證據。
3. **Lightweight Review** (輕量化審查) — 不需要 Fagan-style 重型流程, over-the-shoulder 或 tool-assisted 即可開始。
4. **Fagan Inspection** (Fagan 形式審查) — IBM 1976 年提出的正式審查流程, 是 defect detection 數據的歷史源頭。
5. **ROI of Review** (審查投資報酬率) — Atwood 用「paid back tenfold」框架說服 manager 撥出時間做。

81.3 對 CS146S 的意義

這是 Week 7「Modern Software Support / AI Code Review」單元的歷史定位文章 — 在進入 AI reviewer (Diamond、CodeRabbit、Copilot Code Review) 的討論前, 先確立為什麼 code review 本身值得做。Atwood 2008 年的論點到 2026 年依然成立: AI reviewer 取代的是 review 過程中的部分 mechanical work, 而非取代 review 本身的價值主張。理解 inspection 55-60% defect detection 這個 baseline 數據, 後續才有辦法評估 AI reviewer 的 incremental contribution。

81.4 對 Vibe Coder 的 Takeaway

對非資工背景的 vibe coder 來說, 這篇是「為什麼我該幫自己寫的 code 做 review, 即使是一人專案」的最簡入門。實務上: (1) 用 AI reviewer (Diamond / CodeRabbit) 當作沒有同事時的替代 reviewer; (2) git commit 前自己 self-review 一次 diff, 等於 over-the-shoulder review; (3) 不要追求 Fagan-level 嚴謹, 有做就贏。

81.5 原文連結

[Code Reviews: Just Do It](#)

82. How to Review Code Effectively: A GitHub Staff Engineer's Philosophy

83. How to Review Code Effectively: A GitHub Staff Engineer's Philosophy

一句話摘要: GitHub staff engineer Sarah Vessels 把 code review 視為對話而非把關,強調「以提問取代命令、明示阻擋與否、給具體例子」三原則,讓 review 同時達成 quality control 與 knowledge transfer。

83.1 核心論點 (150-200 字繁中)

Vessels 把 review 拉高到「shape the product's implementation」的高度 — review 不是末端的 quality gate, 而是設計階段的延伸。她的 philosophy 圍繞三個操作原則: (1) **Ask questions over issuing demands**: 與其說「this won't work」, 不如問「will this perform well? how does this handle unexpected data?」, 把 reviewer 的疑慮轉成 author 自己思考的契機; (2) **Separate substance from preference**: 明確區分 blocker (會弄壞 production) 與 nit (個人偏好), 不該為了 nit 卡住 PR; (3) **Provide specific examples**: 抽象的「I don't like this」要換成具體的替代寫法 + relevant issue / 既有 PR 連結。她也提倡 self-review first、affirmation alongside critique (讚賞與批評並存)、draft PR 標示未完成、welcome post-merge review 為 learning opportunity, 以及用 code-owner team 控制 reviewer fatigue。

關鍵概念

83.2

1. **Ask Questions Over Demands** (提問代替命令) — 用疑問句框架 review comment, 降低 author 防衛心並引導自我思辨。
2. **Blocker vs Nit** (阻擋級 vs 偏好級) — 明確標示哪些是非改不可、哪些是可選 polish。
3. **Self-review First** (自審先行) — 開 PR 前自己先 diff 走一次, 省掉 reviewer 抓 trivial 問題的時間。
4. **Reviewer Fatigue** (審閱疲勞) — 用 code-owner team 與 notification filter 避免 reviewer 被同時 ping 爆。
5. **Knowledge Transfer Side Effect** — Review 順帶讓 reviewer 同步 codebase 知識, 不只是抓 bug。

83.3 對 CS146S 的意義

這篇是 Week 7 的 **human side** baseline — 提醒在討論 AI reviewer 之前, 先把「人類 reviewer 該怎麼留好 comment」這件事搞清楚。Vessels 的「ask questions over demands」「specific examples」「blocker vs nit」三原則, 恰恰是 AI reviewer (Diamond、CodeRabbit) 目前還抓不準的 social nuance — Tomas (Graphite) 演講中提到的「only 50% of human comments lead to action」也與 Vessels 提到的 nit-vs-blocker 不分有關。AI reviewer 設計時若 import 這些原則(例:自動標 nit prefix、用問句而非命令句), 就能複製人類 reviewer 的 social capital。

83.4 對 Vibe Coder 的 Takeaway

接到 AI reviewer 的 comment 時也適用同樣的 filter: (1) 它是 blocker 還是 nit? (自己判斷, 不要 100% follow) (2) 它有沒有給具體 example 而非抽象說法? (3) 「ask questions」這招也能反向用 — 你 prompt AI reviewer 時改用「what could go wrong with this?」而非「is this correct?」, 會得到更可行動的回覆。

83.5 原文連結

[How to Review Code Effectively: A GitHub Staff Engineer's Philosophy](#)

84. AI-Assisted Assessment of Coding Practices in Modern Code Review

85. AI-Assisted Assessment of Coding Practices in Modern Code Review

一句話摘要: Google 在內部部署的 LLM-based code review 助手 AutoCommenter, 能涵蓋 68% human reviewer 引用的 best practice, 其中 66% 屬於傳統 linter 抓不到的範疇, 且實測 comment-resolution rate 約 40%。

85.1 核心論點 (150-200 字繁中)

這篇 AIware '24 paper 由 Google + University of Washington 團隊撰寫, 報告 AutoCommenter 在 Google 內部給數萬名 developer 使用的開發、部署與評估經驗。AutoCommenter 是 LLM-backed 的 code review assistant, 覆蓋 C++ / Java / Python / Go 四種語言, 目標是把 best practice violation 檢測自動化、釋放 human reviewer 去看 functionality。三個關鍵 finding: (1) **Coverage**: AutoCommenter 對 68% human reviewer 常引用的 best practice 都能產出 comment, 且其中 66% 是 traditional static analysis (linter) 做不到的 (例: comment clarity、justified naming exception); (2) **Resolution rate**: 約 40% AutoCommenter comment 被 author 在 subsequent commit 中實際 resolve (手動抽樣 40 個確認, 80% 是真的針對該 comment 修的); (3) **Lessons learned**: intrinsic evaluation (offline metrics) 與 real-world performance 經常背離; user trust 一旦因少數 false positive 受損就難回復; URL suppression 這類簡單機制就能把 user acceptance 拉到 80%+。

關鍵概念

85.2

1. **AutoCommenter** — Google 內部 LLM-backed code review assistant, 覆蓋 4 種語言、數萬 developer 日常使用。
2. **Best Practice Coverage Beyond Linter** (超出 linter 範疇的最佳實務涵蓋) — 66% 的 best practice violation 無法用 traditional static analysis 表達。
3. **Comment Resolution Rate** (評論解決率) — Comment 是否真的促成 author 改 code 的比率, AutoCommenter 約 40%, 與 human reviewer comment 的 ~50% 接近。
4. **URL Suppression** (URL 抑制機制) — 對 false-positive 比率高的 best practice URL 直接停止觸發, 是實務上拉高 user acceptance 最有效的單一動作。
5. **Intrinsic vs Extrinsic Evaluation** (內在 vs 外在評估) — Offline metric 漂亮不代表 production 會被接受, 需要 deployment 後持續 user feedback monitoring。

85.3 對 CS146S 的意義

這是 Week 7 唯一的 peer-reviewed academic paper, 提供 Diamond / CodeRabbit / Copilot Code Review 等商用工具背後的 academic baseline。Google 證明了三件對課程很關鍵的事: (1) LLM 真的能補足 linter 抓不到的 nuanced practice; (2) 40% resolution rate 是合理 baseline — 課堂在評估 AI reviewer 效果時可用此數字當參照; (3) deployment 成功的關鍵不是 model quality, 而是 false positive suppression + user feedback loop 這些 system engineering 問題。

85.4 對 Vibe Coder 的 Takeaway

選用 AI reviewer 時, 重點不是它「抓到多少 bug」, 而是它的 false positive 是否好停 (有沒有 suppress 機制 / per-rule mute)。Resolution rate 預期 40-50% 算正常, 不要期待 100%。Self-host 一個 LLM reviewer 成本太高, 先用現成的 Diamond / CodeRabbit / GitHub Copilot Code Review, 把心力花在 prompt 與 rule 微調。

85.5 原文連結

[AI-Assisted Assessment of Coding Practices in Modern Code Review \(arXiv 2405.13565\)](#)

86. AI Code Review Implementation Best Practices

87. AI Code Review Implementation Best Practices

一句話摘要: Graphite 主張 AI code reviewer 應走 human-in-the-loop 路線, 靠 sensitivity tuning、false-positive feedback loop 與 priority level 三件事, 把 signal-to-noise 維持在 developer 願意持續用的水準。

87.1 核心論點 (150-200 字繁中)

Graphite 的 implementation guide 把 AI code review 定位成 **augmenting** (擴增) 而非 **replacing** (取代) human reviewer。整合上採 human-in-the-loop 模式: AI 跑 first pass 抓明顯問題、human 驗證 AI 建議、團隊持續追蹤 acceptance/rejection rate 來精修系統。Signal-to-noise 是首要挑戰, 建議三招: (1) 對不同 issue type 調 sensitivity; (2) 建立 false-positive feedback loop (developer 標 false positive、系統學習); (3) 設 priority level, 把火力集中在 high-impact 區。AI 擅長: style inconsistency、basic logic error、security vulnerability、performance inefficiency (N+1 query 等), syntax issue。Human 仍須處理: architectural decision、complex business logic、context-specific issue、需要 domain knowledge 的 subjective improvement。Workflow 上透過 GitHub webhook 觸發、用 config file 指定 severity 與 focus、custom rule 適配各 codebase、結合 deep codebase context 分析。

關鍵概念

87.2

1. **Human-in-the-Loop AI Review** (人類介入式 AI 審查) — AI first pass + human gate, 不走 full automation。
2. **Signal-to-Noise Ratio** (訊號雜訊比) — AI reviewer 成敗的單一最重要指標, false positive 多就會被 ignore。
3. **False-Positive Feedback Loop** — Developer 標記 false positive → 系統下次降低 confidence/suppress, 是 self-improving 的核心。
4. **Sensitivity Tuning** (敏感度調校) — 對不同 issue type (security 高敏 / style 低敏) 設不同門檻。
5. **Graphite Diamond** — Graphite 自家 AI reviewer, 本篇是其 design philosophy 的 vendor 版說明。

87.3 對 CS146S 的意義

對應 Week 7 的 vendor perspective — 是 academic paper (AutoCommenter) 的商業翻譯版。Graphite 的 false-positive feedback loop 與 Google 的 URL suppression mechanism 在 architectural 層面其實是同一件事「承認 LLM 會錯, 留出 retract 通道」是當前 AI reviewer 工程實務的共識。課程設計上, 這篇可作為「市面有哪些工具、評估哪些指標」的 buyer's guide, 配合 Tomas Reimers 的 YouTube 演講看 Diamond 的 internal metric 演進, 會更立體。

87.4 對 Vibe Coder 的 Takeaway

部署 AI reviewer 到自己的 GitHub repo 時的 setup 順序: (1) 先選 high-impact category (security + obvious bug) 開啟, nit / style 暫時關掉; (2) 兩週後檢視 false positive 比例, >20% 就再降 sensitivity; (3) 用 Graphite Diamond 或 CodeRabbit 都可以, 重點是有 **thumbs-down feedback** 機制的工具, 這樣它會學。

87.5 原文連結

[AI Code Review Implementation Best Practices](#)

88. Code Review Essentials for Software Teams

89. Code Review Essentials for Software Teams

一句話摘要: Blake Smith 把 code review 重新定義成「保持團隊心智模型同步」的對齊工具, 而非單純的 quality gate, 並用「scalpel-driven development」框架主張小而精準的 PR 才能維持 review 的高品質。

89.1 核心論點 (150-200 字繁中)

Smith 的核心 framing 把 code review 視為 hierarchy of needs, 最底層是團隊對齊 (alignment): 「當 Bob 提交 accounting subsystem 的 PR, Amy 在 review 中同步更新她對該 system 的 mental model」。這個 framing 把 review 從「擋 bug」升級為「擴散知識 + 降低未來修改的 onboarding 成本」。實務面分三段: (1) **Before coding** — 寫 code 前先確認 priority 與 design 共識, 把改動拆成「scalpel-like」小 cut; (2) **Pull request descriptions** — 提供 problem statement、high-level structural change、testing verification, 不要逼 reviewer 自己解碼; (3) **Reviewer tone** — 用 clarifying question 取代 judgment, 「how does this handle edge cases?」而非「this design is broken」, 維持 psychological safety。Smith 也提醒不要 over-index 在 style nitpick 上, formatting 應交給 automation 處理, review 本身專注在 architectural alignment。

關鍵概念

89.2

1. **Mental Model Synchronization** (心智模型同步) — Review 的首要功能是讓團隊對 codebase 維持共同理解, bug detection 反而是 side effect。
2. **Scalpel-Driven Development** (手術刀式開發) — 把 PR 切成小而精準的 cut, 避免 machete-like 大刪除, 是高品質 review 的前提。
3. **PR Description Discipline** (PR 說明書紀律) — Problem + structural change + testing 三段, 省 reviewer 解碼成本。
4. **Clarifying Question Tone** (澄清式提問語氣) — 用問句框架 critique, 維持 psychological safety。
5. **Automate the Style, Review the Architecture** — 把 formatting 交給 linter / formatter, review 專注在設計層面。

89.3 對 CS146S 的意義

這篇 2015 年的文章是 Week 7 的「為什麼 review 不能完全交給 AI」基礎論述。Smith 強調的 mental model synchronization 是 AutoCommenter / Diamond 等 AI reviewer 目前完全無法替代的 — LLM 可以指出 N+1 query, 但無法讓你的隊友因為讀 PR 而學到「這個 subsystem 為什麼這樣設計」。Tomas (Graphite) 演講中提到 LLM 可以 catch 但 human 不想收的 comment (add test、extract function), 恰恰對應 Smith 說的「don't over-index on nitpicks」。AI 處理 **mechanical layer**, **human review** 處理 **alignment layer**, 是 Week 7 整體的整合論述。

89.4 對 Vibe Coder 的 Takeaway

即使是一人專案也要寫 PR description — 把「這次改了什麼、為什麼、怎麼測」寫清楚, 三個月後的自己 (與接手的 AI agent) 會感謝你。第二個 takeaway: 不要做超大 PR, 500 行以上的 PR 不論 human 還是 AI reviewer 都會 skim 過去; 切成 5 個 100 行 PR 的 review 品質會好 5 倍。

89.5 原文連結

[Code Review Essentials for Software Teams](#)

90. Lessons from Millions of AI Code Reviews

91. Lessons from Millions of AI Code Reviews

一句話摘要: Graphite co-founder Tomas Reimers 用 2x2 quadrant (LLM 能否 catch × human 是否 want to receive) 解釋為什麼 AI code reviewer 必須只留「右上象限」comment, 並以「comment 是否導致 code 改動」當作 metric, 把 Diamond 推到與 human reviewer 並駕齊驅的 52% action rate。

91.1 核心論點 (150-200 字繁中)

Reimers 的演講源自 Graphite 開發 Diamond(AI code reviewer)的實戰經驗。他先承認 raw LLM review 的失敗模式: 模型會自信地留下「revert this code」「CSS doesn't work this way (其實會)」這類錯誤 comment, 徹底耗損 user trust。關鍵 insight 是用 2x2 quadrant 把 comment 分類: 縱軸 = LLM 能否 reliably catch, 橫軸 = human 是否 want to receive。只有右上象限 (LLM 能 catch 且 human 想收) 的 comment 該被留下。LLM 不擅長: codebase-specific convention (住在資深工程師腦中、沒文件)。Human 不想收但 LLM 愛留: add test、extract function、comment this 等 always-correct-but-unwelcome nit。Metric 設計也是核心: 他們選用「comment 是否導致 author 改 code」當作 success metric, 因為 thumbs-up/down 收集率太低 (<4% 下票率沒區辨力)。Human reviewer 的 comment 約 50% 帶來 action, Diamond 經過調校到 March 已達 52%, 等同 human baseline。

關鍵概念

91.2

1. **Diamond** – Graphite 的 AI code reviewer 產品, 串 GitHub PR 自動找 bug。
2. **LLM-Capability × Human-Wantedness Quadrant** – 把 comment 分類的 2x2 框架, 只留右上象限。
3. **Action Rate Metric** (行動率指標) – Comment 是否導致 author 在該 PR 中改 code, 比 thumbs-up/down 更有區辨力的 success metric。
4. **Codebase-Specific Convention Blind Spot** – LLM 對「住在資深工程師腦中、沒寫成文件」的 convention 完全抓不到。
5. **Always-Correct-But-Unwelcome Comment** – 「add test / extract function」這類技術上沒錯但 reviewer 不該無腦留的 comment, 是 LLM 的常見 over-call。
6. **52% vs 50% Parity** – Diamond 的 action rate 已追平 human reviewer baseline, 是 Reimers 的核心 claim。

91.3 對 CS146S 的意義

這是 Week 7 最 practical 的 industry talk – 給 Diamond 內部 metric 與 prompt evolution 的 first-hand 故事。Reimers 的 quadrant 框架是課程整合 5 篇 reading 的最佳工具: Atwood / Vessels / Smith 講的 human review 原則 → 對應到 quadrant 的「human-wantedness 軸」; AutoCommenter / Graphite guide 講的 false positive 控制 → 對應「LLM-capability 軸」; Action rate 52% 也提供了一個可量化的「AI reviewer 何時算成功」門檻, 比模糊的 user satisfaction 更可被 benchmark。

91.4 對 Vibe Coder 的 Takeaway

評估自己的 AI reviewer 設定時, 問兩個問題: (1) 它留的 comment 我有沒有真的去改? (< 30% 就該調 prompt 或換工具) (2) 它有沒有對 codebase-specific convention 留錯 comment? 有 → 加 system prompt 把該 convention 寫進去 (這就是 LLM 的 blind spot 補強法)。也別期待 AI reviewer 100% 對 – 50% action rate 是 human baseline, Diamond 也才剛追平。

91.5 原文連結

[Lessons from Millions of AI Code Reviews](#)

92. Introduction to Site Reliability Engineering

93. Introduction to Site Reliability Engineering

一句話摘要: SRE (Site Reliability Engineering, 網站可靠度工程)是 Google 把營運當成軟體工程問題的解法 – 用 error budget 把開發速度與系統穩定的衝突量化、用自動化避免團隊隨服務量線性膨脹。

93.1 核心論點 (150-200 字繁中)

傳統 sysadmin (系統管理員) 模式把 dev 與 ops 分家, 導致兩派目標互斥 (dev 想快速上線、ops 想穩定不動)、團隊隨流量線性擴張。Google 提出: 找軟體工程師來設計營運團隊, 把 manual operation 當成 bug 來自動化掉。SRE 的核心紀律是 50% 規則 – 工程師花在 toil (重複瑣事) 的時間不得超過一半, 另一半必須做工程改善; 以及 **error budget** (錯誤預算) – 既然 100% availability (可用性) 既不可能也無價值, 那就明訂目標 (如 99.99%), 剩下 0.01% 是「可以拿來冒險的預算」。新功能上線會吃掉預算, 預算用完就停止 release。這把原本主觀的「能不能上線」變成可量化的決策, 徹底化解 dev/ops 衝突。其餘關鍵紀律包含: blame-free postmortem (不究責檢討)、on-call 一班最多 2 個 incident、monitoring 只通知需要人介入的事件。

關鍵概念

93.2

1. **SLI / SLO / SLA** (**service level indicator / objective / agreement**, 服務水準指標 / 目標 / 協議) – SLI 是測量值 (如 latency p99), SLO 是內部目標 (99.9% 請求 < 200ms), SLA 是對客戶的合約承諾。
2. **Error Budget** (錯誤預算) – 1 減去 SLO 的容忍失效空間; 用來分配給新功能 release 的風險「額度」。
3. **Toil** (瑣事) – 重複、可自動化、無工程價值的營運工作; SRE 必須壓在 50% 以下。
4. **Blame-free Postmortem** (不究責事後檢討) – incident 後檢討聚焦 systemic root cause, 不指責個人, 鼓勵透明回報。
5. **On-call** – 工程師輪班待命處理 production incident; Google 規定一個 8-12 小時班最多收 2 個 paging。

93.3 對 CS146S 的意義

Agent post-deployment 階段就是 SRE 場景。LLM agent 上線後不會「寫完就結束」, 而是要 monitoring、incident response、postmortem。Error budget 概念可直接套用 agent – 例如「allow 0.5% 的 hallucination rate」, 超過就觸發回滾或停止部署新 prompt 版本。

93.4 對 Vibe Coder 的 Takeaway

別追求 100% perfect, 那是無底洞。先決定可接受的失敗率 (error budget), 用它做 release 決策。寫 monitoring 時問自己「這個 alert 響了, 人需要做什麼? 如果答案是「沒事」就不該是 alert (應該是 log 或 ticket)。Postmortem 不是找戰犯, 是找系統性漏洞。

93.5 原文連結

[Introduction to Site Reliability Engineering](#)

94. Observability Basics You Should Know

95. Observability Basics You Should Know

一句話摘要: Observability (可觀測性) = 用 logs (日誌)、metrics (指標)、traces (追蹤) 三柱看穿系統內部狀態, traces 特別適合找出 distributed system 中的瓶頸。

95.1 核心論點 (150-200 字繁中)

Monitoring (監控) 只告訴你「什麼壞了」(CPU 高、5xx 多); observability 要回答「為什麼壞」。在 microservices (微服務) 架構下, 一個 user request 可能跨 10+ 個 service, 光看單一 service 的 metrics 完全不知道哪一段慢。observability 三柱各司其職: **logs** 記錄離散事件(誰在某時做了什麼), **metrics** 是聚合數值 (QPS、p99 latency、error rate), **traces** 把整個 request 從進入點到回應的完整路徑串起來。Trace 由多個 span (區段) 組成, 每個 span 是一個工作單位 (一次 DB query、一次 RPC call), spans 用 parent-child 關係階層組織。文章強調 traces 的關鍵是 **context propagation** (上下文傳播) — trace ID 必須跨 service 傳遞才能串成完整路徑。OpenTelemetry 是業界中立標準。三柱要相互關聯 (trace ID 寫進 log、metric 帶上 trace exemplar), 單看任一柱都不夠。

關鍵概念

95.2

1. **Logs** (日誌) — 離散文字事件記錄, 類似醫院的病歷紀錄(誰、何時、做了什麼、結果)。
2. **Metrics** (指標) — 時間序列數值(CPU usage、request count), 類似生命徵象(HR、BP、SpO₂) 持續監測。
3. **Traces** (追蹤) — 請求流經各 service 的完整路徑, 類似影像檢查 (CT / MRI) — 一次性看到全身結構與問題位置。
4. **Span** (區段) — Trace 的最小單位, 代表一次具名的工作 (如「query DB」「call payment API」)。
5. **OpenTelemetry** (OTel) — 廠商中立的 observability instrumentation 標準。
6. **Context Propagation** (上下文傳播) — Trace ID 在 service 之間傳遞, 確保跨服務 request 能被串成同一條 trace。

95.3 對 CS146S 的意義

LLM agent 系統就是典型的 distributed system — 一個 user query 可能觸發 LLM call → tool call → DB query → 另一個 LLM call。沒有 tracing 等於 debug 黑箱。Agent post-deployment 必備 observability stack, 特別是 traces 來看 token cost、latency、tool failure 在哪一步發生。

95.4 對 Vibe Coder 的 Takeaway

寫 app 時從第一天就加 structured logging (JSON log, 含 request ID)。Vibe coding side project 不必上 Datadog, 用 Sentry / OpenTelemetry SDK 就能撈 80% 價值。Traces 比想像中重要 — 你以為「app 慢」往往是某個 third-party API 慢, traces 一眼看到。

95.5 原文連結

[Observability Basics You Should Know](#)

96. Kubernetes Troubleshooting with AI

97. Kubernetes Troubleshooting with AI

一句話摘要: Resolve.ai 提出 agentic AI 當「24/7 Kubernetes 專家」— 用 knowledge graph 把 pod / node / service 關係串起來,自動跑 hypothesis testing 找 root cause, 解決 K8s 三大痛點: alert fatigue、ephemeral context loss、observability fragmentation。

97.1 核心論點 (150-200 字繁中)

Kubernetes (K8s, 容器編排平台) 的 troubleshooting 出了名地痛苦, 原因有三: (1) **Alert fatigue** (警報疲勞) — 平台會自動產生大量會自我恢復的雜訊 alert, 把真正關鍵問題淹沒; (2) **Ephemeral context loss** (短暫上下文流失) — container — crash 重啟, 裡面的診斷資訊 (log、stack trace) 就消失了; (3) **Observability fragmentation** (觀測資料碎片化) — 同一事件的證據散在 logs、metrics、events、kubectl describe 各處, 工程師要在多個 dashboard 與 CLI 之間來回切換手動串連。Resolve.ai 的解法是部署一個 AI Production Engineer agent, 做三件事: (a) 建 **knowledge graph** (知識圖譜) 把 pod、node、service、deployment 的關係模型化, 看出 systemic pattern; (b) **intelligent filtering** 過濾雜訊只保留相關訊號; (c) **automated investigation workflow** 自動跑假設驗證與 root cause analysis。整個 workflow 從「人類盯 dashboard」轉成「AI 給結論、人類審核」。

關鍵概念

97.2

1. **Kubernetes** (K8s, 容器編排) — 自動化部署、擴展、管理 containerized application 的開源平台; 類似醫院的「自動化護理排班系統」管理大量病床 (containers)。
2. **Pod** — K8s 最小部署單位, 內含一或多個 container; 類似一張病床(可住一或多個相關病人)。
3. **Container Orchestration** (容器編排) — 自動處理 container 的部署、擴縮容、健康檢查、failover。
4. **Knowledge Graph** (知識圖譜) — 把系統中各元件 (pod / service / node) 的關係建成圖結構, 方便推理「A 壞了會影響哪些 B」。
5. **Ephemeral Context** (短暫上下文) — container 生命週期短、重啟後狀態消失的特性。
6. **Hypothesis Testing Workflow** (假設驗證流程) — agent 提出多個 root cause 候選, 逐一驗證證據後排序。

97.3 對 CS146S 的意義

K8s troubleshooting 是 LLM agent 在 SRE 場景的最佳 use case — 它需要 multi-source data fusion、systematic reasoning、tool use (kubectl, prometheus query), 都是 LLM agent 強項。展示了 agent 不是取代工程師, 而是把 manual investigation 自動化到「給結論、人類確認」的層級。

97.4 對 Vibe Coder 的 Takeaway

如果你的 side project 還沒大到要用 K8s, 別被嚇到 — 大部分 vibe coding 用 Vercel / Render / Fly.io 就夠。但要學的概念是: 當系統複雜到單一 dashboard 看不完, 就需要 AI agent 來做第一輪 triage。寫程式時把每個元件的依賴明確寫出來 (compose.yaml, infra-as-code), 未來才好 debug。

97.5 原文連結

[Kubernetes Troubleshooting with AI](#)

98. Your New Autonomous Teammate

99. Your New Autonomous Teammate

一句話摘要: Resolve.ai 的 AI Production Engineer 把 incident response 全流程自動化 — 持續維護整個系統的 dynamic knowledge graph、alert 觸發後自動跑 just-in-time runbook 並在 1 分鐘內提出 root cause hypothesis、最後自動寫 postmortem。

99.1 核心論點 (150-200 字繁中)

工程師時間應該花在「建造」而非「救火」。Resolve.ai 的 product 把自己定位成 autonomous teammate (自主隊友)，核心架構是一個會持續更新的 **dynamic knowledge graph** — 接 Grafana / Datadog 抓 observability 資料、接 Jenkins 抓 CI/CD 部署紀錄、接 GitHub 抓 code change，把整個系統的依賴與狀態建成一張即時更新的圖。Alert 觸發後 agent 自動執行四步驟: (1) 跑 **just-in-time runbook** (即時生成的調查腳本) 抓相關 metrics / log / dashboard / 最近 deployment / code diff; (2) 通常 1 分鐘內提出 root cause theory (不論是 config error、code change、downstream service 故障或 deployment 問題); (3) 給可執行的 remediation step (rollback、restart pod 等)，人類可在 Slack 對話中追問或授權執行; (4) incident 結束後自動產生 detailed post-incident review。關鍵設計哲學: agent 不是取代工程師，而是「teammate」— 提供推理與動手能力，最終決策仍由人控制。

關鍵概念

99.2

1. **Dynamic Knowledge Graph** (動態知識圖譜) — 持續更新的系統元件依賴圖，是 agent reasoning 的基礎。
2. **Just-in-time Runbook** (即時 runbook) — 不靠預寫的 SOP，而是 incident 當下根據 context 動態產生的調查步驟。
3. **Incident Response** (事件應變) — 從 alert 觸發到問題解決的整套流程。
4. **Root Cause Theory** (根因假設) — agent 提出的可能 root cause 候選清單，附證據與信心度。
5. **Post-Incident Review / Postmortem** (事後檢討) — incident 結束後紀錄 timeline、root cause、教訓的文件。
6. **Conversational Delegation** (對話式委派) — 工程師透過 Slack / UI 自然語言指派 agent 執行 rollback、restart 等動作。

99.3 對 CS146S 的意義

這是 W9 主題「agent post-deployment」的具體商業案例。展示一個 production-grade agent 必備元素: (1) 持續更新的 world model (knowledge graph)、(2) tool use (observability platforms、CI/CD、code repo)、(3) human-in-the-loop (不全自動執行破壞性動作)、(4) 自動化文檔產生 (postmortem)。

99.4 對 Vibe Coder 的 Takeaway

「Autonomous teammate」是 agent 設計的好心法 — 不要做「按一下執行 100 步」的黑箱，要做「在每個關鍵節點停下來解釋、讓人類確認」的協作者。寫 agent 時就算只有自己一個 user 也要設計 conversational interface (解釋為什麼這樣做、提出多個 option)，這比 fully autonomous 的 agent 更實用、更安全。

99.5 原文連結

[Your New Autonomous Teammate](#)

100. Role of Multi Agent Systems in Making Software Engineers AI-native

101. Role of Multi Agent Systems in Making Software Engineers AI-native

一句話摘要: Single agent 在 production debugging 是 sequential bottleneck, multi-agent system 讓多個專業 agent 並行調查不同 domain (trace、DB、deployment、security log) 再 merge 結論, 才是真正 AI-native 的工程模式。

101.1 核心論點 (150-200 字繁中)

雖然 AI 已經改變了 code generation (GitHub Copilot、Cursor), 但 production debugging 仍是手動且碎片化。文章主張: 要真正讓 software engineering 成為 AI-native, 工程師應該以「協調 multi-agent system」為主要介面, 而不是用 AI 加速傳統 workflow。Single agent 在 system 變複雜時會變成 **sequential bottleneck** (序列瓶頸) — 它一次只能調查一個假設, 但 production incident 的時間壓力要求並行假設驗證。Multi-agent 架構讓專業 agent 各司其職並行運作: 一個追 microservice trace、一個分析 DB performance、一個 review 最近 deployment、一個掃 security log、一個評估 auto-scaling、一個估算 customer impact, 最後 merge 成統一診斷。但並行的代價是需要 **formal coordination protocol** 來避免 race condition 與 deadlock。建構 production-ready multi-agent system 需要罕見的雙重專業 — 深度 production domain knowledge 加上 sophisticated AI architecture, 缺一邊就會做出「會調查但調查錯方向」的系統。

關鍵概念

101.2

1. **Multi-Agent System** (多代理系統) — 多個各有專業領域的 AI agent 協同工作, 類似醫院的 multidisciplinary team (MDT) 會議 — 心臟科、腫瘤科、放射科專家並行評估同一病人再開會 merge 結論。
2. **Sequential Bottleneck** (序列瓶頸) — Single agent 一次只能處理一個 hypothesis, 無法 scale 到多 domain 並行調查。
3. **Parallel Hypothesis Testing** (並行假設驗證) — 多 agent 同時測試不同 root cause 候選, 比序列快數倍。
4. **Coordination Protocol** (協調協定) — 防止 agent 之間 race condition / deadlock 的 formal communication 規範。
5. **AI Native Engineering** — 工程師主要介面從 IDE / dashboard 變成 agent orchestration, 而非把 AI 當外掛工具。
6. **Specialized Agent Roles** (專業代理角色) — Trace agent、DB agent、deployment agent、security agent 等分工明確的角色。

101.3 對 CS146S 的意義

直接回應 CS146S 課程主軸 — agent system 不只是 single LLM call, 而是多 agent 協作架構。Post-deployment 場景特別需要 multi-agent, 因為 production incident 本質上跨 domain。對應 anthropic-skills 提到的 sub-agent pattern (主 agent 派 sub-agent 並行查詢) 就是同概念在 dev workflow 的應用。

101.4 對 Vibe Coder 的 Takeaway

寫 LLM app 時別執著 single mega-prompt 解決所有事。把任務拆成幾個專業 sub-agent (如 search agent、summarize agent、verify agent) 並行跑, 輸出品質與速度都比 single agent 好。Claude Code 的 Task tool / sub-agent 就是這個 pattern — 學會用它, 是從 vibe coder 進階到 AI-native engineer 的關鍵分水嶺。

101.5 原文連結

[Role of Multi Agent Systems in Making Software Engineers AI-native](#)

102. Top 5 Benefits of Agentic AI in On-call Engineering

103. Top 5 Benefits of Agentic AI in On-call Engineering

一句話摘要: Agentic AI 在 on-call 場景帶來五大好處 — 消除 alert fatigue、維護動態知識、確保調查一致性、提供證據式協作、主動找出潛在問題。

103.1 核心論點 (150-200 字繁中)

On-call engineering (待命工程) 是 SRE 工作中最痛的部分 — 半夜被 page、看著一堆 alert 不知從何下手、查到天亮才發現是 known issue。Agentic AI 從五個面向重塑這個 workflow: (1) 消除 **alert fatigue** — alert 一響 agent 立刻啟動調查、幾分鐘內提 root cause 與行動建議,工程師只需 review; (2) 維護動態知識 — 不靠會過時的 static runbook, agent 持續從每次 incident 學習,即使 senior 離職核心 know-how 也保留下來; (3) 確保調查一致性 — agent 跟著一致 workflow 但會根據每次 incident context 調整,杜絕「忘了檢查某步」的人為失誤;(4) 證據式協作 — agent 即時記錄每個動作與證據,不只加速當下解決,也產出 postmortem 與未來改善的素材;(5) 主動式解決 — 不只被動回應, agent 分析 metrics / log / 行為 pattern 找出尚未爆發的潛在問題,把 incident 攔在發生之前。對應解決的 SRE 痛點: alert fatigue、knowledge loss、inconsistent MTTR、reactive firefighting、preventable downtime。

關鍵概念

103.2

1. **On-call** — 工程師輪班待命處理 production incident, 類似住院醫師的 night float / overnight call。
2. **Alert Fatigue** (警報疲勞) — 因 alert 過多且多為雜訊,導致 responder 對 alert 麻木、漏掉關鍵警報; 類似醫院的 alarm fatigue (病房 monitor 響太多次護理師會自動忽略)。
3. **MTTR** (mean time to resolve / repair, 平均修復時間) — 從 incident 發生到解決的平均時間,是 SRE 核心 KPI。
4. **Static Runbook vs. Dynamic Knowledge** — 預寫的 SOP 文件(容易過時) vs. agent 持續從 incident 累積的活知識。
5. **Proactive Resolution** (主動式解決) — 在 incident 發生前就從 pattern detection 找出徵兆並處理。
6. **Evidence-based Collaboration** (證據式協作) — 每個調查步驟與發現都被記錄、可追溯,方便團隊審查與事後檢討。

103.3 對 CS146S 的意義

對應 W9 主題的核心應用 — agent 在 SRE / on-call 場景的價值主張。五項 benefit 對應五個典型 LLM agent 能力: fast triage、persistent memory / knowledge graph、structured workflow、structured logging、anomaly detection。是評估「agent 該不該部署在某 workflow」的好 checklist。

103.4 對 Vibe Coder 的 Takeaway

就算你是個人 side project 的 solo dev, 這五點仍適用 — 把自己過去的 debug 紀錄餵給 LLM agent 當 dynamic knowledge、用 agent 跑一致的調查 workflow (先看 log、再看 metric、再看最近 commit)、讓 agent 產 incident 紀錄。最有價值的是「proactive resolution」— 定期讓 agent 跑健康檢查找出潛在問題,比等 user 抱怨好太多。

103.5 原文連結

[Top 5 Benefits of Agentic AI in On-call Engineering](#)

104. 9 Weekly Assignments — 自學者可行性評估

CS146S 共有 9 個 weekly assignment (W10 沒有,因為到了 final project demo 階段)。所有 assignment 在 github.com/mihail911/modern-software-dev-assignments。

下表評估每個 assignment 對自學者的可行性(不是 Stanford 學生也能做嗎?需要什麼前置?)。所有 assignment 都連到 [GitHub repo](#)。

104.0.1 各週 assignment 摘要

W1: LLM Prompting Playground - 時間:2-4 hr | 可行性: ★★★★★ 完全可做 - 前置: OpenAI 或 Anthropic API key

W2: First Steps in the AI IDE - 時間:3-5 hr | 可行性: ★★★★★ 完全可做 - 前置: Cursor 或 Claude Code 安裝

W3: Build a Custom MCP Server - 時間:4-8 hr | 可行性: ★★★★★ 完全可做 (需 Node/TS 基礎) - 前置: Node.js + TypeScript SDK

W4: Coding with Claude Code - 時間:4-6 hr | 可行性: ★★★★★ 完全可做 - 前置: Claude Code 訂閱

W5: Agentic Development with Warp - 時間:3-5 hr | 可行性: ★★★★★ 完全可做 - 前置: Warp 安裝 (macOS / Linux / Windows)

W6: Writing Secure AI Code - 時間:4-6 hr | 可行性: ★★★ 可做但偏深 - 前置: Semgrep CLI + 基礎 web security 知識

W7: Code Review Reqs - 時間:3-5 hr | 可行性: ★★★★★ 可做 (需 GitHub PR 經驗) - 前置: GitHub account + Graphite 試用

W8: Multi-stack Web App Builds - 時間:6-12 hr | 可行性: ★★★★★ 完全可做 - 前置: v0 / Lovable / Bolt 任一 + Vercel deploy

W9: 無原始 assignment - 建議自製: 用 Sentry / Better Stack / Vercel Analytics 接 side project 當練習,自選 observability tool

Stage 4 完成後, 每個 assignment 會有獨立的中文化任務說明文件 + step-by-step solution outline (不給 spoiler 答案, 但會引導你怎麼開始)。

104.1 推薦做題順序 (給自學者)

104.1.1 短時間 (2 週內) 讀完課

做 W1 + W2 + W4 — 涵蓋 LLM 基礎 / agent / Claude Code, CP 值最高。

104.1.2 中時間 (1 個月) 讀完課

做 W1 + W2 + W3 + W4 + W8 — 多了 MCP server + UI 生成, 可以做出可展示的 side project。

104.1.3 完整跟課（10 週）

9 個全做,並用 W1-W9 的工具棧整合到 final project (自選一個你想做的東西, 從 prompting → agent → MCP → testing → deploy → observability 全跑一輪)。

104.2 Final Project 建議

CS146S 的 final project 占 80% 評分,但對自學者來說沒有 mentor 指導。建議改成:

1. 挑一個你 **vibe coding** 想做的 **side project** (例: 個人 dashboard、學習工具、生產力 app)
2. 用 **W1-W8** 的工具棧做完整實作: W1 prompt → W2 MCP → W3 PRD → W4 Claude Code → W5 Warp → W6 security check → W7 code review → W8 UI deploy
3. 公開放上 **GitHub** + 寫一篇「我用 CS146S 學到的東西做了什麼」blog post, 當作學習成果

這比交一份 Stanford 教授不會看的作業更有 ROI。